

AutoScholar

The First Autonomous Scientific Research Engine

Full System Documentation
& Engineering Specification

Version 3.0.0

158 pages • **17 chapters** • **3 appendices** • **6,600+ lines**

Euler Gauss Turing Newton Fisher Governor Popper

autoscholar.ai

March 2026

Abstract

AutoScholar is an autonomous scientific research platform that generates publication-quality academic papers in quantitative finance, econometrics, statistics, and machine learning without human intervention. Built on a multi-agent architecture orchestrated by Claude Opus 4.6 with a one-million-token context window, the system employs five specialized sub-agents—Euler (orchestration & writing), Gauss (literature gap analysis), Turing (data collection), Newton (dataset preparation), and Fisher (empirical analysis)—supervised by a rule-based Governor that monitors pipeline health and intervenes on failures.

The platform integrates over 200 Claude-callable tools spanning financial data (230+ FMP endpoints, EODHD, real options chains), macroeconomic data (World Bank, IMF, OECD, FRED), academic databases (Semantic Scholar, OpenAlex, arXiv, CrossRef), web intelligence (Tavily, Firecrawl, Exa), prediction markets (Polymarket), and a sandboxed Python execution environment with access to 17 scientific computing libraries. Papers are compiled to LaTeX PDFs with iterative error repair, validated by an AI peer reviewer (Popper), assigned persistent identifiers (ASIDs), and published to an autonomous journal—all on a configurable one-hour cadence with checkpoint/resume crash recovery.

This document provides a comprehensive 150+ page engineering specification covering the complete system architecture, all agent internals, the full paper generation pipeline, data and storage layers, the modeling and scientific computation engine, failure handling mechanisms, performance characteristics, design patterns, identified weaknesses, and proposed improvements for the next generation of the platform.

Contents

List of Acronyms 19

1 System Overview 20
- 1.1 Introduction . . . 20
- 1.2 Motivation and Vision . . . 20
- 1.3 Technology Stack . . . 21
- 1.4 Scale and Metrics . . . 21
- 1.5 Repository Structure . . . 22
- 1.6 Key Design Decisions . . . 24
- 1.7 Document Organization . . . 24

2 System Architecture 26
- 2.1 High-Level Architecture . . . 26
- 2.2 Component Map . . . 27
  - 2.2.1 Entry Points and Configuration . . . 27
  - 2.2.2 Route Handlers . . . 27
  - 2.2.3 Service Layer . . . 27
  - 2.2.4 Storage Layer . . . 28
  - 2.2.5 Shared Types and Schema . . . 29
- 2.3 Data Flow Architecture . . . 29
  - 2.3.1 Interactive Chat Flow . . . 29
  - 2.3.2 Auto-Paper Pipeline Flow . . . 30
- 2.4 Execution Model . . . 30
  - 2.4.1 Streaming SSE Protocol . . . 30
  - 2.4.2 Token Management . . . 30
  - 2.4.3 Concurrency Model . . . 31
- 2.5 Dependency Graph . . . 31
- 2.6 Security Architecture . . . 32
- 2.7 Configuration Management . . . 32
- 2.8 Deployment Model . . . 32

3 Agentic Architecture 33
- 3.1 Overview of the Multi-Agent System . . . 33
- 3.2 Euler — The Global Orchestrator . . . 33
  - 3.2.1 Role and Responsibilities . . . 33
  - 3.2.2 Decision-Making Process . . . 34

3.2.3 State Transitions . . . 34
3.2.4 Writing Mode . . . 35
3.2.5 Tool Access . . . 35
3.3 Gauss — The Gap Finder . . . 35
3.3.1 Role and Responsibilities . . . 35
3.3.2 The 7-Step Gap Discovery Pipeline . . . 35
3.3.3 Robust JSON Parsing . . . 36
3.3.4 External Tools Used . . . 37
3.3.5 Failure Modes . . . 37
3.4 Turing — The Data Collector . . . 37
3.4.1 Role and Responsibilities . . . 37
3.4.2 Data Source Hierarchy . . . 37
3.4.3 Data Collection Methods . . . 38
3.4.4 Data Validation . . . 38
3.4.5 Failure Modes . . . 38
3.5 Newton — The Data Preparer . . . 39
3.5.1 Role and Responsibilities . . . 39
3.5.2 Data Preparation Pipeline . . . 39
3.5.3 Automatic Error Patching . . . 39
3.5.4 Failure Modes . . . 40
3.6 Fisher — The Analysis Engine . . . 40
3.6.1 Role and Responsibilities . . . 40
3.6.2 Methodology Tracks . . . 40
3.6.3 Pre-Built Analysis Services . . . 41
3.6.4 Figure Generation . . . 41
3.6.5 Failure Modes . . . 42
3.7 Governor — The Pipeline Supervisor . . . 42
3.7.1 Architecture . . . 42
3.7.2 State Model . . . 42
3.7.3 Decision Rules . . . 43
3.7.4 Prompt Profiles . . . 43
3.7.5 Publication Verdict . . . 44
3.7.6 Decision Logging . . . 44
3.8 Popper — The Peer Reviewer . . . 44
3.8.1 Role and Responsibilities . . . 44
3.8.2 Review Process . . . 44
3.8.3 Configuration . . . 45
3.8.4 Failure Modes . . . 45
3.9 Agent Interaction Patterns . . . 45
3.9.1 Sequential Handoff . . . 45
3.9.2 Governor Intervention . . . 45
3.9.3 Popper Feedback Loop . . . 45

4 Paper Generation Pipeline 47
4.1 Pipeline Overview . . . 47
4.2 Phase 1: Trigger and Initialization . . . 47
4.2.1 Trigger Mechanisms . . . 47

4.2.2 Prompt Selection . . . 47
4.2.3 Surprise Prompt Generation . . . 48
4.2.4 Pipeline Run Initialization . . . 48
4.3 Phase 2: Research Prompt Enhancement . . . 49
4.3.1 Research Detection . . . 49
4.3.2 Methodology Detection . . . 49
4.3.3 Prompt Upgrading . . . 49
4.4 Phase 3: Literature Review (Gauss Phase) . . . 49
4.4.1 System Prompt Activation . . . 49
4.4.2 Execution Sequence . . . 50
4.4.3 Outputs . . . 50
4.5 Phase 4: Data Acquisition (Turing Phase) . . . 50
4.5.1 Data Strategy . . . 50
4.5.2 Typical Tool Sequences . . . 50
4.5.3 Data Provenance . . . 51
4.6 Phase 5: Data Engineering (Newton Phase) . . . 51
4.6.1 Execution . . . 51
4.6.2 Script Management . . . 51
4.7 Phase 6: Empirical Analysis (Fisher Phase) . . . 51
4.7.1 Model Selection Logic . . . 51
4.7.2 Robustness Checks . . . 52
4.7.3 Figure Generation . . . 52
4.8 Phase 7: Paper Writing (Euler Write Phase) . . . 52
4.8.1 Section-by-Section Generation . . . 52
4.8.2 Writing Style Enforcement . . . 53
4.8.3 Section Rejection Tracking . . . 53
4.9 Phase 8: LaTeX Compilation . . . 54
4.9.1 Section Assembly . . . 54
4.9.2 Template Selection . . . 54
4.9.3 Compilation Pipeline . . . 55
4.9.4 LaTeX Sanitization . . . 55
4.9.5 Iterative Repair . . . 55
4.10 Phase 9: Packaging . . . 56
4.11 Phase 10: Validation (Popper Phase) . . . 56
4.11.1 Review Trigger . . . 57
4.11.2 Publication Decision . . . 57
4.12 Phase 11: Publication . . . 57
4.12.1 ASID Assignment . . . 57
4.12.2 Storage . . . 57
4.12.3 Domain Classification . . . 58
4.13 Checkpoint and Resume System . . . 58
4.13.1 Motivation . . . 58
4.13.2 Checkpoint Structure . . . 58
4.13.3 Checkpoint Lifecycle . . . 58
4.14 Pipeline Metrics . . . 59

5 Autonomous Journal System **60**
5.1 What Makes It a Journal . . . 60
5.2 Scheduling Architecture . . . 60
5.2.1 The AutoPaper Scheduler . . . 60
5.2.2 Heartbeat Monitoring . . . 61
5.2.3 Admin Controls . . . 62
5.3 Quality Control . . . 62
5.3.1 Multi-Layer Quality Assurance . . . 62
5.3.2 Quality Metrics . . . 62
5.3.3 Publication Threshold . . . 63
5.4 Publication Mechanics . . . 63
5.4.1 ASID System . . . 63
5.4.2 Paper Storage . . . 64
5.4.3 Repository Browser . . . 64
5.5 Journal Analytics . . . 64
5.6 Continuous Generation Model . . . 65
5.7 Comparison with Traditional Journals . . . 65

6 Data and Storage Architecture **66**
6.1 Overview . . . 66
6.2 External Data Sources . . . 66
6.2.1 Financial Market Data . . . 66
6.2.2 Macroeconomic Data . . . 68
6.2.3 Academic Data Sources . . . 68
6.2.4 Web Intelligence . . . 69
6.2.5 Prediction Markets . . . 69
6.3 Transient Computational Data . . . 70
6.3.1 Python Execution Environment . . . 70
6.3.2 Generated Data Files . . . 71
6.4 Persistent Storage . . . 71
6.4.1 Database Schema . . . 71
6.4.2 Storage Layer Implementation . . . 72
6.4.3 Database Migrations . . . 73
6.5 File System Usage . . . 73
6.6 Data Lifecycle . . . 73

7 Modeling and Scientific Computation Layer **74**
7.1 Overview . . . 74
7.2 Python Execution Architecture . . . 74
7.2.1 The TypeScript–Python Bridge Pattern . . . 74
7.2.2 Custom Python Executor . . . 75
7.3 Dedicated Analysis Services . . . 76
7.3.1 GARCH Volatility Modeling . . . 76
7.3.2 Value at Risk (VaR) . . . 77
7.3.3 Monte Carlo Simulation . . . 77
7.3.4 Portfolio Optimization . . . 78

7.3.5 Risk Metrics . . . 78
7.3.6 Options Pricing . . . 79
7.3.7 Volatility Surface . . . 79
7.4 Scientific Computing Libraries . . . 79
7.4.1 Econometric Modeling with Statsmodels . . . 79
7.4.2 Machine Learning with Scikit-learn . . . 80
7.4.3 Gradient Boosting with XGBoost/LightGBM . . . 80
7.4.4 Convex Optimization with CVXPY . . . 80
7.4.5 Volatility Modeling with ARCH . . . 80
7.5 Figure Generation Pipeline . . . 80
7.5.1 Style Configuration . . . 80
7.5.2 Figure Capture . . . 81
7.5.3 Vision Analysis . . . 81
7.6 Script Management and Repair . . . 81
7.6.1 Script Lifecycle . . . 81
7.6.2 Diff Engine . . . 82
7.6.3 Structured Error Parsing . . . 82

8 Paper Generation Engine 83
8.1 Overview . . . 83
8.2 Text Generation . . . 83
8.2.1 Section-by-Section Writing . . . 83
8.2.2 Citation Handling . . . 84
8.2.3 Figure Integration . . . 84
8.3 LaTeX Templates . . . 84
8.3.1 Common Preamble . . . 84
8.3.2 Article Template . . . 85
8.3.3 Working Paper Template . . . 85
8.3.4 Econometrica Template . . . 85
8.4 Content Sanitization . . . 85
8.4.1 Special Character Escaping . . . 86
8.4.2 Table Auto-Fitting . . . 86
8.4.3 Blank Figure Detection . . . 86
8.5 Compilation Pipeline . . . 86
8.5.1 Binary Discovery . . . 86
8.5.2 Multi-Pass Compilation . . . 87
8.5.3 Error Classification . . . 87
8.5.4 Iterative Repair Loop . . . 87
8.6 Document Export . . . 88
8.6.1 PDF via Puppeteer . . . 88
8.6.2 DOCX via Pandoc . . . 88
8.6.3 HTML Export . . . 88
8.7 Reproducible Project Packaging . . . 89
8.7.1 Directory Structure . . . 89
8.7.2 Automatic Dependency Detection . . . 89
8.7.3 Master Pipeline Script . . . 89

9 Memory and State System **91**
9.1 Overview . . . 91
9.2 Context Window as Working Memory . . . 91
9.2.1 The 1M Token Context . . . 91
9.2.2 Token Budget Management . . . 91
9.2.3 Implications . . . 92
9.3 Session State . . . 92
9.3.1 Conversation Sessions . . . 92
9.3.2 Real-Time State Broadcasting . . . 92
9.3.3 Session Persistence . . . 92
9.4 Pipeline State . . . 93
9.4.1 Governor State . . . 93
9.4.2 Decision Log . . . 93
9.4.3 Checkpoint State . . . 94
9.5 Cross-Paper Memory . . . 94
9.5.1 Current State: No Cross-Paper Memory . . . 94
9.5.2 Implications . . . 94
9.5.3 Implicit Memory via Prompt Bank . . . 95
9.6 Platform State . . . 95
9.6.1 Site Configuration . . . 95
9.6.2 Analytics Accumulation . . . 95
9.6.3 Published Paper Corpus . . . 95
9.7 State Consistency and Recovery . . . 96
9.7.1 Crash Recovery . . . 96
9.7.2 Data Integrity . . . 96

10 Failure Handling and Recovery **97**
10.1 Overview . . . 97
10.2 Claude API Failure Handling . . . 97
10.2.1 Retry with Exponential Backoff . . . 97
10.2.2 Transient Error Categories . . . 98
10.2.3 Token Overflow Prevention . . . 98
10.3 Python Execution Failure Handling . . . 98
10.3.1 Pre-Execution Patching . . . 98
10.3.2 Error Classification . . . 98
10.3.3 Script Repair Loop . . . 99
10.3.4 Retry Budget System . . . 99
10.4 LaTeX Compilation Failure Handling . . . 99
10.4.1 Error Detection . . . 99
10.4.2 Compilation Fallback Strategies . . . 100
10.4.3 Repair Budget . . . 100
10.5 External API Failure Handling . . . 100
10.5.1 Rate Limiting . . . 100
10.5.2 Data Source Fallback Chains . . . 100
10.5.3 Timeout Configuration . . . 101

10.6 Pipeline-Level Failure Handling . . . 101
10.6.1 Heartbeat Monitoring . . . 101
10.6.2 Consecutive Failure Detection . . . 101
10.6.3 Server Restart Recovery . . . 101
10.6.4 Stale Cleanup . . . 102
10.7 LLM Output Failure Handling . . . 102
10.7.1 Malformed JSON . . . 102
10.7.2 Tool Call Failures . . . 102
10.7.3 Infinite Loop Prevention . . . 102
10.8 Error Reporting and Observability . . . 103
10.8.1 Logging . . . 103
10.8.2 SSE Error Events . . . 103
10.8.3 Admin Dashboard . . . 103

11 Performance and Scalability 104
11.1 Overview . . . 104
11.2 Latency Analysis . . . 104
11.2.1 End-to-End Pipeline Latency . . . 104
11.2.2 Claude API Latency . . . 104
11.2.3 Python Execution Latency . . . 105
11.2.4 External API Latency . . . 105
11.3 Throughput . . . 105
11.3.1 Paper Generation Rate . . . 105
11.3.2 Bottlenecks . . . 106
11.4 Memory Usage . . . 106
11.4.1 Node.js Process . . . 106
11.4.2 Python Subprocesses . . . 106
11.4.3 Database . . . 106
11.5 Compute Constraints . . . 106
11.5.1 API Cost Model . . . 106
11.5.2 External API Costs . . . 107
11.6 Scalability Assessment . . . 107
11.6.1 Horizontal Scaling Limitations . . . 107
11.6.2 Vertical Scaling Opportunities . . . 107
11.6.3 Scaling Recommendations . . . 108
11.7 Monitoring and Observability . . . 108
11.7.1 Built-in Metrics . . . 108
11.7.2 Metrics Access . . . 108

12 Design Patterns and Architecture Philosophy 109
12.1 Overview . . . 109
12.2 The Monolithic Streaming Loop . . . 109
12.2.1 Pattern Description . . . 109
12.2.2 Rationale . . . 109
12.2.3 Trade-offs . . . 109

12.3 Prompt-Based Agent Architecture . . . 110
12.3.1 Pattern Description . . . 110
12.3.2 Rationale . . . 110
12.3.3 Trade-offs . . . 110
12.4 The TypeScript–Python Bridge . . . 110
12.4.1 Pattern Description . . . 110
12.4.2 Rationale . . . 110
12.4.3 Trade-offs . . . 111
12.5 The Governor as a Pure Rule Engine . . . 111
12.5.1 Pattern Description . . . 111
12.5.2 Rationale . . . 111
12.5.3 Trade-offs . . . 111
12.6 Tool-Rich Agent Paradigm . . . 111
12.6.1 Pattern Description . . . 111
12.6.2 Rationale . . . 111
12.6.3 Trade-offs . . . 112
12.7 Barrel Re-Export Pattern . . . 112
12.7.1 Pattern Description . . . 112
12.7.2 Rationale . . . 112
12.8 Defensive Sanitization . . . 112
12.8.1 Pattern Description . . . 112
12.8.2 Rationale . . . 113
12.9 Implicit Convention Enforcement . . . 113
12.9.1 Pattern Description . . . 113
12.9.2 Rationale . . . 113
12.9.3 Trade-offs . . . 113
12.10 Design Philosophy Summary . . . 113

13 Critical Evaluation and Weaknesses 115
13.1 Scientific Rigor . . . 115
13.1.1 Identification Strategy . . . 115
13.1.2 Statistical Inference . . . 115
13.1.3 Data Quality . . . 115
13.2 Reproducibility . . . 116
13.2.1 Strengths . . . 116
13.2.2 Weaknesses . . . 116
13.3 Robustness . . . 116
13.3.1 Single Point of Failure: Claude API . . . 116
13.3.2 Monolithic Architecture . . . 116
13.3.3 File System Dependency . . . 117
13.4 Quality Control Limitations . . . 117
13.4.1 Self-Review Bias . . . 117
13.4.2 Low Rejection Threshold . . . 117
13.4.3 No Human Review . . . 117
13.5 Security Concerns . . . 117
13.5.1 Python Sandbox . . . 117

13.5.2 Admin Authentication . . . 118
13.5.3 API Key Storage . . . 118
13.6 Scalability Weaknesses . . . 118
13.7 Content Quality Weaknesses . . . 118
13.7.1 Hallucination Risk . . . 118
13.7.2 Template Repetitiveness . . . 119
13.7.3 Depth vs. Breadth . . . 119
13.8 Ethical Considerations . . . 119

14 Proposed Improvements 120
14.1 AutoScholar v2: Deep Search Architecture . . . 120
14.1.1 Motivation . . . 120
14.1.2 Proposed Architecture . . . 120
14.1.3 Benefits . . . 121
14.2 Journal++ Model . . . 121
14.2.1 Multi-Tier Publication . . . 121
14.2.2 Multi-Pass Review . . . 121
14.2.3 Citation Network . . . 121
14.2.4 Revision Workflow . . . 122
14.3 Better Memory System . . . 122
14.3.1 Cross-Paper Knowledge Base . . . 122
14.3.2 Implementation . . . 122
14.4 True Novelty Detection . . . 122
14.4.1 Current Limitation . . . 123
14.4.2 Proposed System . . . 123
14.4.3 Technical Requirements . . . 123
14.5 Dataset Versioning . . . 123
14.5.1 Problem . . . 123
14.5.2 Proposed System . . . 123
14.6 Reproducibility Layer . . . 124
14.6.1 Enhanced Project Archives . . . 124
14.6.2 Verification Service . . . 124
14.7 Additional Improvements . . . 124
14.7.1 Governor v2: Hybrid Rule-LLM . . . 124
14.7.2 Interactive Revision Mode . . . 124
14.7.3 Multi-Language Support . . . 125
14.7.4 Real-Time Data Integration . . . 125
14.7.5 Performance Optimization . . . 125

15 System Visualizations 126
15.1 Full System Diagram . . . 126
15.2 Pipeline Flow Diagram . . . 127
15.3 Agent Interaction Diagram . . . 127
15.4 Data Flow Diagram . . . 128
15.5 Governor Decision Flow . . . 128
15.6 Storage Schema Diagram . . . 129

15.7 Tool Ecosystem Map 129

16 Frontend Architecture **130**

16.1 Overview 130
16.2 Technology Stack 130
16.3 Application Structure 130
16.3.1 Routing 130
16.3.2 Context Providers 131
16.4 Core Components 131
16.4.1 Research Interface (`pages/home.tsx`) 131
16.4.2 Streaming Answer (`components/streaming-answer.tsx`) 132
16.4.3 Agent Steps (`components/chat/agent-steps.tsx`) 132
16.4.4 Search Bar (`components/chat/search-bar.tsx`) 132
16.4.5 Python Figure Gallery (`components/chat/custom-python-figure.tsx`) 133
16.5 State Management 133
16.5.1 Chat Session Hook (`hooks/useChatSession.ts`) 133
16.5.2 Chat Mutation Hook (`hooks/useChatMutation.ts`) 134
16.5.3 Analytics Hook (`hooks/useAnalytics.ts`) 134
16.6 Document Export 134
16.6.1 Server-Side Export 134
16.6.2 Client-Side Export 134
16.7 Configuration 134
16.8 Build and Development 135
16.8.1 Development 135
16.8.2 Production Build 135

17 Public REST API **136**

17.1 Overview 136
17.2 Authentication 136
17.2.1 API Key Management 136
17.2.2 Authentication Methods 136
17.2.3 Scopes 136
17.2.4 Rate Limiting 137
17.3 Response Format 137
17.4 Endpoints 137
17.4.1 Papers 137
17.4.2 Reviews 138
17.4.3 Reports 138
17.4.4 Search 138
17.4.5 Pipeline 139
17.4.6 Statistics 139
17.4.7 Sessions 139
17.4.8 Prompts 139
17.5 Pagination 140
17.6 Error Codes 140

17.7 Internal API Endpoints . . . 140
17.7.1 Admin Endpoints . . . 141

Appendices **142**

A Complete Tool Catalog **143**
A.1 Financial Tools (Part 1 — Core) . . . 143
A.2 Financial Tools (Part 2 — Market) . . . 144
A.3 Financial Tools (Parts 3–4 — Extended & Advanced) . . . 145
A.4 Academic Tools . . . 145
A.5 Web Tools . . . 145
A.6 Macroeconomic Data Tools . . . 146
A.7 Python Tools . . . 146
A.8 Research Paper Tools . . . 146
A.9 Polymarket Tools . . . 147

B Prompt Bank Specification **148**
B.1 Construction Algorithm . . . 148
B.2 Research Domains . . . 148
B.3 Methodological Approaches . . . 149
B.4 Domain–Method Compatibility Matrix . . . 150
B.5 Topic Examples . . . 150
B.5.1 Market Microstructure Topics . . . 150
B.5.2 ESG & Sustainable Finance Topics . . . 151
B.6 Prompt Format . . . 151

C Database Schema Reference **152**
C.1 User Tables . . . 152
C.1.1 `users` . . . 152
C.1.2 `emailVerificationTokens` . . . 152
C.1.3 `passwordResetTokens` . . . 153
C.2 Content Tables . . . 153
C.2.1 `messages` . . . 153
C.2.2 `sharedReports` . . . 153
C.3 Research Output Tables . . . 153
C.3.1 `publishedPapers` . . . 153
C.3.2 `peerReviews` . . . 154
C.4 Pipeline Management Tables . . . 154
C.4.1 `paperRuns` . . . 154
C.4.2 `governorDecisions` . . . 155
C.4.3 `asidCounters` . . . 155
C.5 Analytics Tables . . . 155
C.5.1 `conversationSessions` . . . 155
C.5.2 `activeUsers` . . . 156
C.5.3 `analyticsMetrics` . . . 156
C.5.4 `requestLogs` . . . 156

C.6 Administrative Tables 156
C.6.1 `siteConfig` 156
C.6.2 `donations` 156
C.6.3 `apiKeys` 156
C.7 Schema Differences: SQLite vs PostgreSQL 157

List of Figures

1.1 Top-level repository structure of the AutoScholar codebase. 23

2.1 High-level system architecture of AutoScholar. 26
2.2 Layered dependency structure. 31

5.1 Pipeline run state machine. 61

15.1 Complete AutoScholar system architecture diagram showing all agents, services, storage, and external integrations. 126
15.2 End-to-end paper generation pipeline with all 10 phases. 127
15.3 Agent interaction patterns showing sequential handoff (blue), governor oversight (red dashed), and Popper review feedback (purple). 127
15.4 Data flow from external sources through processing stages to persistent storage. 128
15.5 Governor decision flow showing error detection, budget checking, and action selection. 128
15.6 Core database schema relationships (simplified, 20+ tables total). 129
15.7 Tool ecosystem organized by category showing the breadth of AutoScholar's capabilities. 129

List of Tables

1.1 Core Technology Stack 21

2.1 Service modules by domain. 28
2.2 SSE event types in the streaming protocol. 30

3.1 Agent roles in the AutoScholar system. 33
3.2 Pre-built quantitative analysis services. 41
3.3 Governor prompt profiles. 43

4.1 Typical pipeline execution metrics. 59

5.1 Scheduler configuration parameters. 61
5.2 Comparison between AutoScholar journal and traditional academic journals. 65

6.1 FMP service modules and coverage. 66
6.2 Academic data sources. 69
6.3 Pre-loaded Python API clients. 70
6.4 Embedded DuckDB databases available in Python. 70

7.1 TypeScript–Python service pairs. 75
7.2 Automatic Python code patches. 76
7.3 Risk and performance metrics. 78

8.1 LaTeX special character handling. 86

10.1 Claude API transient error handling. 98
10.2 Python error classification. 99
10.3 External API timeout configuration. 101

11.1 Pipeline latency breakdown (typical). 104
11.2 Python service execution times. 105
11.3 External API response times. 105
11.4 API cost per paper (Claude Opus 4.6). 107

16.1 Frontend technology stack. 130
16.2 Frontend routes. 131

17.1 API key scopes. 136
17.2 API error codes. 140

A.1 Financial tools — Core (55+ tools). 143

A.2 Financial tools — Market data, options, discovery. 144
A.3 Academic research tools. 145
A.4 Web search, scraping, and extraction tools. 145
A.5 Macroeconomic data tools (all free, no API key). 146
A.6 Python execution and repair tools. 146
A.7 Research paper generation and compilation tools. 146
A.8 Prediction market tools (all public, no auth). 147

B.1 24 research domains. 149
B.2 24 methodological approaches. 149

C.1 `users` table schema. 152
C.2 `messages` table schema. 153
C.3 `sharedReports` table schema. 153
C.4 `publishedPapers` table schema. 153
C.5 `peerReviews` table schema. 154
C.6 `paperRuns` table schema. 155
C.7 `conversationSessions` table schema. 156
C.8 `apiKeys` table schema. 156
C.9 Key differences between SQLite and PostgreSQL schemas. 157

List of Algorithms

1 Euler's Research Pipeline Orchestration 34
2 Governor Decision Engine 43
3 Prompt Selection in the Auto-Paper Scheduler 48
4 LaTeX Compilation Pipeline 55
5 LaTeX Iterative Repair 88
6 Claude API Retry Strategy 97
7 Prompt Bank Construction 148

List of Acronyms

ASID AutoScholar Identifier. 20, 27, 57

LLM Large Language Model. 33

ORM Object-Relational Mapping. 27

RRE Reproducible Research Engine. 56, 89

SPA Single-Page Application. 26

SSE Server-Sent Events. 26

SWDES Structured Web Data Extraction System. 38, 50

System Overview

1.1 Introduction

AutoScholar is an autonomous scientific research engine that generates publication-quality academic papers in quantitative finance, econometrics, statistics, machine learning, and artificial intelligence—entirely without human intervention. The system represents a fundamental departure from traditional AI-assisted writing tools: rather than augmenting a human researcher, AutoScholar *replaces* the entire research pipeline, from literature review through data collection, statistical analysis, and manuscript preparation.

At its core, the platform leverages Claude Opus 4.6—Anthropic's flagship large language model with a one-million-token context window—as the cognitive backbone of a multi-agent architecture. Five specialized sub-agents, each named after a foundational figure in mathematics and science, collaborate through a carefully orchestrated pipeline to produce papers that target the quality standards of top-tier academic journals such as the *Journal of Finance*, *Journal of Financial Economics*, and the *Review of Financial Studies*.

The system operates continuously. On a configurable cadence (default: one paper per hour), AutoScholar autonomously selects a research topic from a bank of over 1,000 pre-constructed prompts spanning 24 research domains and 24 methodological approaches, executes the full research pipeline, validates the output through AI-powered peer review, and publishes the result to its self-hosted autonomous journal. Each paper receives a persistent identifier (the AutoScholar Identifier, or AutoScholar Identifier (ASID)), is stored with full LaTeX source, BibTeX references, Python analysis scripts, and raw data—packaged as a reproducible research archive.

1.2 Motivation and Vision

The academic research process is notoriously slow, expensive, and bottlenecked by human bandwidth. A typical empirical finance paper requires 6–18 months from conception to submission, involving literature review (weeks), data acquisition and cleaning (weeks to months), econometric modeling (months), writing (months), and revision cycles. AutoScholar compresses this timeline to approximately 45–90 minutes per paper by automating every stage.

The vision is not to produce perfect papers—many generated papers will contain limitations that a human reviewer would flag—but to demonstrate that the *entire pipeline* of empirical research can be automated end-to-end, creating a foundation for:

(i) **Rapid hypothesis exploration.** Researchers can use AutoScholar to quickly test whether a research direction is viable before investing months of manual effort.

(ii) **Reproducible research at scale.** Every paper is packaged with its complete computational pipeline, enabling verification and extension.

(iii) **Continuous scientific production.** The autonomous journal model demonstrates that research output can be "always on," analogous to continuous integration in software engineering.

(iv) **Democratization of quantitative research.** By encoding expert-level econometric and statistical knowledge into the system prompt and tool definitions, AutoScholar makes sophisticated quantitative methods accessible to researchers without deep programming expertise.

1.3 Technology Stack

AutoScholar is a full-stack TypeScript/Python application with the following major technology choices:

Table 1.1: Core Technology Stack

Layer	Technologies
AI Engine	Claude Opus 4.6 (1M context), Claude Haiku 4.5 (vision), OpenRouter (400+ models)
Backend	Node.js, Express 4.21, TypeScript 5.6
Frontend	React 18.3, Vite 5, TailwindCSS 3, TanStack React Query, Framer Motion
Database	SQLite (better-sqlite3) / PostgreSQL (Neon Serverless), Drizzle ORM
Python Runtime	Python 3.x sandbox with NumPy, Pandas, SciPy, Statsmodels, Scikit-learn, XGBoost, LightGBM, ARCH, CVXPY, Matplotlib
Document Engine	pdflatex, bibtex, 50+ LaTeX packages; Puppeteer (HTML→PDF); Pandoc (DOCX)
Financial Data	FMP (230+ endpoints), EODHD, Financial Data API (real options)
Macroeconomic Data	World Bank (16,000+ indicators), IMF (DataMapper + SDMX 3.0), OECD (1,475+ dataflows), FRED (840,000+ series)
Academic Data	Semantic Scholar, OpenAlex, arXiv, CrossRef, Google Scholar (SerpAPI)
Web Intelligence	Tavily (5 tools), Firecrawl (7 tools), Exa (2 tools), SerpAPI (11 verticals)
Prediction Markets	Polymarket (Gamma, Data, CLOB APIs)

1.4 Scale and Metrics

The codebase comprises approximately 40,000+ lines of source code across 155+ files, exposing 270+ HTTP API endpoints and 213 Claude-callable tool definitions. Key quantitative characteristics include:

- **Tool definitions:** 213 tools across 7 categories (financial, academic, web, macro, Python, research paper, prediction markets), totaling ∼6,100 lines of JSON schema definitions.

- **System prompt:** 1,798 lines defining the complete research pipeline, sub-agent protocols, data fallback chains, and academic writing standards.

- **Prompt bank:** 1,000+ deterministically generated research prompts spanning 24 domains × 24 methods × 14 topics per domain.

- **Python execution environment:** 17 pre-installed scientific computing libraries, 5 pre-loaded API clients, 4 embedded databases (SDC Platinum IPOs, US Housing via DuckDB), and 8 file export functions.

- **Financial data coverage:** 230+ FMP endpoints, EODHD historical data to IPO date, real-time options with Greeks from Financial Data API, and 840,000+ FRED macroeconomic series.

- **Tool execution depth:** Up to 180 recursive tool calls per auto-paper run (45 for interactive chat).

1.5 Repository Structure

The repository follows a monorepo structure with clear separation between client, server, and shared code:

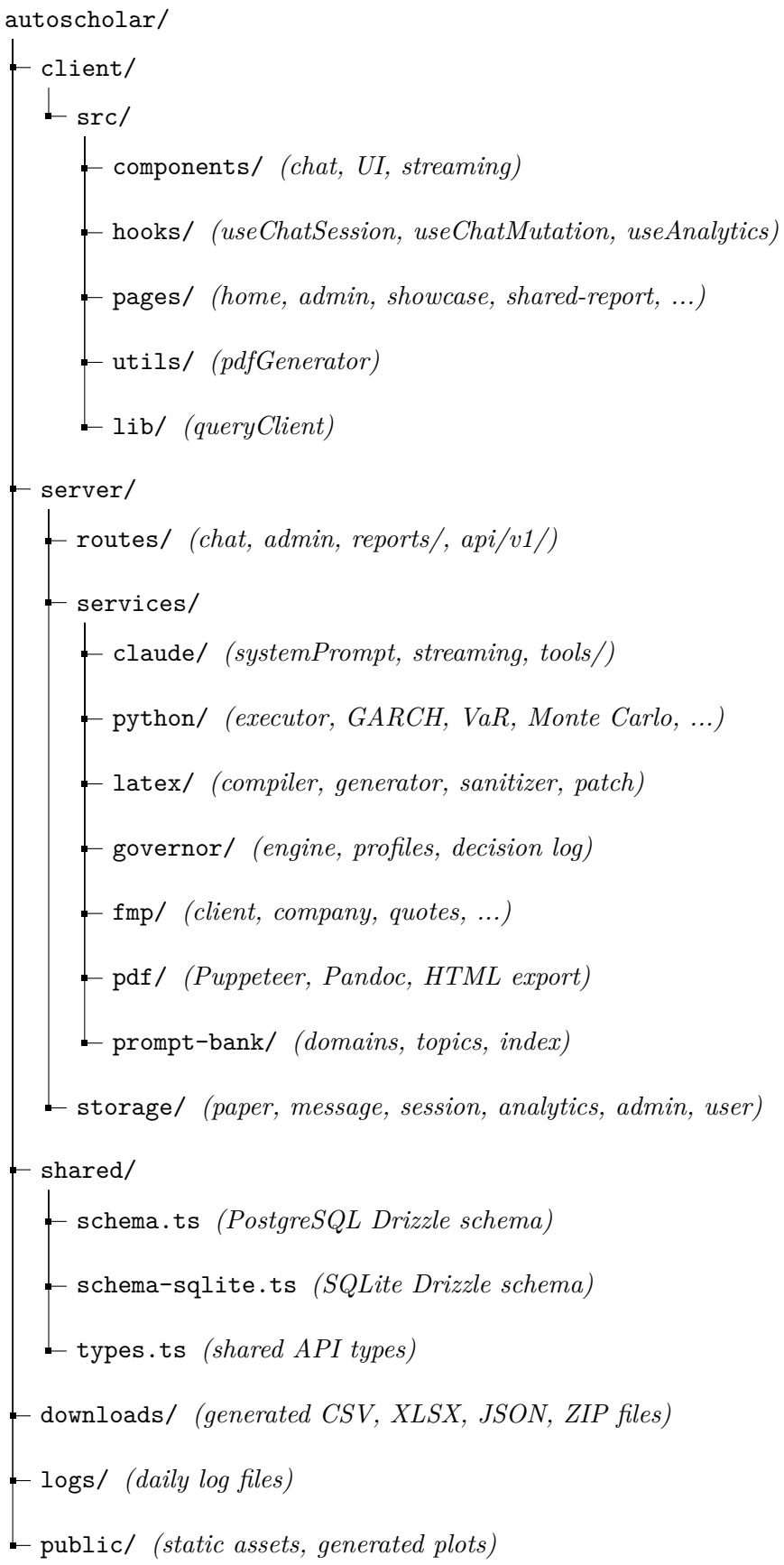


Figure 1.1: Top-level repository structure of the AutoScholar codebase.

1.6 Key Design Decisions

Several architectural decisions shape the character of the system:

Monolithic streaming loop.
Rather than using a message-queue or actor framework, the entire research pipeline executes within a single HTTP streaming connection. Claude issues tool calls, the server executes them synchronously, returns results, and Claude continues—all within one SSE stream that can run for 45–90 minutes. This simplifies state management but creates challenges for resilience (addressed by the checkpoint system).

Claude as the cognitive core.
All reasoning, planning, and writing is performed by Claude Opus 4.6. The sub-agents (Gauss, Turing, Newton, Fisher) are not separate model instances but *conceptual roles* described in the system prompt. Claude switches between these roles based on the stage of the pipeline. The only exception is the Governor, which is a pure rule-based engine with zero LLM calls.

TypeScript–Python bridge pattern.
Computationally intensive operations (GARCH estimation, portfolio optimization, Monte Carlo simulation, custom analysis) run in Python subprocesses spawned by TypeScript wrappers. Each TypeScript module validates inputs, spawns the Python script, parses structured JSON output, and handles errors. This pattern appears in 10+ service pairs across the codebase.

Dual database support.
The system supports both SQLite (for development and self-hosted deployments) and PostgreSQL (for production via Neon Serverless). Drizzle ORM provides a unified query interface, with separate schema definitions for each backend.

Tool-rich agent paradigm.
With 213 tool definitions, AutoScholar takes the "tool-rich" approach to AI agents rather than the "few-tools" approach. This gives Claude maximum flexibility but requires careful prompt engineering to guide tool selection and prevent misuse.

1.7 Document Organization

The remainder of this document is organized as follows:

Chapter 2 presents the complete system architecture, including component diagrams, dependency graphs, and the execution model.

Chapter 3 provides deep dives into each of the five sub-agents plus the Governor and Popper validator.

Chapter 4 reconstructs the full paper generation pipeline from trigger to publication.

Chapter 5 describes the autonomous journal system, including scheduling, quality control, and publication mechanics.

Chapter 6 details the data acquisition, storage, and management infrastructure.

Chapter 7 examines the modeling and scientific computation layer.

Chapter 8 explains the LaTeX paper generation and compilation engine.

Chapter 9 covers the memory, state, and deduplication systems.

Chapter 10 analyzes failure handling, retry logic, and recovery mechanisms.

Chapter 11 discusses performance, scalability, and resource constraints.

Chapter 12 identifies architectural patterns and design philosophy.

Chapter 13 provides a critical evaluation of system weaknesses.

Chapter 14 proposes improvements for the next generation.

Chapter 15 contains system diagrams and pipeline visualizations.

Chapter 16 documents the frontend architecture.

Chapter 17 specifies the public REST API.

System Architecture

2.1 High-Level Architecture

AutoScholar follows a three-tier architecture with a React Single-Page Application (SPA) frontend, a Node.js/Express backend, and a SQLite/PostgreSQL persistence layer. The AI reasoning engine (Claude Opus 4.6) operates as an external service accessed via the Anthropic HTTP API, while Python computational services run as short-lived subprocesses.

The defining architectural characteristic is the **streaming tool-execution loop**: the backend maintains a long-lived Server-Sent Events (SSE) connection with the client while simultaneously conducting a multi-turn conversation with Claude. Each turn may involve Claude issuing one or more tool calls, the server executing them (potentially spawning Python subprocesses, calling external APIs, or querying databases), and returning the results to Claude for the next reasoning step. This loop can execute up to 180 tool calls in a single auto-paper run.

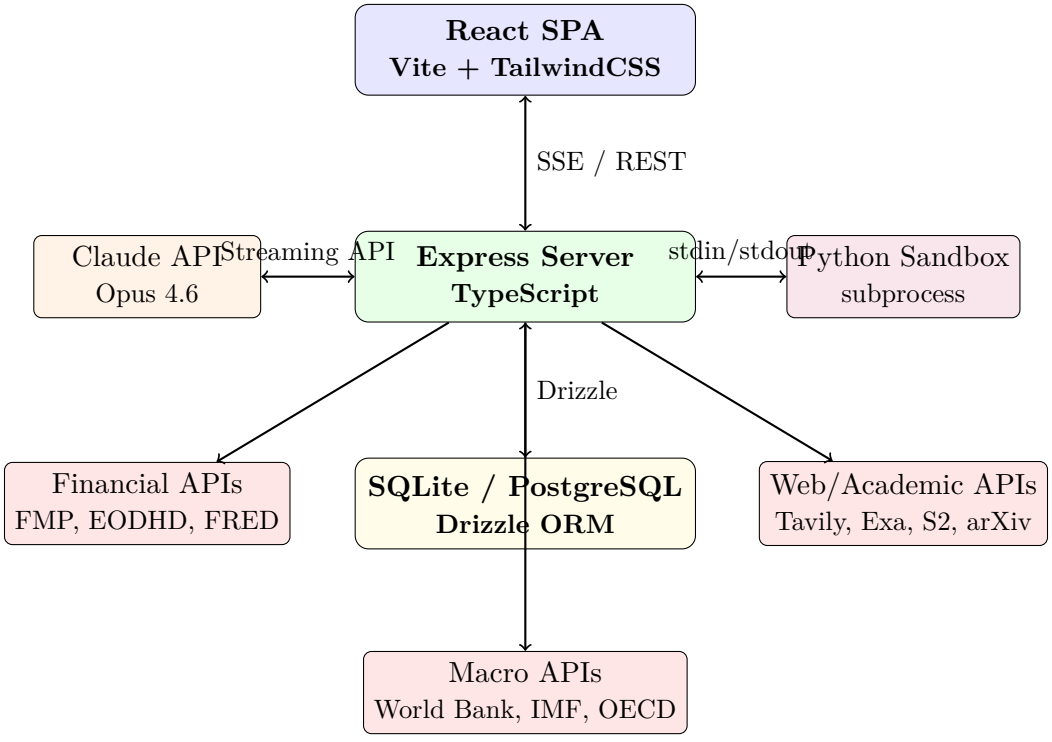


Figure 2.1: High-level system architecture of AutoScholar.

2.2 Component Map

The server-side codebase is organized into five major component groups, each with distinct responsibilities:

2.2.1 Entry Points and Configuration

`server/index.ts` (187 lines) — The application entry point. Initializes Express with CORS, session management (SQLite or PostgreSQL session store), request logging, route registration, and Vite dev server or static file serving. On startup, it initializes the ASID system and launches the auto-paper scheduler. Installs global handlers for `unhandledRejection` and `uncaughtException` to prevent server crashes.

`server/routes.ts` (34 lines) — Central route registration hub. Imports and mounts 8 route modules (chat, auth, admin, analytics, FMP, reports, prompts, public API) and schedules periodic cleanup of stale LaTeX compilations every 30 minutes.

`server/db.ts` (241 lines) — Database abstraction layer supporting three backends: SQLite via better-sqlite3, local PostgreSQL via pg, and Neon Serverless PostgreSQL. For SQLite, performs extensive inline schema migrations (creating 10+ tables, adding columns via safe `ALTER TABLE` statements). Exports a unified Drizzle Object-Relational Mapping (ORM) instance and raw database pool.

`server/storage.ts` (1 line) — Barrel re-export pointing to the composed storage singleton.

2.2.2 Route Handlers

The routing layer is organized into three tiers:

1. **Core routes** — `routes/chat.ts` (1,066 lines), `routes/admin.ts` (675 lines), `routes/auth.ts` (189 lines), `routes/fmp.ts` (317 lines), `routes/prompts.ts` (41 lines).
2. **Report sub-routes** — 9 modules in `routes/reports/` covering document generation, journal papers, DOI/ASID management, figures, peer reviews, shared reports, sessions, platform stats, and the repository browser.
3. **Public API v1** — 9 modules in `routes/api/v1/` providing authenticated access to papers, reviews, reports, stats, pipeline monitoring, search, prompts, sessions, and key management.

The single largest route file is `routes/toolExecutor.ts` at 2,945 lines, containing the massive dispatch function that maps Claude's tool call names to their service implementations.

2.2.3 Service Layer

Services are organized by domain:

Table 2.1: Service modules by domain.

Domain	Key Modules	Lines
Claude/AI	`claude/systemPrompt.ts`, `claude/claudeStreaming.ts`, `claude/tokenManagement.ts`, `claude/tools/` (13 files)	~9,900
Python Computation	`python/customPythonExecutor.py`, `python/customPython.ts`, plus 9 service pairs (GARCH, VaR, Monte Carlo, portfolio, risk, volatility surface, options, data download, plots)	~8,500
LaTeX Engine	`latex/paperGenerator.ts`, `latex/compiler.ts`, `latex/sanitizer.ts`, `latex/latexPatchManager.ts`, `latex/latexLogParser.ts`, `latex/packaging.ts`	~2,870
Governor	`governor/governorEngine.ts`, `governor/types.ts`, `governor/promptProfiles.ts`, `governor/jfWritingStyle.ts`, `governor/academicFigureStyle.ts`	~1,507
Financial Data	`fmp/` (15 files), `eodhd_service.ts`, `financialDataService.ts`	~2,000
Web/Academic	`academicSearchService.ts`, `tavilyService.ts`, `exaService.ts`, `firecrawlService.ts`, `serpapiService.ts`	~2,160
Macro	`worldBankService.ts`, `imfService.ts`, `oecdService.ts`	~320
Pipeline	`autoPaperScheduler.ts`, `researchPaperService.ts`, `peerReviewService.ts`, `literatureGapFinderService.ts`	~2,310
Other	`webDatasetService.ts`, `doiService.ts`, `polymarketService.ts`, `promptBank.ts`, `figureAnalysisService.ts`, `sessionBroadcaster.ts`, `cryptoDonationService.ts`	~2,300

2.2.4 Storage Layer

The storage layer is decomposed into 6 domain-specific modules:

- `storage/user-storage.ts` — User CRUD with cascading session deletion.
- `storage/message-storage.ts` — Chat messages and shared reports.
- `storage/session-storage.ts` — Conversation session lifecycle.

- `storage/paper-storage.ts` — Published papers, pipeline runs, and cost tracking.
- `storage/analytics-storage.ts` — Active users, metrics, and request logs.
- `storage/admin-storage.ts` — Governor state, peer reviews, donations, API keys, site config.

These are composed into a single `storage` object via the barrel file `storage/index.ts`, implementing the 70+ method `IStorage` interface.

2.2.5 Shared Types and Schema

The `shared/` directory contains:

- `schema.ts` (357 lines) — PostgreSQL Drizzle schema defining 20 tables.
- `schema-sqlite.ts` (456 lines) — SQLite Drizzle schema (canonical superset, includes `apiKeys` table).
- `types.ts` (44 lines) — Shared API contract types used by both client and server.

2.3 Data Flow Architecture

2.3.1 Interactive Chat Flow

The interactive chat flow follows this sequence:

1. The client sends a POST to `/api/chat` with the query, conversation history, optional image/data, and session ID.
2. The server validates the request via Zod schemas (`routes/validation.ts`).
3. If the query matches a research paper pattern, it is upgraded via `upgradeResearchPrompt()` using Claude Opus.
4. The system prompt is built via `buildSystemPrompt()` (1,798 lines), incorporating the current date, author profile, and research context.
5. `callClaudeStreamingWithRetry()` initiates the streaming conversation with up to 5 retries on transient failures.
6. For each response chunk:
   - Text deltas are forwarded to the client via SSE.
   - Tool calls trigger `executeToolCall()` (2,945-line dispatch), which routes to the appropriate service.
   - Tool results are sent back to Claude for the next turn.
7. Session data (messages, tool results, figures) is persisted every 8 seconds.
8. On completion, final session state is saved and the SSE stream is closed.

2.3.2 Auto-Paper Pipeline Flow

The autonomous pipeline adds several layers on top of the interactive flow:

1. The scheduler (`autoPaperScheduler.ts`) selects a prompt from the 1,000+ entry prompt bank.
2. A pipeline run is created in the database with status `queued`.
3. The pipeline internally calls the same `/api/chat` endpoint as a client.
4. A GOVERNOR engine monitors tool results and injects directives.
5. On completion, POPPER validates the paper via AI peer review.
6. Papers scoring $\geq 45/100$ are published; below that threshold they are rejected.
7. The run status is updated to `published`, `rejected`, or `failed`.

2.4 Execution Model

2.4.1 Streaming SSE Protocol

All chat communication uses Server-Sent Events with the following event types:

Table 2.2: SSE event types in the streaming protocol.

Event Type	Description
`text`	Incremental text delta from Claude's response.
`sources`	Array of web search results with titles, URLs, and snippets.
`done`	Stream completion signal with final session ID and token usage.
`error`	Error message (transient or fatal).
`python_code`	Python code executed, with script ID and description.
`python_figures`	Base64-encoded matplotlib figures from Python execution.
`python_output`	Captured stdout from Python scripts.
`tool_result`	Structured result from a tool execution (name, summary, data).
`search_query`	Search query dispatched to an external API.
`agent_step`	Pipeline progress indicator (agent name, step, status).
`monte_carlo`	Monte Carlo simulation results (paths, statistics).
`options_pricing`	Black-Scholes pricing results with Greeks.
`download_link`	URL for a generated file (CSV, XLSX, PDF, ZIP).
`section_written`	Research paper section committed (title, word count).
`paper_compiled`	Final paper PDF URL and compilation status.
`retry`	Retry attempt notification (attempt number, delay).

2.4.2 Token Management

The system operates within Claude's 1M-token context window using several strategies:

Token estimation. A 4-characters-per-token heuristic (`estimateTokens()`) provides fast approximate counts without tokenizer overhead.

Tool result truncation. Results exceeding 15,000 characters are truncated with a notice, applied selectively to prevent context overflow from large API responses.

History summarization. When estimated message tokens exceed the 500K safety threshold, the system calls Claude to compress older conversation history into a summary, preserving key findings while freeing context space.

Cost tracking. Real-time cost calculation using model-specific pricing (Opus 4.6: \$5/\$25 per MTok for input/output) enables budget monitoring.

2.4.3 Concurrency Model

The Express server runs in a single Node.js event loop. Concurrency is achieved through:

- **Async I/O:** All external API calls use `async/await` with axios, allowing multiple concurrent HTTP requests.
- **Python subprocesses:** Computational tasks spawn child processes via `child_process.spawn`, running in parallel with the Node.js event loop.
- **Pipeline parallelism:** The auto-paper scheduler supports up to 3 concurrent paper generations via the `maxParallel` configuration parameter.
- **Parallel search:** Academic search functions run all 5 sources (Semantic Scholar, OpenAlex, arXiv, CrossRef, Google Scholar) concurrently via `Promise.allSettled`.

2.5 Dependency Graph

The internal dependency structure follows a layered pattern:

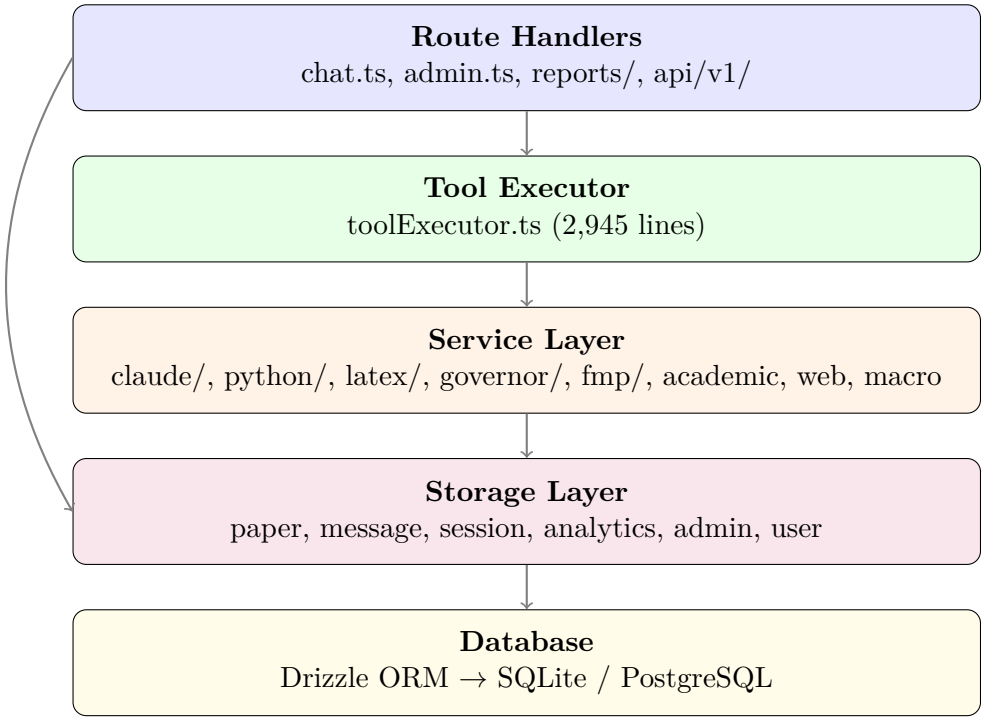


Figure 2.2: Layered dependency structure.

2.6 Security Architecture

Security is implemented at multiple levels:

Authentication. Session-based authentication with SHA-256 + salt password hashing. Admin access requires a separate password. API keys use SHA-256 hashing with raw key visible only at creation.

API key obfuscation. User-provided Anthropic API keys are stored using XOR obfuscation with the session secret, preventing plain-text storage in the database.

Rate limiting. Per-key in-memory rate limiting for the public API (default 100 requests/hour) with automatic cleanup every 10 minutes.

Python sandboxing. Python execution runs in subprocess isolation with JSON stdin/stdout protocol, no filesystem write access outside designated directories.

Input validation. Zod schemas validate all request bodies. Tool inputs are schema-validated by Claude's tool-use protocol.

CORS. Origin-restricted CORS allowing only `localhost` and `autoscholar.ai` domains.

2.7 Configuration Management

System configuration uses a three-tier approach:

1. **Environment variables** (`.env`) — API keys, database URLs, passwords, feature flags.
2. **Database configuration** (`site_config` table) — Runtime-adjustable settings (scheduler cadence, Popper threshold, custom system prompt overrides).
3. **Code constants** — Model selection (`streamingConfig.ts`), token limits, retry parameters, prompt profiles (`governor/promptProfiles.ts`).

2.8 Deployment Model

AutoScholar supports two deployment modes:

Development. Vite dev server with HMR for the frontend, tsx for TypeScript server execution, SQLite for zero-configuration database. All runs locally with `npm run dev`.

Production. Vite builds the frontend to `dist/public/`, esbuild compiles the server to `dist/index.js`, and the app runs as a single Node.js process serving both the API and static files. Production typically uses Neon Serverless PostgreSQL.

The CI pipeline (`.github/workflows/ci.yml`) runs linting, type checking, and tests on push.

Agentic Architecture

3.1 Overview of the Multi-Agent System

AutoScholar employs a *prompt-based multi-agent architecture* where a single Large Language Model (LLM) instance (Claude Opus 4.6) assumes different roles throughout the research pipeline. Unlike frameworks where each agent is a separate model instance with its own context, AutoScholar's agents are conceptual personas defined in the system prompt (`server/services/claude/systemPrompt.ts`, 1,798 lines). Claude switches between these personas based on the current pipeline stage, guided by explicit instructions in the system prompt and reinforced by Governor directives.

The architecture comprises seven distinct agent roles:

Table 3.1: Agent roles in the AutoScholar system.

Agent	Named After	Role
Euler	Leonhard Euler	Global orchestrator, academic writer, LaTeX compiler
Gauss	Carl Friedrich Gauss	Literature gap analysis and research question formulation
Turing	Alan Turing	Data collection from APIs, databases, and web sources
Newton	Isaac Newton	Dataset preparation, cleaning, feature engineering
Fisher	Ronald Fisher	Empirical analysis, econometric modeling, figure generation
Governor	—	Rule-based pipeline supervisor (no LLM)
Popper	Karl Popper	AI-powered peer review and quality validation

3.2 Euler — The Global Orchestrator

3.2.1 Role and Responsibilities

Euler is the master agent that coordinates the entire research pipeline. It is the "default mode" of Claude throughout the session and has three primary responsibilities:

1. **Pipeline orchestration.** Euler decides when to transition between pipeline phases (literature review → data collection → analysis → writing → compilation).

2. **Academic writing.** EULER authors all paper sections in LaTeX, following the Journal of Finance writing style directive (282 lines of style instructions).

3. **LaTeX compilation.** EULER manages the paper compilation process, including section assembly, BibTeX generation, and iterative error repair.

3.2.2 Decision-Making Process

EULER's decision-making is governed by the system prompt's 7-step research pipeline:

Algorithm 1: Euler's Research Pipeline Orchestration

Input: Research prompt p, available tools $\mathcal{T}$
Output: Compiled PDF paper, project ZIP
Step 1: Gap Analysis (invoke GAUSS);
Search academic databases for existing literature;
Identify research gaps and formulate hypotheses;
Step 2: Data Collection (invoke TURING);
Query financial APIs, macroeconomic databases, web sources;
Download datasets to the Python environment;
Step 3: Data Preparation (invoke NEWTON);
Clean, merge, and transform raw datasets;
Construct derived variables and panel structures;
Step 4: Empirical Analysis (invoke FISHER);
Estimate models, run robustness checks;
Generate publication-quality figures and tables;
Step 5: Paper Writing (invoke EULER writing mode);
Write each section using `write_paper_section` tool;
Target 60–90 pages of content;
Step 6: Compilation (invoke EULER compilation mode);
Compile LaTeX via `compile_research_paper`;
If errors, enter repair loop (read → patch → recompile);
Step 7: Packaging;
Create reproducible project ZIP via `compile_research_paper`;

3.2.3 State Transitions

EULER does not maintain explicit state—its "state" is the conversation history and the accumulated tool results within the 1M-token context window. State transitions are implicit: when EULER finishes analyzing literature, it naturally transitions to data collection by invoking TURING-associated tools.

The system prompt provides explicit ordering constraints:

> *"The 7-step pipeline is MANDATORY. You cannot write a paper section before collecting data, and you cannot collect data before identifying what data you need through gap analysis."*

3.2.4 Writing Mode

When EULER enters writing mode, it follows the Journal of Finance style directive (`governor/jfWritingStyle.ts`, 282 lines), which includes:

- Prose identity guidance from editors Campbell Harvey, John Cochrane, Andrei Shleifer, Lubos Pastor, and Craig Holden.
- Official JF formatting rules (100-word abstract limit, Roman numeral tables, unnumbered introduction).
- A banned language registry with 100+ banned words/phrases and tier-based restrictions.
- Section-by-section writing conventions.
- Advanced prose craft techniques (emphatic stress, topic-comment structure).
- AI detection countermeasures (lexical diversity, syntactic variation).
- An 8-pass verification checklist.

3.2.5 Tool Access

EULER has access to all 213 tools, but primarily uses:

- `write_paper_section` — Write individual paper sections in LaTeX.
- `compile_research_paper` — Assemble sections and compile to PDF.
- `read_latex_source` / `inspect_latex_errors` / `patch_latex` / `recompile_latex` — The 4-step LaTeX repair loop.
- `generate_research_paper` — Direct single-call paper generation (legacy path).

3.3 Gauss — The Gap Finder

3.3.1 Role and Responsibilities

GAUSS is the literature discovery and gap analysis agent, responsible for identifying what research has been done and—critically—what has *not* been done. It is implemented as both a conceptual role in the system prompt and a standalone service (`literatureGapFinderService.ts`, 811 lines).

3.3.2 The 7-Step Gap Discovery Pipeline

GAUSS implements a sophisticated multi-stage pipeline:

Step 1: Search Strategy Planning. Claude generates 7 diverse search queries from different angles for the target topic. This ensures broad coverage rather than narrow keyword matching.

Step 2: Multi-Source Parallel Search. Queries are dispatched simultaneously to 4 academic databases:

- **Semantic Scholar** — AI-powered academic search with citation data, retry with exponential backoff on rate limits.
- **OpenAlex** — Open bibliographic database with 200M+ works.
- **arXiv** — Preprint server for cutting-edge research.
- **CrossRef** — DOI registration agency with 130M+ records.

Results are standardized into `AcademicPaper` objects with auto-generated BibTeX.

Step 3: Citation Network Expansion. For high-impact papers (>50 citations), GAUSS retrieves their references and citing papers via OpenAlex and CrossRef, expanding the literature landscape.

Step 4: Scientific Structure Extraction. Claude analyzes each paper to extract:

- Methods used (econometric, statistical, ML)
- Datasets employed
- Key findings
- Stated limitations
- Temporal coverage

Step 5: Literature Synthesis. Claude clusters papers into thematic groups and identifies:

- Consensus findings
- Contradictions between papers
- Methodological trends
- Temporal evolution of the field

Step 6: Gap Detection. Six specialized gap detection engines run in parallel:

1. **Method gaps** — Methods used in adjacent fields but not applied to this topic.
2. **Dataset gaps** — Available datasets that have not been exploited.
3. **Temporal gaps** — Time periods or recent data not yet studied.
4. **Contradiction gaps** — Conflicting findings that need resolution.
5. **Limitation gaps** — Explicitly stated limitations in existing papers.
6. **Cross-disciplinary gaps** — Insights from other fields applicable here.

Step 7: Gap Validation and Scoring. Each identified gap is scored for novelty, feasibility, and impact, producing a ranked list of research opportunities.

3.3.3 Robust JSON Parsing

A critical implementation detail is the 6-strategy JSON parsing system in `parseJsonResponse()`. Since LLM outputs frequently contain malformed JSON (trailing commas, unescaped quotes, markdown code fences), the parser attempts:

1. Direct `JSON.parse`.
2. Strip markdown code fences and re-parse.
3. Extract JSON from mixed text using regex.
4. Fix common issues (trailing commas, single quotes).
5. Parse partial JSON up to the last valid closing bracket.
6. Return a fallback structure with the raw text.

3.3.4 External Tools Used

- `academic_search_multi` — Parallel search across all 5 academic databases.
- `literature_gap_finder` — The full 7-step pipeline.
- `semantic_scholar_search`, `arxiv_search`, `crossref_lookup`, `openalex_search` — Individual source queries.

3.3.5 Failure Modes

- **Rate limiting:** Semantic Scholar frequently returns 429s; the system uses exponential backoff (up to 3 retries with 1s, 2s, 4s delays).
- **Sparse results:** For niche topics, some databases may return zero results. The multi-source approach ensures at least some results are found.
- **Malformed LLM output:** The 6-strategy JSON parser handles most cases, but extreme malformation can produce degraded gap analysis.

3.4 Turing — The Data Collector

3.4.1 Role and Responsibilities

TURING is responsible for acquiring all raw data needed for the research paper. This encompasses financial market data, macroeconomic indicators, academic datasets, and web-sourced information.

3.4.2 Data Source Hierarchy

The system prompt defines a strict data source hierarchy with fallback chains:

1. **Primary: FMP API** (230+ endpoints) — Company fundamentals, market quotes, historical prices, technical indicators, SEC filings, insider trading, ESG scores, earnings data, analyst estimates, and more.
2. **Secondary: EODHD** — Extended historical data back to IPO date, intraday bars, US options chains with Greeks.
3. **Tertiary: FRED** — 840,000+ macroeconomic time series with unit transformations.
4. **Macro: World Bank / IMF / OECD** — Free, keyless access to 16,000+ development indicators, WEO data, and 1,475+ OECD dataflows.

5. **Web: Tavily / Firecrawl / Exa** — When structured APIs lack the required data, web search and extraction fill the gap.
6. **Prediction Markets: Polymarket** — Implied probabilities for event-driven research.

3.4.3 Data Collection Methods

TURING uses three classes of data collection tools:

Direct API tools. 170+ tools that call specific API endpoints (e.g., `get_income_statement`, `get_wb_indicator_data`, `get_fred_series`). Each returns structured JSON.

Python data clients. 5 pre-loaded Python clients (`fmp_client.py`, `fred_client.py`, `eodhd_client.py`, `macro_clients.py`) enable DataFrame-native data acquisition directly in analysis scripts.

Web data extraction. The Structured Web Data Extraction System (SWDES) (`webDatasetService.`
1,431 lines) provides autonomous large-scale web data collection:

1. Claude Haiku plans search queries and extraction schemas.
2. Parallel multi-source search across Firecrawl, Tavily, and Exa.
3. 6-strategy extraction pipeline (schema-based, portal crawling, batch scraping, download links, fallback).
4. Schema enforcement and deduplication via Claude Haiku.
5. Statistical validation and completeness checks.
6. CSV/JSON output with metadata sidecar.

Capable of collecting up to 100,000 rows from 25+ sources in a single call.

3.4.4 Data Validation

TURING validates data at collection time:

- API response schema validation.
- Null/NaN sanitization (especially for EODHD options data).
- Ticker normalization (e.g., "AAPL" → "AAPL.US" for EODHD).
- Date format handling (Unix timestamps, ISO strings, datetime objects).
- Result size limiting to prevent context overflow (e.g., historical prices capped at 504 points).

3.4.5 Failure Modes

- **API key exhaustion:** FMP has daily request limits; the system has no automatic quota management.
- **Data gaps:** Some tickers or time periods may have missing data; the system relies on Claude to detect and handle gaps.

- **Timeout:** External API calls use 15–60s timeouts; slow services may cause tool execution failures.

3.5 Newton — The Data Preparer

3.5.1 Role and Responsibilities

NEWTON transforms raw data into analysis-ready datasets. This involves cleaning, merging, feature construction, and quality validation.

3.5.2 Data Preparation Pipeline

NEWTON operates primarily through the `execute_custom_python_analysis` tool, running Python code that:

1. **Cleans raw data.** Handles missing values (forward fill, interpolation, or explicit NaN treatment based on data type), removes duplicates, and corrects data types.
2. **Merges datasets.** Aligns time series by date, merges cross-sectional data by identifiers, and handles frequency mismatches (e.g., daily prices with quarterly fundamentals).
3. **Constructs derived variables.** Calculates returns (simple, log, excess), volatility measures, momentum indicators, financial ratios, and interaction terms.
4. **Builds panel structures.** Constructs entity × time panels with proper indexing for fixed-effects regression.
5. **Validates quality.** Checks for:
   - Stationarity (ADF/KPSS tests)
   - Multicollinearity (VIF)
   - Outliers (winsorization at 1%/99%)
   - Missing data patterns (MCAR/MAR/MNAR)
   - Sample size adequacy

3.5.3 Automatic Error Patching

The `customPython.ts` module applies several automatic fixes to NEWTON's code before execution:

- **Deprecated pandas frequencies:** Replaces `'M'` with `'ME'`, `'Q'` with `'QE'`, etc.
- **NumPy deprecations:** Replaces `np.trapz` with `np.trapezoid`, removes `np.math` references.
- **Scikit-learn API changes:** Renames `n_iter` to `max_iter` for t-SNE.
- **Matplotlib conflicts:** Patches `tight_layout` to handle colorbar conflicts.
- **Data fabrication detection:** Flags scripts that generate random data without API calls.

3.5.4 Failure Modes

- **Memory errors:** Large datasets can exceed the Python subprocess memory limit.
- **Key errors:** Missing columns due to API response schema changes.
- **Type mismatches:** Mixed types in columns causing downstream computation failures.

3.6 Fisher — The Analysis Engine

3.6.1 Role and Responsibilities

Fisher conducts all empirical analysis: econometric estimation, statistical inference, machine learning modeling, and figure generation. It is the most computationally intensive agent, heavily relying on the Python execution environment.

3.6.2 Methodology Tracks

The system prompt defines four methodology tracks, each with specific model inventories:

Track 1: Econometrics

- OLS, WLS, GLS, 2SLS, GMM, panel fixed/random effects
- Time series: ARIMA, VAR, VECM, GARCH(p,q), DCC-GARCH
- Causal inference: difference-in-differences, regression discontinuity, instrumental variables, propensity score matching
- High-frequency: realized volatility, Amihud illiquidity, Kyle's λ

Track 2: Statistics

- Hypothesis testing: t-tests, F-tests, χ^2 tests, Kolmogorov-Smirnov, Mann-Whitney
- Non-parametric: kernel density estimation, bootstrap confidence intervals
- Bayesian: MCMC, variational inference, Bayesian model averaging
- Distribution: quantile regression, extreme value theory, copulas

Track 3: Machine Learning

- Supervised: random forests, gradient boosting (XGBoost, LightGBM), SVMs, elastic net
- Unsupervised: K-means, hierarchical clustering, PCA, t-SNE, UMAP
- Validation: cross-validation, grid/random search, learning curves
- Interpretability: SHAP values, partial dependence plots, feature importance

Track 4: AI/Deep Learning

- Architectures: LSTM, GRU, Transformer, attention mechanisms
- NLP: sentiment analysis, named entity recognition, topic modeling
- Transfer learning: pre-trained embeddings, fine-tuning
- Evaluation: ablation studies, attention visualization

3.6.3 Pre-Built Analysis Services

In addition to custom Python execution, FISHER has access to 7 dedicated analysis services:

Table 3.2: Pre-built quantitative analysis services.

Service	Capabilities	Lines
GARCH	GARCH(p,q) estimation, volatility forecasting, persistence metrics	506
VaR	Historical, parametric, Monte Carlo, and CVaR/Expected Shortfall	555
Monte Carlo	Investment projections with configurable simulations and horizons	453
Portfolio Optimizer	Markowitz MVO, efficient frontier, max Sharpe, min volatility	672
Risk Metrics	20+ metrics: Sharpe, Sortino, Calmar, beta, alpha, drawdown	795
Options Pricing	Black-Scholes, Greeks, implied volatility (Newton-Raphson)	589
Volatility Surface	3D IV surface construction with cubic interpolation	520

3.6.4 Figure Generation

FISHER generates publication-quality figures following the academic figure style directive (`governor/academicFigureStyle.ts`, 429 lines):

- **Design philosophy:** Monochrome-first, print-first, Tufte data-ink ratio maximization.
- **Typography:** 9pt serif axis labels, 0.5pt spines, inward ticks.
- **Color policy:** Black > grayscale > Paul Tol/Okabe-Ito palettes. Rainbow/jet palettes are explicitly prohibited.
- **Chart conventions:** 16 chart types with specific formatting rules (time-series, event-study CAR, coefficient plots, bar charts, heatmaps, impulse response, RD plots, DiD, fan charts, survival curves).
- **Export:** 300 DPI PNG for inclusion, PDF with TrueType fonts for publication.
- **Watermark:** AutoScholar watermark added to all figures.

The matplotlib configuration (`figures/matplotlib_config.py`, 79 lines) sets serif fonts, an 8-color muted palette, inward ticks, minimal chrome, and print-safe grid styling. Blank figure detection (`figures/figure_capture.py`, 104 lines) uses two-stage analysis: axes artist inspection followed by pixel-level variance measurement.

3.6.5 Failure Modes

- **Convergence failures:** GARCH and maximum likelihood models may fail to converge with insufficient or degenerate data.
- **Singular matrices:** Covariance matrices with perfectly correlated or constant variables.
- **Memory errors:** Large Monte Carlo simulations or high-dimensional panel datasets.
- **Module import failures:** Missing Python packages (mitigated by the `install_python_package` tool).

3.7 Governor — The Pipeline Supervisor

3.7.1 Architecture

The GOVERNOR is unique among AUTOSCHOLAR's agents: it is a **pure rule-based system** with zero LLM calls and zero database access. Implemented in `governor/governorEngine.ts` (472 lines), it observes tool results in real-time and issues corrective actions when problems are detected.

3.7.2 State Model

The GOVERNOR maintains a typed state object (`governor/types.ts`, 164 lines):

Listing 3.1: Governor state model (simplified).

```
interface AppState {
  runId: string;
  stage: PipelineStage;  // gauss | turing | newton | fisher |
     euler_write | euler_compile | popper | complete
  profile: string;        // active prompt profile
  pythonErrors: number;
  latexErrors: number;
  dataIssues: number;
  retryBudgets: {
    python: number;    // default: 3
    latex: number;     // default: 3
    data: number;      // default: 2
  };
  confidence: number;    // 0.0 - 1.0, starts at 1.0
  figureCount: number;
  sectionWordCounts: Map<string, number>;
  warnings: string[];
  publicationVerdict: PublicationVerdict | null;
}
```

3.7.3 Decision Rules

The GOVERNOR implements 7 decision rules, evaluated in priority order:

Algorithm 2: Governor Decision Engine

Input: Batch of tool results B, current state S
Output: Governor evaluation (actions, updated state)

foreach *tool result* $r \in B$ **do**
- **Rule 1:** Detect pipeline stage from tool name, update S.stage;
- **Rule 2:** If Python error detected:
  - Parse error pattern (ModuleNotFoundError, MemoryError, KeyError, convergence, singular matrix)
  - Generate pattern-specific recovery advice
  - Decrement S.retryBudgets.python; if exhausted, emit `force_finalize`
  - Degrade S.confidence by 0.15
  - Switch to `debug_recovery` profile
- **Rule 3:** If data quality issue (empty dataset, missing columns, NaN ratio):
  - Decrement S.retryBudgets.data
  - Switch to `conservative` profile
- **Rule 4:** If LaTeX compilation failure:
  - Decrement S.retryBudgets.latex
  - Switch to `latex_fallback` profile (restricts to basic LaTeX)
- **Rule 5:** Track figure count from Python results;
- **Rule 6:** Track section word counts from written sections;
- **Rule 7:** On successful tool execution, recover confidence by 0.05;

end
Sort actions by priority: `force_finalize` > `block_publication` > `switch_profile` > `inject_directive` > `continue`;
return *Evaluation with prioritized actions, updated state*;

3.7.4 Prompt Profiles

The GOVERNOR can switch between 7 prompt profiles (`governor/promptProfiles.ts`, 160 lines):

Table 3.3: Governor prompt profiles.

Profile	Trigger	Max Figs	Simple?
`default`	Initial state	15	No
`debug_recovery`	Python errors	8	Yes
`conservative`	Data quality issues	10	Yes
`latex_fallback`	Compilation failures	10	No
`revision_mode`	Weak Popper review	12	No
`jf_rewrite`	Writing phase	15	No
`academic_figures`	Analysis phase	15	No

3.7.5 Publication Verdict

The GOVERNOR issues the final publication verdict based on Popper review scores:

$$\text{verdict} = \begin{cases} \text{allow} & \text{if score} \geq 80 \\ \text{allow_with_note} & \text{if } 65 \leq \text{score} < 80 \\ \text{revise} & \text{if } 45 \leq \text{score} < 65 \\ \text{block} & \text{if score} < 45 \\ \text{block_fatal} & \text{if confidence} < 0.2 \end{cases} \tag{3.1}$$

3.7.6 Decision Logging

All GOVERNOR decisions are recorded in a `DecisionLog` (`governor/decisionLog.ts`, 68 lines), an in-memory accumulator that:

- Assigns each decision a UUID and timestamp.
- Records the pipeline stage, trigger, action, reasoning, and confidence delta.
- Supports serialization to/from JSON for database persistence.
- Tracks intervention count (excluding "continue" decisions).

3.8 Popper — The Peer Reviewer

3.8.1 Role and Responsibilities

POPPER conducts AI-powered academic peer review, named after Karl Popper and his philosophy of falsificationism. It evaluates papers across 6 dimensions and issues a publication recommendation.

3.8.2 Review Process

Implemented in `peerReviewService.ts` (204 lines):

1. The full paper content (title, abstract, LaTeX sections, BibTeX) is submitted to Claude.
2. A detailed system prompt emulates a top-tier finance journal reviewer.
3. Claude evaluates 6 dimensions on a 0–100 scale:
   - **Originality** — Novelty of the research question and approach.
   - **Methodology** — Rigor and appropriateness of methods.
   - **Results** — Strength and interpretation of findings.
   - **Writing Quality** — Clarity, organization, and prose quality.
   - **References** — Completeness and relevance of citations.
   - **Reproducibility** — Whether the paper can be replicated.
4. An overall score (0–100) and recommendation are produced:

- **Accept** (≥ 80)
- **Minor Revision** (65–79)
- **Major Revision** (45–64)
- **Reject** (< 45)

5. Section-by-section feedback identifies specific strengths and weaknesses.

3.8.3 Configuration

POPPER uses low temperature (0.3) for evaluation consistency and is called via the Anthropic API with the same model (Claude Opus 4.6) as the main research pipeline. The minimum publication threshold (default 45/100) is configurable via the admin interface.

3.8.4 Failure Modes

- **Inconsistent scoring:** Despite low temperature, LLM evaluations can vary across runs.
- **Self-evaluation bias:** The same model generates and reviews the paper, creating potential positive bias.
- **Superficial review:** With limited context, POPPER may miss methodological errors that a domain expert would catch.

3.9 Agent Interaction Patterns

3.9.1 Sequential Handoff

The primary interaction pattern is **sequential handoff**: agents execute in order (Gauss → Turing → Newton → Fisher → Euler) with each agent's output becoming the next agent's context. Since all agents share a single Claude context window, there is no explicit message passing—the accumulated conversation history serves as the shared memory.

3.9.2 Governor Intervention

The GOVERNOR can interrupt the sequential flow by injecting directives. For example, if FISHER encounters a Python convergence failure, the GOVERNOR may:

1. Switch the profile to `debug_recovery`.
2. Inject a directive: "Simplify analysis. Use basic OLS instead of GMM. Limit figures to 8."
3. If retries are exhausted, emit `force_finalize` to skip remaining analysis and proceed to writing.

3.9.3 Popper Feedback Loop

In revision mode, POPPER's feedback can trigger a second pass:

1. POPPER reviews the paper and scores it < 65.

2. The GOVERNOR switches to `revision_mode` profile.
3. EULER re-enters writing mode to address specific criticisms.
4. The paper is recompiled and re-reviewed.

However, the current implementation limits revision to a single pass to prevent infinite loops.

Paper Generation Pipeline

4.1 Pipeline Overview

The paper generation pipeline is the central execution path of AutoScholar, transforming a research prompt into a published, peer-reviewed academic paper. The entire process executes within a single streaming HTTP connection lasting 45–90 minutes, involving up to 180 recursive tool calls, and consuming 500K–1M tokens of Claude context.

This chapter reconstructs the pipeline phase by phase, documenting the code involved, data flows, decision points, and failure modes at each stage.

4.2 Phase 1: Trigger and Initialization

4.2.1 Trigger Mechanisms

The pipeline can be triggered through three mechanisms:

Scheduled generation. The `autoPaperScheduler.ts` (1,285 lines) runs on a configurable cadence (default: 1 hour). The scheduler maintains a cursor into the 1,000+ entry prompt bank and selects the next prompt.

Admin-triggered generation. Administrators can trigger generation via `POST /api/admin/trigger-` (random prompt) or `POST /api/admin/trigger-paper-custom` (specified prompt).

Interactive generation. Users submit research prompts through the web interface, which may be detected as paper-generation requests and upgraded.

4.2.2 Prompt Selection

For scheduled generation, the prompt selection algorithm works as follows:

Algorithm 3: Prompt Selection in the Auto-Paper Scheduler

Input: Prompt bank $\mathcal{P}$ with 1,000+ entries, cursor position c, selection mode m
Output: Selected prompt entry p

if $m = \textit{sequential}$ **then**
  $p \leftarrow \mathcal{P}[c \bmod |\mathcal{P}|]$;
  $c \leftarrow c + 1$;
else if $m = \textit{weighted_random}$ **then**
  $w_i \leftarrow 1.0$ for all i;
  foreach *recently used domain* d **do**
    $w_i \leftarrow w_i \times 0.5$ for all p_i with domain d;
  end
  $p \leftarrow \text{WeightedSample}(\mathcal{P}, w)$;
else
  $p \leftarrow \text{UniformRandom}(\mathcal{P})$;
end
return p *with fields: domain, method, topic, full prompt text*;

The prompt bank is constructed deterministically at import time from three sources:

- **24 domains** — 18 finance (market microstructure, volatility modeling, asset allocation, ...) + 6 stats/ML/AI domains.
- **24 methods** — 18 econometric (panel GMM, event studies, DCC-GARCH, ...) + 6 stats/ML/AI methods.
- **14 topics per domain** — Specific, publishable research questions.
- **Compatibility matrix** — Maps each domain to its compatible methods, preventing nonsensical pairings.

The iteration order (variation, topic, domain) ensures consecutive prompts always switch domains, providing topic diversity.

4.2.3 Surprise Prompt Generation

For interactive sessions, the "surprise paper" feature (`generateSurprisePrompt()`) picks a random prompt from the full pool, weighted to avoid recently selected ones (tracked in localStorage with a 30-entry cap).

4.2.4 Pipeline Run Initialization

When a scheduled run begins:

1. A `PaperRun` record is created in the database with status `queued`.
2. A session ID is generated for the conversation.
3. The GOVERNOR is initialized with default state (confidence 1.0, all retry budgets full).
4. A heartbeat timer starts (10-second interval) to detect stalled runs.
5. The run status transitions to `researching`.

4.3 Phase 2: Research Prompt Enhancement

4.3.1 Research Detection

Before reaching Claude, the prompt passes through `isResearchPaperRequest()` (`researchPromptEnhan` which uses two-part regex matching:

- **Verb patterns:** "write", "generate", "create", "produce", "draft", "compose"
- **Noun patterns:** "paper", "article", "study", "thesis", "report", "analysis"

4.3.2 Methodology Detection

`detectResearchMethodology()` classifies the prompt into one of four tracks:

- **Econometrics:** Keywords include "regression", "panel", "GMM", "GARCH", "causal", "instrumental".
- **Statistics:** Keywords include "hypothesis", "bootstrap", "Bayesian", "distribution", "non-parametric".
- **Machine Learning:** Keywords include "random forest", "XGBoost", "neural", "classification", "prediction".
- **AI:** Keywords include "deep learning", "transformer", "NLP", "attention", "fine-tuning".

4.3.3 Prompt Upgrading

`upgradeResearchPrompt()` calls Claude Opus non-streaming (temperature 1.0 for creative variety) to transform vague prompts into detailed 400–700 word research specifications. Each methodology type receives tailored guidance:

Econometrics: Must include identification strategies, 6–10 specific variable definitions, and endogeneity considerations.

Statistics: Must specify distributional assumptions, test statistics, and power analysis considerations.

Machine Learning: Must name a baseline + 4 models with cross-validation strategy and hyperparameter search space.

AI: Must specify architecture details, training regime, and ablation study design.

The upgrader preserves the language of the original prompt (supports French and English).

4.4 Phase 3: Literature Review (Gauss Phase)

4.4.1 System Prompt Activation

When the upgraded prompt reaches Claude, the 1,798-line system prompt activates the Gauss role for the first phase. The system prompt instructs:

"Begin with Agent Gauss. Search at least 3 academic databases. Identify 15–30 relevant papers. Extract methods, datasets, and limitations. Identify 3–5 research gaps."

4.4.2 Execution Sequence

GAUSS typically executes the following tool sequence:

1. `academic_search_multi` — Parallel search across Semantic Scholar, OpenAlex, arXiv, CrossRef, and Google Scholar.
2. `literature_gap_finder` — If the initial search returns sufficient results, run the full 7-step gap analysis pipeline.
3. Individual database queries for targeted follow-up.
4. `web_search_exa` or `tavily_search` — For recent working papers not yet indexed in academic databases.

4.4.3 Outputs

The GAUSS phase produces:

- A list of 15–30 relevant papers with BibTeX entries.
- Identified research gaps ranked by novelty, feasibility, and impact.
- A refined research question based on gap analysis.
- Suggested methodology and data requirements.

4.5 Phase 4: Data Acquisition (Turing Phase)

4.5.1 Data Strategy

Based on GAUSS's findings, EULER transitions to the TURING role and plans a data acquisition strategy. The system prompt defines strict data source hierarchies with fallback chains:

1. For stock/company data: FMP → EODHD → web scraping.
2. For macro data: FRED → World Bank → IMF → OECD.
3. For academic datasets: structured APIs → SWDES web extraction.
4. For alternative data: Polymarket → Google Trends → news sentiment.

4.5.2 Typical Tool Sequences

A typical TURING phase for a cross-sectional asset pricing study might involve:

1. `get_historical_price` for multiple assets (portfolio returns).
2. `get_income_statement` / `get_balance_sheet` for fundamental factors.
3. `get_treasury_rates` for the risk-free rate.

4. `download_fmp_data` to export bulk data to CSV.
5. `get_wb_indicator_data` for country-level controls.
6. `execute_custom_python_analysis` to merge and validate datasets.

4.5.3 Data Provenance

The system prompt mandates data provenance documentation:

> *"For every dataset used, record: source API, endpoint, parameters, date range, number of observations, and any transformations applied."*

This information is later embedded in the paper's data section and the project ZIP's metadata files.

4.6 Phase 5: Data Engineering (Newton Phase)

4.6.1 Execution

NEWTON primarily operates through `execute_custom_python_analysis`, running code that:

1. Loads downloaded CSV/JSON files into pandas DataFrames.
2. Cleans and standardizes column names, date formats, and numeric types.
3. Merges datasets by date or entity identifiers.
4. Constructs derived variables (returns, ratios, dummies, interactions).
5. Runs preliminary diagnostics (summary statistics, correlation matrices, stationarity tests).
6. Saves the prepared dataset for the analysis phase.

4.6.2 Script Management

Each Python execution is tracked by the ScriptManager (`scriptManager.ts`, 276 lines):

- Scripts are saved to session-scoped temp directories with unique IDs.
- Up to 8 repair attempts are allowed per script.
- The diffEngine supports unified diffs, search-replace blocks, and fuzzy matching.
- Structured errors are parsed and formatted for Claude's next turn.

4.7 Phase 6: Empirical Analysis (Fisher Phase)

4.7.1 Model Selection Logic

FISHER's model selection follows the methodology track detected during prompt enhancement:

Econometrics track: Start with OLS baseline, add fixed effects, instrument with 2SLS if endogeneity is suspected, robustness checks with alternative specifications, clustering of standard errors.

Statistics track: Formulate null/alternative hypotheses, select appropriate test statistics, compute p-values and confidence intervals, assess power.

ML track: Train baseline (logistic regression or OLS), train 4 advanced models, cross-validate, compare via appropriate metrics (RMSE, AUC, F1), compute SHAP values for interpretation.

AI track: Implement architecture, train with early stopping, run ablation studies, visualize attention patterns.

4.7.2 Robustness Checks

The system prompt mandates robustness checks:

1. Alternative variable definitions (e.g., different return windows).
2. Subsample analysis (pre/post crisis, by sector, by size).
3. Alternative model specifications (e.g., Newey-West vs. clustered SE).
4. Placebo tests where applicable.
5. Sensitivity to outlier removal.

4.7.3 Figure Generation

Each analysis produces figures following the academic figure style directive. The Governor tracks figure count and enforces limits based on the active profile (8–15 figures depending on error state). Generated figures pass through:

1. Matplotlib rendering with JF-style configuration.
2. Blank figure detection (axes artist inspection + pixel variance analysis).
3. AutoScholar watermark addition.
4. Base64 encoding and SSE transmission to the client.
5. Optional vision analysis via Claude Opus for figure description.

4.8 Phase 7: Paper Writing (Euler Write Phase)

4.8.1 Section-by-Section Generation

Euler writes the paper one section at a time using the `write_paper_section` tool. The tool definition specifies extremely detailed length and quality requirements, targeting 60–90 page papers.

Empirical Paper Sections

1. **Introduction** (untitled, following JF convention) — Motivation, contribution, preview of results.
2. **Literature Review** — Organized by theme, positions the paper relative to existing work.
3. **Methodology** — Formal model specification, estimation strategy, identification.
4. **Data** — Sources, construction, summary statistics, preliminary analysis.
5. **Results** — Main findings, robustness checks, subsample analysis.
6. **Discussion** — Economic interpretation, implications, comparison with existing literature.
7. **Conclusion** — Summary, contributions, limitations, future research.

Theoretical Paper Sections

1. **Economic Environment** — Model setup, agents, preferences, technology.
2. **Equilibrium** — Equilibrium concept, characterization, existence/uniqueness.
3. **Computational Method** — Numerical solution, calibration, simulation.
4. **Quantitative Results** — Model implications, moment matching, counterfactuals.
5. **Extensions** — Model variants, sensitivity analysis.

4.8.2 Writing Style Enforcement

During the writing phase, the GOVERNOR injects the JF Writing Style Directive (282 lines), which includes:

- 100+ banned words/phrases organized by severity tier.
- Section-specific conventions (e.g., Introduction is unnumbered, abstract $\leq$ 100 words).
- Citation formatting rules ("Author (year)" for narrative, "(Author, year)" for parenthetical).
- Advanced prose techniques: emphatic stress positioning, topic-comment sentence structure.
- AI detection countermeasures: lexical diversity targets, syntactic variation requirements.
- An 8-pass verification checklist for self-review.

4.8.3 Section Rejection Tracking

The tool executor (`toolExecutor.ts`) tracks section rejection to prevent infinite loops. If a section is rejected more than 3 times (due to insufficient length or quality), the system accepts the best available version and moves on.

4.9 Phase 8: LaTeX Compilation

4.9.1 Section Assembly

The `compile_research_paper` tool (`latex/compiler.ts`, 91 lines) sorts sections into canonical academic order using a predefined ordering map:

Listing 4.1: Section ordering map.

```
const SECTION_ORDER = {
  'introduction': 1,
  'literature_review': 2,
  'methodology': 3,
  'data': 4,
  'results': 5,
  'discussion': 6,
  'conclusion': 7,
  'appendix': 100,
};
```

Appendix sections automatically receive a `\appendix` command.

4.9.2 Template Selection

Three document templates are available (`latex/paperGenerator.ts`, 1,235 lines):

`article` — Standard two-column academic article with `natbib` bibliographic style.

`working_paper` — Single-column working paper format with wider margins.

`econometrica` — Mimics the *Econometrica* journal style.

All templates share a common preamble with 50+ LaTeX packages and custom macros.

4.9.3 Compilation Pipeline

Algorithm 4: LaTeX Compilation Pipeline

Input: Sections, BibTeX, figures, template
Output: Compiled PDF path

```
Write paper.tex, references.bib, and figure files to temp directory;
Sanitize LaTeX content (escape special chars, fix Greek letters, handle math mode);
Auto-fit wide tables (wrap in adjustbox or resizebox);
Pass 1: Run pdflatex -interaction=nonstopmode paper.tex;
if BibTeX present then
    Pass 2: Run bibtex paper;
    Pass 3: Run pdflatex (resolve references);
    Pass 4: Run pdflatex (finalize);
end
if compilation failed then
    Register compilation with latexPatchManager;
    for attempt ← 1 to 8 do
        Parse errors via latexLogParser;
        Apply diff patch via diffEngine;
        Recompile;
        if success then
            break;
        end
    end
end
return PDF path or error;
```

4.9.4 LaTeX Sanitization

The sanitizer (`latex/sanitizer.ts`, 670 lines) fixes common Claude-generated LaTeX issues:

- Unescaped special characters (%, &, $, #, _).
- Greek letter handling outside math mode.
- Mismatched math delimiters.
- Em-dash and en-dash normalization.
- Wide table detection and automatic wrapping.
- Landscape environment stripping (known to cause compilation issues).
- Blank PNG figure detection (decompresses IDAT chunks and measures pixel variance).

4.9.5 Iterative Repair

The LaTeX patch manager (`latex/latexPatchManager.ts`, 369 lines) provides a read-patch-recompile cycle:

1. `inspectLatexErrors` parses the log file, classifying errors into 8 types (undefined control sequence, math, unicode, environment, package, fatal, missing character, other).
2. `patchLatex` applies diffs using the shared diffEngine with fuzzy matching.
3. `recompileLatex` runs the full pdflatex+bibtex pipeline.
4. Up to 8 repair attempts are allowed before giving up.

4.10 Phase 9: Packaging

The `latex/packaging.ts` (358 lines) creates a ZIP archive following the Reproducible Research Engine (RRE) directory structure:

```
project/
  paper.tex              % Main LaTeX source
  references.bib         % BibTeX references
  outputs/
    paper.pdf            % Compiled PDF
    figures/             % All generated figures
  code/
    01_data_download.py  % Pipeline-ordered scripts
    02_data_prep.py
    03_analysis.py
    ...
  data/
    raw/api/             % API-sourced data
    raw/web/             % Web-extracted data
    processed/           % Cleaned datasets
    metadata/            % Data provenance files
  run_pipeline.py        % Auto-generated master script
  requirements.txt       % Auto-detected dependencies
  README.md              % Build instructions and documentation
```

Key features:

- Scripts are ordered by pipeline stage using numeric prefixes.
- Data files are routed to structured subdirectories based on source type.
- `requirements.txt` is auto-generated by parsing imports from all scripts and mapping to pip package names.
- `run_pipeline.py` is auto-generated with sequential script execution.
- `README.md` includes abstract, build instructions, project structure, and data provenance.

4.11 Phase 10: Validation (Popper Phase)

4.11.1 Review Trigger

After successful compilation, the scheduler invokes POPPER:

1. The run status transitions to `validating`.
2. The full paper content (title, abstract, sections, BibTeX) is submitted to `generatePeerReview()`.
3. Claude reviews the paper at temperature 0.3 for consistency.
4. The review result includes:
   - Overall score (0–100).
   - Recommendation (accept/minor_revision/major_revision/reject).
   - 6-dimension detailed scores.
   - Section-by-section feedback.
   - Strengths and weaknesses lists.

4.11.2 Publication Decision

The GOVERNOR evaluates the Popper review via `evaluatePopper()`:

- Score ≥ 45: Paper is published (run status $\rightarrow$ `published`).
- Score < 45: Paper is rejected (run status $\rightarrow$ `rejected`).
- The review is stored in the `peer_reviews` table.

4.12 Phase 11: Publication

4.12.1 ASID Assignment

Published papers receive an ASID via `doiService.ts` (195 lines):

$$\text{ASID} = \texttt{ASID-}\langle\text{year}\rangle\texttt{-}\langle\text{sequence}\rangle \tag{4.1}$$

where sequence is a zero-padded 4-digit number generated by atomic database counter increments (thread-safe for concurrent generation).

4.12.2 Storage

The published paper record includes:

- Title, abstract, domain classification, template.
- Full LaTeX sections (stored as JSON).
- BibTeX references.
- Figure count, word count, section count, LaTeX line count.
- Peer review score and recommendation.

- Project ZIP file path.
- ASID.
- Associated session ID for cost tracking.

4.12.3 Domain Classification

The domain detector (`utils/domainDetector.ts`) classifies papers into domains based on keyword analysis of the title and abstract, using a weighted scoring system across 24 domain categories.

4.13 Checkpoint and Resume System

4.13.1 Motivation

Given that pipeline runs last 45–90 minutes, crash recovery is essential. The checkpoint system allows runs to resume from the last saved state after server restarts.

4.13.2 Checkpoint Structure

Listing 4.2: Pipeline checkpoint structure.

```
interface PipelineCheckpoint {
  runId: string;
  sessionId: string;
  prompt: string;
  messages: ClaudeMessage[];
  governorState: AppState;
  decisionLog: DecisionLogEntry[];
  toolResults: ToolResult[];
  writtenSections: { name: string; content: string }[];
  figureCount: number;
  stage: PipelineStage;
}
```

4.13.3 Checkpoint Lifecycle

1. Checkpoints are saved every 8 seconds during active generation.
2. On server restart, the scheduler queries for runs with status `researching`, `analyzing`, `drafting`, or `validating`.
3. Active runs are marked as `failed` (to prevent zombie runs).
4. Administrators can manually resume from a checkpoint via the admin API.
5. Resumed runs restore the full conversation history, governor state, and continue from the last tool call.

4.14 Pipeline Metrics

Typical pipeline execution metrics:

Table 4.1: Typical pipeline execution metrics.

Metric	Typical Value
Total duration	45–90 minutes
Tool calls	80–180
Python executions	15–40
Figures generated	8–15
Context tokens consumed	500K–1M
API cost per paper	$15–$40 (Opus 4.6 pricing)
Paper length	60–90 pages
BibTeX references	30–80
LaTeX compilation attempts	1–4
Popper review score	50–85 (mean ≈ 65)

Autonomous Journal System

5.1 What Makes It a Journal

AutoScholar implements what can be described as the first **autonomous academic journal**—a system that continuously generates, reviews, and publishes research papers without human editorial oversight. The term "journal" is used deliberately to evoke the key characteristics shared with traditional academic publishing:

Continuous publication. Papers are generated and published on a regular cadence, similar to a journal's issue schedule, but at a much higher frequency (one paper per hour vs. quarterly or monthly issues).

Peer review. Every paper undergoes AI-powered peer review (Popper) before publication, with explicit acceptance/rejection criteria.

Persistent identifiers. Published papers receive AutoScholar Identifiers (ASIDs) analogous to DOIs, enabling stable references.

Quality thresholds. Papers scoring below 45/100 on the peer review are rejected, enforcing a minimum quality standard.

Archival storage. All papers, including rejected ones, are stored with full source code, data, and reproducibility packages.

Public access. Published papers are accessible via the web interface and REST API.

However, it is critical to note that this is a *machine-generated* journal with no human reviewers, editors, or authors. The quality standards, while enforced systematically, are evaluated by the same class of AI model that generated the content.

5.2 Scheduling Architecture

5.2.1 The AutoPaper Scheduler

The scheduler (`autoPaperScheduler.ts`, 1,285 lines) is the heartbeat of the autonomous journal. It manages the full lifecycle of paper generation runs.

Configuration

The scheduler exposes a comprehensive configuration interface:

Table 5.1: Scheduler configuration parameters.

Parameter	Description	Default
`enabled`	Whether the scheduler is active	`true`
`cadenceMinutes`	Time between paper generation starts	60
`maxParallel`	Maximum concurrent paper generations	3
`selectionMode`	Prompt selection strategy	`sequential`
`popperThreshold`	Minimum peer review score for publication	45
`heartbeatTimeout`	Seconds before a run is considered stalled	300
`cleanupEnabled`	Auto-cleanup of stale runs	`true`
`maxConsecutiveFailures`	Failures before auto-pausing	5

Lifecycle States

Each pipeline run transitions through a defined state machine:

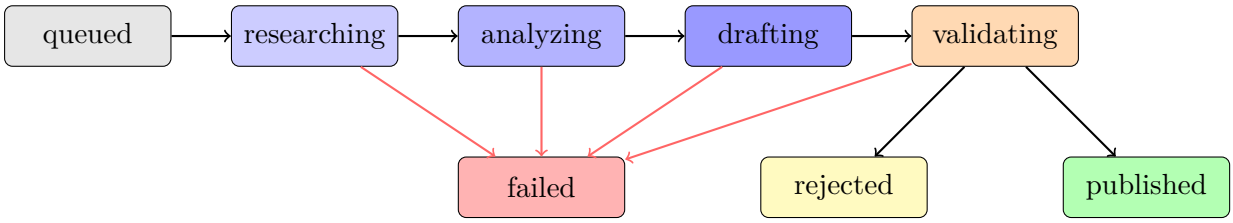


Figure 5.1: Pipeline run state machine.

Parallel Execution

The scheduler supports up to 3 concurrent paper generations. Before starting a new run, it checks:

1. The number of active runs (status $\in$ {queued, researching, analyzing, drafting, validating}).
2. If active runs $\geq$ `maxParallel`, the new run is deferred to the next cadence tick.
3. Each run executes independently with its own session, governor state, and checkpoint.

5.2.2 Heartbeat Monitoring

Every active run updates its heartbeat timestamp every 10 seconds. The scheduler monitors heartbeats and:

- Marks runs as `failed` if no heartbeat is received for `heartbeatTimeout` seconds (default: 300s = 5 minutes).
- Logs stale run detection for debugging.
- On server restart, force-fails all active runs to prevent zombie processes.

5.2.3 Admin Controls

Administrators have full control over the scheduler via the admin API:

- **Enable/disable** the scheduler globally.
- **Update configuration** (cadence, parallel limit, selection mode, thresholds).
- **Trigger** immediate paper generation (random or custom prompt).
- **Abort** running papers by run ID.
- **Resume** failed runs from checkpoints.
- **Cleanup** stale or orphaned runs.
- **View** active run status and real-time progress.

5.3 Quality Control

5.3.1 Multi-Layer Quality Assurance

Quality is enforced at multiple pipeline stages:

1. **Prompt quality:** The 1,000+ entry prompt bank is curated with specific, publishable research questions. Prompts are enhanced by the research prompt upgrader.
2. **Data quality:** The GOVERNOR monitors data collection and flags empty datasets, missing columns, and high NaN ratios. The `conservative` profile adds extra validation steps.
3. **Analysis quality:** The system prompt mandates robustness checks, alternative specifications, and subsample analysis. The GOVERNOR enforces figure count limits.
4. **Writing quality:** The JF Writing Style Directive enforces publication-quality prose with banned words, section conventions, and an 8-pass verification checklist.
5. **Compilation quality:** The LaTeX sanitizer and iterative repair loop ensure compilable output.
6. **Peer review:** POPPER evaluates 6 dimensions and issues a publication verdict.

5.3.2 Quality Metrics

The system tracks the following quality metrics per paper:

- Popper overall score (0–100).
- Popper dimension scores (originality, methodology, results, writing, references, reproducibility).
- Word count and section count.
- Figure count and LaTeX line count.
- Governor confidence at publication time.
- Number of governor interventions.

- Number of Python error recoveries.
- Number of LaTeX compilation attempts.

5.3.3 Publication Threshold

The configurable publication threshold (default 45/100) represents the minimum Popper score required for publication. This threshold was calibrated based on early runs:

- Papers scoring >80 demonstrate clear methodology, original findings, and coherent writing.
- Papers scoring 65–80 are acceptable with minor issues.
- Papers scoring 45–65 have significant weaknesses but contain enough substance to publish.
- Papers scoring <45 have fundamental flaws (fabricated data, incoherent methodology, missing analysis) and are rejected.

5.4 Publication Mechanics

5.4.1 ASID System

The AutoScholar Identifier (ASID) system (`doiService.ts`, 195 lines) provides persistent, stable references for published papers.

Format

$$\texttt{ASID-}\langle\text{year}\rangle\texttt{-}\langle\text{sequence}\rangle \tag{5.1}$$

Examples: `ASID-2026-0001`, `ASID-2026-0042`, `ASID-2026-1337`.

Generation

ASIDs are generated using atomic database counter increments:

- **SQLite:** Uses `better-sqlite3` transactions with `UPDATE ... SET counter = counter + 1 RETURNING counter`.
- **PostgreSQL:** Uses `ON CONFLICT ... DO UPDATE ... RETURNING counter`.

This ensures collision-free generation even under concurrent paper production.

Resolution

ASIDs resolve to paper landing pages via:

- Web: `https://autoscholar.ai/paper/ASID-2026-0001`
- API: `GET /api/asid/ASID-2026-0001`
- API v1: `GET /api/v1/papers/:id`

5.4.2 Paper Storage

Published papers are stored in the `published_papers` table with the following schema:

- `id` (auto-increment primary key)
- `title`, `abstract`, `authors`
- `domain` (auto-classified), `template` (article/working_paper/econometrica)
- `latexSections` (JSON: section name → LaTeX content)
- `bibtexContent` (full BibTeX string)
- `figureCount`, `wordCount`, `sectionCount`, `latexLineCount`
- `peerReviewScore`, `peerReviewRecommendation`
- `projectZipPath` (path to the reproducible archive)
- `asid` (AutoScholar Identifier)
- `sessionId` (for cost tracking)
- `isVisible` (admin-toggleable publication visibility)
- `createdAt`, `updatedAt`

5.4.3 Repository Browser

The repository routes (`routes/reports/repositoryRoutes.ts`, 550 lines) provide a GitHub-like interface for browsing project ZIPs:

- File tree extraction from ZIP archives.
- Inline file viewing for code, LaTeX, markdown, and data files.
- Raw file download.
- Separate listings for scripts, figures, and data files.
- Auto-generated README with abstract, download links, and quick-start instructions.
- Full project ZIP download.

5.5 Journal Analytics

The platform tracks comprehensive analytics:

Platform stats. Total papers, words, figures, sections, LaTeX lines, Python executions, tokens consumed, and API cost.

Domain distribution. Papers per domain, word/figure counts per domain.

Timeline. Monthly publication trends.

Pipeline stats. Average duration, Popper score distribution, status breakdown.

Cost tracking. Per-paper cost via session token counts and Opus 4.6 pricing ($5/$25 per MTok). Known baseline of $679.51 for costs incurred before tracking was implemented.

5.6 Continuous Generation Model

The autonomous journal operates as a continuous production system:

1. Every hour, the scheduler checks for available capacity.
2. A prompt is selected from the bank (sequential or weighted-random).
3. The full pipeline executes (45–90 minutes).
4. The paper is validated and published (or rejected).
5. The cursor advances, and the cycle repeats.

At maximum throughput (3 parallel, 1-hour cadence), the system can produce approximately 24–48 papers per day, depending on pipeline duration and failure rates. Over a month, this translates to 500–1,000 papers covering all 24 research domains multiple times.

5.7 Comparison with Traditional Journals

Table 5.2: Comparison between AutoScholar journal and traditional academic journals.

Feature	AutoScholar	Traditional
Publication frequency	Hourly	Quarterly/Monthly
Time from submission to publication	45–90 min	6–24 months
Peer review	AI (single pass)	Human (2–3 reviewers)
Author	AI agent	Human researcher
Reproducibility package	Mandatory (ZIP)	Rare
Persistent identifier	ASID	DOI
Open access	Yes (free)	Often paywalled
Quality assurance	Automated threshold	Human editorial judgment
Domain coverage	24 domains	Typically 1–3

Data and Storage Architecture

6.1 Overview

AutoScholar's data architecture spans three distinct layers: (1) **external data acquisition** from 15+ API providers, (2) **transient computational data** flowing through Python subprocesses, and (3) **persistent storage** in SQLite or PostgreSQL via Drizzle ORM. This chapter documents each layer in detail.

6.2 External Data Sources

6.2.1 Financial Market Data

Financial Modeling Prep (FMP)

The primary financial data provider, accessed via 230+ endpoints organized into 15 service modules (`server/services/fmp/`):

Table 6.1: FMP service modules and coverage.

Module	Coverage
`company.ts`	Profiles, financials (income/balance/cash flow), TTM variants, as-reported, growth, scores, enterprise values, revenue segmentation
`quotes.ts`	Real-time quotes, historical prices, intraday, aftermarket, batch quotes (stocks, ETFs, mutual funds, commodities, crypto, forex, indices), charts
`earnings.ts`	Calendar, surprises, analyst estimates, price targets, upgrades/downgrades, transcripts, ratings
`search.ts`	Symbol search, name search, exchange filtering, CIK/ISIN/CUSIP lookup
`technical.ts`	RSI, MACD, EMA, SMA, ADX, Williams %R, CCI, Stochastic, WMA, DEMA, TEMA, standard deviation
`news.ts`	Stock news, crypto news, forex news, press releases, general financial news
`insider.ts`	Insider trading, congressional trading (Senate + House)

Module	Coverage
sec.ts	SEC filings (8-K, 10-K, 10-Q), by symbol/CIK/form type, RSS feeds
etf.ts	ETF holdings, ETF asset exposure, mutual fund disclosures
economic.ts	Treasury rates, economic calendar, economic indicators, market risk premium
esg.ts	ESG scores, ESG ratings, ESG benchmarks, ESG disclosures
options.ts	(placeholder for future FMP options)
cryptoForex.ts	Crypto quotes and lists, forex pairs and historical data
bulk.ts	Batch operations, index constituents (S&P 500, NASDAQ, Dow Jones)
client.ts	Base HTTP client with v3/v4/stable endpoints and 60s timeout

EODHD (End of Day Historical Data)

eodhd_service.ts (289 lines) provides extended historical coverage:

- **End-of-day prices:** US stocks back to IPO date, non-US from 2000. OHLCV with adjusted close.
- **Intraday data:** 1-minute, 5-minute, 1-hour bars with pre/after-market data.
- **Options chains:** US options for 6,000+ stocks with end-of-day Greeks (δ, γ, θ, ν).
- **Live quotes:** Real-time price data.

Key implementation details:

- Auto-normalizes bare tickers to `.US` suffix format.
- Accepts flexible date inputs (Unix timestamps, ISO strings, datetime objects).
- Sanitizes null/NaN values in options data.

Financial Data API (Real Options)

financialDataService.ts (295 lines) provides real-time options market data:

- **Option chains:** All available contracts (calls and puts) with identifiers.
- **Option prices:** Last price, bid/ask, volume, open interest, implied volatility, intrinsic/time value.
- **Option Greeks:** Delta (δ), Gamma (γ), Theta (θ), Vega (ν), Rho (ρ), plus implied volatility.

All functions validate the OCC identifier format: `SYMBOL+YYMMDD+C/P+Strike×1000`.

6.2.2 Macroeconomic Data

FRED (Federal Reserve Economic Data)

Accessed via the Python client `fred_client.py` (135 lines) and JavaScript service:

- 840,000+ macroeconomic time series.
- Unit transformations (change, percent change, log).
- Frequency aggregation (daily, weekly, monthly, quarterly, annual).
- Series search with tag filtering and sort options.
- Category navigation and release tracking.

World Bank

`worldBankService.ts` (130 lines), no API key required:

- 16,000+ development indicators across 200+ countries.
- Metadata: countries with income/region filtering, topics, regions, income levels, data sources.
- Client-side keyword search (WB API v2 lacks native search).
- Data retrieval with date ranges, frequency, and source filtering.

IMF (International Monetary Fund)

`imfService.ts` (93 lines), dual-interface:

DataMapper API: Simple access to WEO macroeconomic indicators (GDP growth, inflation, unemployment).

SDMX 3.0 API: Advanced multi-dimensional queries across CPI, IFS, BOP, and other datasets. CSV output.

OECD

`oecdService.ts` (99 lines), no API key required:

- 1,475+ SDMX dataflows covering economics, health, education, environment.
- Dot-separated dimension filters with CSV-with-labels output.
- Dimension structure and codelist discovery.
- Client-side keyword search across all dataflows.

6.2.3 Academic Data Sources

`academicSearchService.ts` (567 lines) provides a unified interface to 5 academic databases:

Table 6.2: Academic data sources.

Source	Capabilities	Auth
Semantic Scholar	AI-powered search, citation data, paper details, references	API key
OpenAlex	200M+ works, citation network traversal, institutional data	Free
arXiv	Preprint search, PDF access, category filtering	Free
CrossRef	DOI resolution, 130M+ records, reference expansion	Free
Google Scholar	Full-text search via SerpAPI, citation counts, PDF links	SerpAPI key

The `multiSourceAcademicSearch()` function:

1. Runs all 5 sources in parallel via `Promise.allSettled`.
2. Deduplicates by DOI/title similarity.
3. Ensures unique BibTeX cite keys (appending letters for collisions).
4. Sorts by citation count.
5. Returns combined results with a merged `bibtex_block`.

6.2.4 Web Intelligence

Three web data providers offer complementary capabilities:

Tavily (`tavilyService.ts`, 367 lines) — 5 endpoints: AI-enhanced search with topic filtering, URL content extraction with relevance ranking, graph-based crawling, site mapping, and autonomous research agent.

Firecrawl (`firecrawlService.ts`, 530 lines) — 7 endpoints: single-page scraping (with PDF support), web search with auto-scraping, multi-page crawling (async with polling), AI-powered structured extraction, URL mapping, autonomous AI agent, and batch scraping.

Exa (`exaService.ts`, 149 lines) — 2 endpoints: neural/keyword semantic search with domain and category filtering, and URL content extraction with AI summaries.

6.2.5 Prediction Markets

`polymarketService.ts` (140 lines) provides read-only access to Polymarket via 3 public APIs:

- **Gamma API:** Events, markets, search, tags.
- **Data API:** Trades, leaderboard.
- **CLOB API:** Order books, bid/ask prices, price history, spreads.

All endpoints are public with no authentication required.

6.3 Transient Computational Data

6.3.1 Python Execution Environment

The Python sandbox (`customPythonExecutor.py`, 1,774 lines) provides a rich data environment:

Pre-Loaded API Clients

Table 6.3: Pre-loaded Python API clients.

Variable	Client	Capabilities
`fmp`	`FMPClient`	Quotes, historical prices, financials, technical indicators
`fred`	`FREDClient`	840K+ series with unit transforms and frequency aggregation
`eodhd`	`EODHDClient`	EOD/intraday prices, options chains with Greeks
`wb/imf/oecd`	`WorldBankClient` etc.	Macro data (16K+ WB indicators, IMF WEO, OECD SDMX)
`claude`	`ClaudeClient`	NLP tasks (summarize, sentiment, classify, extract entities)
`openrouter`	`OpenRouterClient`	400+ LLMs, 12+ embedding models, semantic analysis

Embedded Databases

Table 6.4: Embedded DuckDB databases available in Python.

Client	Records	Description
`sdc_ipo`	452K	SDC Platinum IPO data (via DuckDB)
`housing`	839K+	Zillow US housing market data (via DuckDB)
`hf_data`	7.7B	High-frequency tick records
`options_data`	3.8B	Historical options data

Data Flow Through Python

1. Claude writes Python code via the `execute_custom_python_analysis` tool.
2. The TypeScript bridge (`customPython.ts`) applies automatic patches and spawns the Python subprocess.
3. Python code receives context via JSON stdin (API keys, session ID, previous outputs).
4. The code executes with access to all pre-loaded clients and databases.
5. Results are returned as structured JSON: figures (base64), files (paths), stdout text, and errors.

6. The ScriptManager saves the script for potential repair iterations.

6.3.2 Generated Data Files

The `downloads/` directory accumulates generated data files:

- **CSV files:** Tabular data exports from FMP, FRED, and custom analysis.
- **XLSX files:** Styled Excel workbooks with conditional formatting.
- **JSON files:** Metadata sidecars and structured results.
- **ZIP files:** Reproducible research project archives.
- **PNG/PDF files:** Generated figures and charts.

Files are named with timestamp prefixes for uniqueness and served via the download endpoint.

6.4 Persistent Storage

6.4.1 Database Schema

The database schema (`shared/schema-sqlite.ts`, 456 lines; `shared/schema.ts`, 357 lines) defines 20+ tables:

User and Authentication Tables

`users` — User accounts with username, email, hashed password, display name, institution, department, ORCID, API key, role, and token.

`userProfiles` — Extended user profile information.

`emailVerificationTokens` — Email verification token tracking.

`passwordResetTokens` — Password reset token tracking.

Content Tables

`messages` — Chat messages with session ID, role, content, serialized sources, and timestamps.

`sharedReports` — Published research reports with share IDs, questions, answers, sources, tool results, and figures.

`crawledPages` — Web page content for indexing.

`embeddings` — Vector embeddings for semantic search (stored as JSON text in SQLite, native vector in PostgreSQL).

Research Output Tables

`publishedPapers` — Published papers with title, abstract, domain, template, LaTeX sections (JSON), BibTeX, figure/word/section/line counts, peer review score, project ZIP path, ASID, visibility, and timestamps.

peerReviews — AI peer review results with scores, recommendation, strengths, weaknesses, detailed scores (JSON), section reviews (JSON), and full report text.

savedPrompts — Public prompt library entries.

Pipeline Management Tables

paperRuns — Pipeline run tracking with status, prompt, session ID, heartbeat, Popper score, governor state (serialized JSON), timestamps.

governorDecisions — Individual governor decision records with run ID, stage, trigger, action, reasoning, confidence.

asidCounters — Atomic counters for ASID generation, keyed by year.

Analytics Tables

conversationSessions — Session metadata with user ID, session ID, title, message count, token usage, cost, and timestamps.

activeUsers — Heartbeat-based active user tracking with 2-minute activity window.

analyticsMetrics — Time-series platform metrics (requests, users, performance, tokens, errors).

requestLogs — API request audit log with method, path, status, duration, user agent.

Administrative Tables

siteConfig — Key-value configuration store with upsert semantics.

donations — Crypto and fiat donation records.

apiKeys — Public API keys with SHA-256 hashing, scopes, rate limits, usage tracking.

6.4.2 Storage Layer Implementation

The storage layer is decomposed into 6 domain-specific modules that are composed into a single `IStorage` interface (70+ methods):

`user-storage.ts` (49 lines) — CRUD with cascading session deletion.

`message-storage.ts` (64 lines) — Messages and shared reports.

`session-storage.ts` (78 lines) — Session lifecycle management.

`paper-storage.ts` (232 lines) — Papers, pipeline runs, cost calculation (5/$25 per MTok for Opus 4.6).

`analytics-storage.ts` (117 lines) — Active users, metrics, request logs.

`admin-storage.ts` (167 lines) — Governor state, reviews, donations, API keys, site config.

6.4.3 Database Migrations

For SQLite, migrations are performed inline in `db.ts` (241 lines) via safe `ALTER TABLE` statements. Each migration:

1. Checks if the column exists using `PRAGMA table_info`.
2. Adds the column if missing.
3. Seeds default data where needed (e.g., initial donation records).

For PostgreSQL, Drizzle's `db:push` command handles schema synchronization.

6.5 File System Usage

`downloads/` — Generated files (CSV, XLSX, JSON, ZIP, PNG, PDF). Timestamped filenames for uniqueness.

`public/plots/` — Generated matplotlib plots served as static assets. Timestamped PNG files with AutoScholar watermark.

`logs/` — Daily log files (`vibesurf-YYYY-MM-DD.log`).

`/tmp/` — Temporary directories for LaTeX compilation, Python script storage, and ZIP extraction. Cleaned up after use.

`local.db` — SQLite database file (with WAL journal).

6.6 Data Lifecycle

1. **Acquisition:** External APIs → JSON responses → tool results in Claude context.
2. **Processing:** Python subprocesses load, clean, merge, and analyze data.
3. **Intermediate storage:** CSV/JSON files in `downloads/`, figures in `public/plots/`.
4. **Paper integration:** Data summaries embedded in LaTeX sections, figures referenced.
5. **Packaging:** All artifacts bundled into the reproducible project ZIP.
6. **Publication:** Paper metadata and ZIP path stored in the database.
7. **Access:** Papers accessible via web UI, REST API, and repository browser.

Modeling and Scientific Computation Layer

7.1 Overview

The scientific computation layer is the analytical engine of AutoScholar, responsible for all quantitative analysis, model estimation, and figure generation. It consists of a sandboxed Python execution environment with 17 pre-installed scientific libraries, 7 dedicated analysis services, and a custom figure generation pipeline. All computation runs in short-lived Python subprocesses spawned by TypeScript bridge modules.

7.2 Python Execution Architecture

7.2.1 The TypeScript–Python Bridge Pattern

Every Python computation follows a consistent architectural pattern:

1. A TypeScript module (`*.ts`) defines input/output types, validates parameters, and generates displayable code.
2. The bridge spawns a Python subprocess (`*.py`) via `child_process.spawn`.
3. Input is passed as JSON via stdin.
4. Python executes the analysis and returns structured JSON via stdout.
5. The TypeScript module parses the output and returns typed results.

This pattern is replicated across 10+ service pairs:

Table 7.1: TypeScript–Python service pairs.

TypeScript Bridge	Python Service	Function
garch.ts	garchService.py	GARCH volatility modeling
varCalculator.ts	varService.py	Value at Risk
monteCarlo.ts	monteCarloService.py	Monte Carlo simulation
portfolioOptimizer.ts	portfolioOptimizer.py	Portfolio optimization
riskMetrics.ts	riskMetrics.py	Risk/performance metrics
optionsPricing.ts	optionsPricingService.py	Black-Scholes pricing
volatilitySurface.ts	volatilitySurfaceService.py	IV surface construction
plotExecutor.ts	plotService.py	Chart generation
dataDownload.ts	dataDownloadService.py	Data export
customPython.ts	customPythonExecutor.py	Arbitrary analysis

7.2.2 Custom Python Executor

The custom executor (customPythonExecutor.py, 1,774 lines) is the most powerful and complex Python service. It provides an unrestricted execution environment for Claude-generated Python code with:

Pre-Installed Libraries

- NumPy
- Pandas
- SciPy
- Matplotlib
- Scikit-learn
- Statsmodels
- XGBoost
- LightGBM
- ARCH (for GARCH)
- CVXPY
- Seaborn
- Plotly
- OpenPyXL
- python-docx
- DuckDB
- Requests
- BeautifulSoup

Utility Functions

save_excel() Creates styled multi-sheet XLSX workbooks with title rows, auto-column widths, conditional formatting (color scales, data bars), summary statistics (sum, average, min, max, count), and formula support.

save_word() Generates formatted DOCX documents with styled headings, tables, and paragraphs.

df_to_latex() Converts DataFrames to LaTeX table markup with proper formatting.

save_latex() Compiles LaTeX content into PDF with embedded figures.

Automatic Error Patching

The TypeScript bridge (customPython.ts, 371 lines) applies automatic fixes before execution:

Table 7.2: Automatic Python code patches.

Pattern	Fix
Deprecated pandas frequencies	`'M'` → `'ME'`, `'Q'` → `'QE'`, `'Y'` → `'YE'`
NumPy deprecations	`np.trapz` → `np.trapezoid`, remove `np.math`
Scikit-learn changes	t-SNE `n_iter` → `max_iter`
Matplotlib conflicts	Monkey-patch `tight_layout` for colorbar compatibility
fillna deprecation	`fillna(method='ffill')` → `ffill()`
Data fabrication	Flag random data generation without API calls

7.3 Dedicated Analysis Services

7.3.1 GARCH Volatility Modeling

`garchService.py` (224 lines) implements GARCH(p,q) volatility estimation:

$$\sigma_t^2 = \omega + \sum_{i=1}^{p} \alpha_i \varepsilon_{t-i}^2 + \sum_{j=1}^{q} \beta_j \sigma_{t-j}^2 \tag{7.1}$$

Implementation Details

1. Fetches historical prices from FMP API.
2. Calculates percentage returns: $r_t = 100 \times \frac{P_t - P_{t-1}}{P_{t-1}}$.
3. Fits a constant-mean GARCH model with normal distribution using the arch library.
4. Produces:
   - Conditional volatility time series.
   - Volatility forecasts (1–252 days ahead).
   - Model parameters (ω, α_i, β_j).
   - Persistence: $\sum \alpha_i + \sum \beta_j$.
   - AIC/BIC for model comparison.
   - Interpretive labels (mean-reverting, shock impact, clustering strength).

Validation

Input validation (`garch.ts`, 282 lines):

- $p, q \in [1, 5]$.
- Forecast horizon $\in [1, 252]$ days.
- Data period $\in [30, 2520]$ days ($\approx$ 10 years max).

7.3.2 Value at Risk (VaR)

`varService.py` (307 lines) implements four VaR methodologies:

Historical VaR

$$\text{VaR}_\alpha^{\text{hist}} = -\text{Quantile}(r_1, r_2, \ldots, r_T; \alpha) \tag{7.2}$$

Uses the empirical return distribution directly.

Parametric VaR

$$\text{VaR}_\alpha^{\text{param}} = -(\mu + z_\alpha \sigma) \tag{7.3}$$

where z_α is the standard normal quantile. Assumes normally distributed returns.

Monte Carlo VaR

$$\text{VaR}_\alpha^{\text{MC}} = -\text{Quantile}\left(\{r_i^{\text{sim}}\}_{i=1}^{N}; \alpha\right) \tag{7.4}$$

where $r_i^{\text{sim}} \sim \mathcal{N}(\hat{\mu}, \hat{\sigma}^2)$ with N simulations (default: 10,000).

Conditional VaR (Expected Shortfall)

$$\text{CVaR}_\alpha = \mathbb{E}\left[r \mid r < -\text{VaR}_\alpha\right] \tag{7.5}$$

The average loss beyond the VaR threshold.

Outputs

Results are computed at three confidence levels (90%, 95%, 99%) with both percentage and dollar amounts. Distribution statistics include skewness, kurtosis, and Jarque-Bera normality test. The system recommends the appropriate VaR method based on distribution characteristics.

7.3.3 Monte Carlo Simulation

`monteCarloService.py` (152 lines) performs investment projections:

$$P_{t+1} = P_t \times (1 + r_t), \quad r_t \sim \mathcal{N}(\hat{\mu}_{\text{daily}}, \hat{\sigma}^2_{\text{daily}}) \tag{7.6}$$

Features

- Two modes: symbol-based (auto-fetches from FMP) and manual (user-provided prices).
- Configurable simulations (1–100,000), time horizon, and initial investment.
- Output: mean/median/percentile terminal values, probability of profit, sample paths, distribution data.

7.3.4 Portfolio Optimization

`portfolioOptimizer.py` (434 lines) implements Markowitz Mean-Variance Optimization:

$$\max_{\mathbf{w}} \frac{\mathbf{w}^T \boldsymbol{\mu} - r_f}{\sqrt{\mathbf{w}^T \boldsymbol{\Sigma} \mathbf{w}}} \quad \text{s.t.} \quad \sum_i w_i = 1, \quad w_i \in [l_i, u_i] \tag{7.7}$$

Outputs

- Maximum Sharpe ratio portfolio weights.
- Minimum volatility portfolio weights.
- 50-point efficient frontier.
- Correlation matrix.
- Comparison against equal-weight benchmark.

Uses SciPy's SLSQP optimizer for constrained optimization.

7.3.5 Risk Metrics

`riskMetrics.py` (543 lines) computes 20+ performance and risk metrics:

Table 7.3: Risk and performance metrics.

Metric	Formula / Description
Annualized Return	$(1 + \bar{r}_{\text{daily}})^{252} - 1$
Annualized Volatility	$\sigma_{\text{daily}} \times \sqrt{252}$
Sharpe Ratio	$\frac{R_p - R_f}{\sigma_p}$
Sortino Ratio	$\frac{R_p - R_f}{\sigma_{\text{downside}}}$
Calmar Ratio	$\frac{R_p}{\text{MaxDrawdown}}$
Information Ratio	$\frac{R_p - R_b}{\sigma(R_p - R_b)}$
Treynor Ratio	$\frac{R_p - R_f}{\beta_p}$
Beta	$\frac{\text{Cov}(R_p, R_m)}{\text{Var}(R_m)}$
Alpha (Jensen's)	$R_p - [R_f + \beta(R_m - R_f)]$
R^2	From OLS of R_p on R_m
Max Drawdown	$\max_t \frac{\text{Peak}_t - \text{Value}_t}{\text{Peak}_t}$
Recovery Days	Days from max drawdown to recovery
Tracking Error	$\sigma(R_p - R_b)$
Upside Capture	$\frac{\bar{R}_p \mid R_m > 0}{\bar{R}_m \mid R_m > 0}$
Downside Capture	$\frac{\bar{R}_p \mid R_m < 0}{\bar{R}_m \mid R_m < 0}$
Win Rate	Fraction of positive return days
Profit Factor	$\frac{\sum \text{gains}}{\sum \lvert\text{losses}\rvert}$

Uses SPY as the default benchmark with date-aligned return series.

7.3.6 Options Pricing

`optionsPricingService.py` (238 lines) implements Black-Scholes for European options:

$$C = S\Phi(d_1) - Ke^{-rT}\Phi(d_2) \tag{7.8}$$
$$P = Ke^{-rT}\Phi(-d_2) - S\Phi(-d_1) \tag{7.9}$$
$$d_1 = \frac{\ln(S/K) + (r + \sigma^2/2)T}{\sigma\sqrt{T}} \tag{7.10}$$
$$d_2 = d_1 - \sigma\sqrt{T} \tag{7.11}$$

Greeks

$$\Delta_C = \Phi(d_1), \quad \Delta_P = \Phi(d_1) - 1 \tag{7.12}$$
$$\Gamma = \frac{\phi(d_1)}{S\sigma\sqrt{T}} \tag{7.13}$$
$$\Theta_C = -\frac{S\phi(d_1)\sigma}{2\sqrt{T}} - rKe^{-rT}\Phi(d_2) \tag{7.14}$$
$$\mathcal{V} = S\sqrt{T}\phi(d_1) \tag{7.15}$$
$$\rho_C = KTe^{-rT}\Phi(d_2) \tag{7.16}$$

Implied Volatility

Computed via Newton-Raphson iteration:

$$\sigma_{n+1} = \sigma_n - \frac{C(\sigma_n) - C_{\text{market}}}{\mathcal{V}(\sigma_n)} \tag{7.17}$$

7.3.7 Volatility Surface

`volatilitySurfaceService.py` (378 lines) constructs 3D implied volatility surfaces:

1. Fetches option chain from Financial Data API.
2. Intelligently samples up to 60 contracts across strikes and expirations.
3. Retrieves implied volatility for each contract.
4. Interpolates a smooth surface using SciPy's cubic/linear `griddata`.
5. Renders a 3D matplotlib plot with scatter points for actual data.

7.4 Scientific Computing Libraries

7.4.1 Econometric Modeling with Statsmodels

Used for:

- OLS, WLS, GLS regression.

- Panel data models (fixed effects, random effects).
- Time series models (ARIMA, VAR, VECM).
- Hypothesis testing (t-tests, F-tests, Jarque-Bera, ADF, KPSS).
- Robust standard errors (Newey-West, clustered).

7.4.2 Machine Learning with Scikit-learn

Used for:

- Classification and regression models.
- Cross-validation and hyperparameter search.
- Feature selection and dimensionality reduction.
- Clustering and anomaly detection.
- SHAP value integration for interpretability.

7.4.3 Gradient Boosting with XGBoost/LightGBM

Used for:

- High-performance predictive modeling.
- Feature importance ranking.
- Early stopping and regularization.

7.4.4 Convex Optimization with CVXPY

Used for:

- Portfolio optimization beyond mean-variance (CVaR, risk parity).
- Robust optimization problems.
- Constrained optimization with complex constraints.

7.4.5 Volatility Modeling with ARCH

Used for:

- GARCH, EGARCH, GJR-GARCH models.
- Multivariate GARCH (DCC, BEKK).
- Volatility forecasting.

7.5 Figure Generation Pipeline

7.5.1 Style Configuration

`matplotlib_config.py` (79 lines) sets the publication-quality defaults:

- **Fonts:** DejaVu Serif, Times New Roman (serif family).
- **Colors:** 8-color muted palette (navy, crimson, forest green, brown, purple, teal, olive, charcoal).
- **Layout:** Inward ticks, no top/right spines, print-safe grid (alpha 0.25).
- **Resolution:** 150 DPI for display, 300 DPI for save.
- **Layout engine:** Constrained layout (avoids `tight_layout` conflicts).

7.5.2 Figure Capture

`figure_capture.py` (104 lines) processes all open matplotlib figures:

1. Iterates over `plt.get_fignums()`.
2. For each figure, checks if it is empty using two-stage analysis:
   - **Stage 1:** Inspects axes for artists (lines, patches, images, collections, texts, tables).
   - **Stage 2:** Renders to image and measures pixel variance (threshold: 50.0).
3. Non-empty figures receive the AutoScholar watermark (teal, bottom-right).
4. Figures are encoded as base64 PNG strings.
5. All figures are closed to free memory.

7.5.3 Vision Analysis

`figureAnalysisService.ts` (113 lines) optionally sends figures to Claude Opus for vision-based analysis:

- Chart type identification.
- Data description.
- Key trend identification.
- Notable value extraction.
- Parallel processing via `Promise.all`.

7.6 Script Management and Repair

7.6.1 Script Lifecycle

The ScriptManager (`scriptManager.ts`, 276 lines) tracks Python scripts through their lifecycle:

1. **Save:** Script content saved to session-scoped temp directory with unique ID.
2. **Execute:** Script run via the Python subprocess.
3. **Error capture:** Structured error parsing via the StructuredLogger.

4. **Repair:** Claude reads the script and errors, generates a patch.
5. **Patch:** DiffEngine applies the patch (up to 8 attempts).
6. **Re-run:** Patched script re-executed.
7. **Cleanup:** Session scripts cleaned up on session end or after 24 hours.

7.6.2 Diff Engine

The DiffEngine (`diffEngine.ts`, 400 lines) supports three patching strategies:

Strategy 1: Standard unified diff with fuzzy matching (up to 50-line offset tolerance). Parses `@@` hunk headers and applies line-by-line.

Strategy 2: Search-replace blocks in three LLM-friendly formats:

- Marker-based: `«« SEARCH / ==== / »» REPLACE`
- Labeled format with code blocks.
- Raw `-/+` line extraction.

Strategy 3: Trimmed/stripped matching as a last-resort fallback that ignores whitespace differences.

7.6.3 Structured Error Parsing

The StructuredLogger (`structuredLogger.ts`, 145 lines) converts Python tracebacks into structured JSON:

Listing 7.1: Structured error output format.

```
interface StructuredError {
  type: string;       // "KeyError", "ImportError", etc.
  message: string;
  file: string;
  line: number;
  traceback: string;
}
```

This structured format enables the GOVERNOR to classify errors by type and generate pattern-specific recovery advice.

8 1

Paper Generation Engine

8.1 Overview

The paper generation engine transforms research analysis into compiled LaTeX PDFs. It encompasses template selection, section writing, content sanitization, multi-pass compilation, iterative error repair, and reproducible project packaging. The engine is distributed across 6 modules in `server/services/latex/` totaling approximately 2,870 lines.

8.2 Text Generation

8.2.1 Section-by-Section Writing

Papers are written one section at a time via the `write_paper_section` tool. Claude receives detailed instructions for each section type, including minimum word counts, required subsections, and specific content expectations.

Empirical Paper Sections

Introduction (untitled per JF convention): Must present the research question, motivate its importance, preview the methodology, summarize key findings, and outline the paper's contribution relative to existing literature. Target: 2,000–3,000 words.

Literature Review: Organized thematically (not chronologically). Must cover 15–30 papers, identify research gaps, and position the paper. Target: 3,000–5,000 words.

Methodology: Formal model specification with numbered equations, estimation strategy, identification approach, and assumptions. Must include economic intuition for each variable. Target: 3,000–5,000 words.

Data: Source description, sample construction, variable definitions, summary statistics table, preliminary analysis (correlation matrix, unit root tests). Target: 2,000–4,000 words.

Results: Main regression tables, robustness checks (alternative specifications, subsamples, placebo tests), figure interpretation. Target: 4,000–7,000 words.

Discussion: Economic interpretation, comparison with existing literature, implications for practitioners and policymakers. Target: 2,000–3,000 words.

Conclusion: Summary of contributions, limitations, and directions for future research. Target: 1,000–2,000 words.

Section Rejection and Retry

The tool executor tracks section rejection attempts. If a section is rejected more than 3 times (typically due to insufficient length or quality), the system accepts the best available version to prevent infinite loops.

8.2.2 Citation Handling

Citations are managed through BibTeX:

1. During the GAUSS phase, academic search results include auto-generated BibTeX entries.
2. Claude accumulates citations throughout the paper, using `\cite{key}` and `\citep{key}` commands.
3. The BibTeX file is assembled from all accumulated entries.
4. `natbib` with author-year style handles citation formatting.
5. Duplicate cite keys are resolved by appending letters (e.g., `smith2020a`, `smith2020b`).

8.2.3 Figure Integration

Figures are referenced in LaTeX via:

Listing 8.1: Typical figure inclusion pattern.

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\textwidth]{fig_01.png}
  \caption{Conditional volatility estimates from GARCH(1,1) model.}
  \label{fig:garch-vol}
\end{figure}
```

Figure files are extracted from base64-encoded Python execution results and written to the compilation directory.

8.3 LaTeX Templates

8.3.1 Common Preamble

The shared preamble (`autoScholarPreamble()`, ~200 lines) includes 50+ packages:

- amsmath
- amssymb
- amsthm
- mathtools
- geometry
- graphicx
- booktabs
- longtable
- multirow
- float
- natbib
- hyperref
- cleveref
- caption
- subcaption
- enumitem
- microtype
- setspace

- algorithm2e
- adjustbox
- threeparttable
- listings
- rotating
- xcolor
- dcolumn

8.3.2 Article Template

The default template (`buildArticleTemplate()`):

- 12pt, A4, one-column.
- 1-inch margins with 1.5 line spacing.
- Running headers with abbreviated title.
- Numbered sections, Roman numeral tables (per JF convention).
- Author block: "Euler — The AutoScholar Research Agent."
- Agent disclosure section in footnotes.

8.3.3 Working Paper Template

`buildWorkingPaperTemplate()`:

- Single-column with wider margins.
- Working paper header.
- Draft watermark option.
- Otherwise identical preamble.

8.3.4 Econometrica Template

`buildEconometricaTemplate()`:

- Two-column layout mimicking *Econometrica* style.
- Serif fonts, narrow margins.
- Custom theorem environments (Assumption, Proposition, Theorem, Corollary).

8.4 Content Sanitization

The sanitizer (`latex/sanitizer.ts`, 670 lines) addresses common issues in LLM-generated LaTeX:

8.4.1 Special Character Escaping

Table 8.1: LaTeX special character handling.

Character	Treatment
`%`	Escaped to `\%` outside comments
`&`	Escaped to `\&` outside tabular environments
`$`	Context-aware (preserved in math mode, escaped outside)
`#`	Escaped to `\#`
`_`	Escaped to `\_` outside math mode
Greek letters	Wrapped in `$...$` if found outside math mode
Em/en dashes	Normalized to `--`/`-`

8.4.2 Table Auto-Fitting

Wide tables are automatically handled by three progressive strategies:

1. **Column counting:** Tables with >6 columns are flagged.
2. **`adjustbox` wrapping:** First attempt wraps the table in `\begin{adjustbox}{max width=\textwidth}`.
3. **`resizebox` fallback:** If `adjustbox` fails, uses `\resizebox{\textwidth}{!}`.
4. **Safety wrapping:** Final pass adds overflow protection to all remaining tables.

8.4.3 Blank Figure Detection

`isBlankPng()` detects empty figures at the binary level:

1. Reads the PNG file and locates IDAT (image data) chunks.
2. Decompresses the raw pixel data via zlib inflate.
3. Strips PNG filter bytes (one per row).
4. Computes pixel variance across the image.
5. If variance < 50.0, the figure is classified as blank and excluded.

8.5 Compilation Pipeline

8.5.1 Binary Discovery

The system searches for `pdflatex` and `bibtex` across multiple macOS installation paths:

- `/Library/TeX/texbin/`
- `/usr/local/texlive/2024/bin/universal-darwin/`
- `/usr/texbin/`
- System PATH

8.5.2 Multi-Pass Compilation

Standard academic compilation requires multiple passes:

1. **Pass 1:** `pdflatex` — Initial compilation, generates `.aux` files.
2. **Pass 2:** `bibtex` — Processes `.aux` to resolve citations from `.bib`.
3. **Pass 3:** `pdflatex` — Re-processes with resolved citations.
4. **Pass 4:** `pdflatex` — Final pass to resolve all cross-references.

All passes run with `-interaction=nonstopmode` to prevent interactive prompts.

8.5.3 Error Classification

The LaTeX log parser (`latexLogParser.ts`, 146 lines) classifies errors into 8 types:

Undefined control sequence: Misspelled commands or missing packages.

Math errors: Mismatched delimiters, missing $ signs.

Unicode errors: Non-ASCII characters in text mode.

Environment errors: Mismatched `\begin`/`\end` pairs.

Package errors: Package conflicts or missing options.

Fatal errors: Emergency stops, capacity exceeded.

Missing characters: Font encoding issues.

Other: Unclassified errors.

Additionally tracks overfull/underfull box warnings (hbox/vbox).

8.5.4 Iterative Repair Loop

When compilation fails, the LaTeX Patch Manager (`latexPatchManager.ts`, 369 lines) orchestrates repair:

Algorithm 5: LaTeX Iterative Repair

Input: Failed compilation, max attempts = 8
Output: Compiled PDF or final failure

```
Register failed compilation metadata;
for i ← 1 to 8 do
    errors ← inspectLatexErrors()();
    Format errors for Claude with line numbers and context;
    Claude generates a diff patch;
    result ← patchLatex()(diff);
    if patch failed then
        Try search-replace fallback;
        Try trimmed matching fallback;
    end
    pdf ← recompileLatex()();
    if compilation succeeded then
        return pdf;
    end
end
return failure with last error log;
```

8.6 Document Export

Beyond LaTeX PDF, the system supports three additional export formats:

8.6.1 PDF via Puppeteer

`pdf/puppeteerPdf.ts` (75 lines):

1. Converts markdown to HTML with KaTeX math rendering.
2. Loads in headless Chromium.
3. Waits for math rendering completion.
4. Generates A4 PDF with AutoScholar-branded footer.

8.6.2 DOCX via Pandoc

`pdf/pandocDocx.ts` (50 lines):

1. Writes markdown to temp file.
2. Invokes `pandoc -f markdown -t docx`.
3. Returns DOCX buffer.

8.6.3 HTML Export

`pdf/htmlExport.ts` (9 lines):

- Self-contained HTML with embedded CSS, KaTeX, and syntax highlighting.

- Responsive design with print-optimized CSS.
- AutoScholar branding with embedded SVG logo.

8.7 Reproducible Project Packaging

The packaging module (`latex/packaging.ts`, 358 lines) creates ZIP archives following the RRE structure:

8.7.1 Directory Structure

```
project/
  paper.tex               Main LaTeX source
  references.bib          BibTeX references
  outputs/
    paper.pdf             Compiled PDF
    figures/              All generated figures (PNG)
  code/
    01_data_download.py   Pipeline-ordered scripts
    02_data_prep.py
    03_analysis.py
    ...
  data/
    raw/api/              API-sourced data (CSV, JSON)
    raw/web/              Web-extracted data
    processed/            Cleaned datasets
    metadata/             Data provenance (JSON sidecars)
  run_pipeline.py         Auto-generated master script
  requirements.txt        Auto-detected dependencies
  README.md               Build instructions
```

8.7.2 Automatic Dependency Detection

The `requirements.txt` generator:

1. Scans all Python scripts for `import` and `from ... import` statements.
2. Maps module names to pip package names (e.g., `sklearn` → `scikit-learn`, `cv2` → `opencv-python`).
3. Deduplicates and sorts alphabetically.
4. Excludes standard library modules.

8.7.3 Master Pipeline Script

`run_pipeline.py` is auto-generated with:

1. Sequential execution of all `code/*.py` scripts in numeric order.
2. Error handling with script-level try/catch.

3. Summary output listing successful/failed scripts.
4. Working directory setup for data and output paths.

Memory and State System

9.1 Overview

AutoScholar operates with a multi-layered state management system that spans the lifetime of individual sessions, pipeline runs, and the entire platform. Unlike traditional agent systems with explicit long-term memory stores, AutoScholar relies primarily on the LLM's context window as its "working memory" and the database as its persistent state.

This chapter documents how state is managed, how previous work influences new papers, and what mechanisms exist for avoiding duplication.

9.2 Context Window as Working Memory

9.2.1 The 1M Token Context

Claude Opus 4.6's one-million-token context window serves as the primary working memory during paper generation. Within a single pipeline run, all of the following accumulate in the context:

- The system prompt (1,798 lines, ∼30K tokens).
- The research prompt (enhanced, 400–700 words).
- Literature search results (15–30 papers with abstracts and BibTeX).
- Data collection results (API responses, dataset summaries).
- Python code and execution outputs (scripts, figures, errors).
- Written paper sections (LaTeX content).
- Tool results and governor directives.
- Conversation history (all previous turns).

9.2.2 Token Budget Management

The system manages the context budget through two mechanisms:

Tool result truncation. Results exceeding 15,000 characters are truncated with a "[truncated]" marker. This is applied selectively—larger results (e.g., full API response dumps) are truncated while smaller results are preserved in full.

History summarization. When estimated message tokens exceed the 500K safety threshold (half the 1M window), the system calls Claude to compress older conversation history:

1. The first 50% of messages are selected for summarization.
2. Claude generates a concise summary preserving key findings, data sources, and methodology decisions.
3. The original messages are replaced with a single summary message.
4. This frees context space for the remaining pipeline stages.

9.2.3 Implications

Within a single paper generation run, the system has excellent "memory"—it can reference any earlier finding, dataset, or decision because all information remains in context. However, there is no memory *across* runs: each paper starts with a fresh context, and the system cannot learn from previous papers' successes or failures.

9.3 Session State

9.3.1 Conversation Sessions

Each interactive chat or pipeline run is tracked as a conversation session:

`conversationSessions` table: Stores session metadata including user ID, session ID, title, message count, token usage, cost, and timestamps.

`messages` table: Stores individual messages with role (user/assistant), content, serialized sources, and timestamps.

9.3.2 Real-Time State Broadcasting

The SessionBroadcaster (`sessionBroadcaster.ts`, 180 lines) manages real-time state for live monitoring:

- Maintains per-session event buffers (last 200 events).
- Manages subscriber sets for SSE streaming.
- Tracks session activity with last-heartbeat timestamps.
- Supports reconnection with buffered event replay (catch-up for disconnected clients).
- Automatic cleanup of sessions idle for 30 minutes or older than 1 hour.

9.3.3 Session Persistence

Session data is persisted every 8 seconds during active generation:

- Messages (question-answer pairs).
- Tool results (structured JSON).

- Figures (base64-encoded or URL references).
- Token counts and cost accumulation.
- Agent step progress.

9.4 Pipeline State

9.4.1 Governor State

The GOVERNOR maintains typed state throughout a pipeline run (see Section 3.7). This state includes:

- Current pipeline stage.
- Active prompt profile.
- Error counts (Python, LaTeX, data).
- Retry budgets (3/3/2 for Python/LaTeX/data).
- Confidence score (0.0–1.0).
- Figure and word counts.
- Warning accumulator.
- Publication verdict.

Governor state is serialized to JSON and stored in the `paper_runs.governor_state` column, enabling:

- Post-mortem analysis of failed runs.
- Checkpoint resume with full governor state recovery.
- Historical analysis of governor intervention patterns.

9.4.2 Decision Log

All governor decisions are recorded in the DecisionLog with:

- UUID per decision.
- Timestamp.
- Pipeline stage.
- Trigger description.
- Action taken.
- Reasoning.
- Confidence delta.
- Profile transition (if any).

The decision log persists in both the `paper_runs` table (serialized JSON) and the `governor_decisions` table (individual records).

9.4.3 Checkpoint State

The checkpoint system captures a snapshot of the entire pipeline state:

- Full conversation history (all Claude messages).
- Governor state and decision log.
- Tool results accumulated so far.
- Written paper sections.
- Figure count and metadata.
- Pipeline stage at checkpoint time.

Checkpoints enable crash recovery: after a server restart, an administrator can resume a run from its last checkpoint.

9.5 Cross-Paper Memory

9.5.1 Current State: No Cross-Paper Memory

AutoScholar currently has **no mechanism for cross-paper memory**. Each paper generation run starts with a fresh context window, and the system has no awareness of:

- Topics previously covered.
- Methods previously applied.
- Data sources previously used.
- Findings from previous papers.
- Reviewer feedback from previous runs.

9.5.2 Implications

This absence has several consequences:

Topic duplication. While the sequential prompt selection ensures topic diversity, there is no mechanism to prevent the system from revisiting a topic if the prompt bank contains similar entries or if prompts are selected non-sequentially.

No learning from failures. If a particular data source consistently fails or a method doesn't converge for certain data types, the system cannot learn to avoid these combinations.

No citation of own work. The system cannot cite its own previously published papers, even when they would be relevant.

No methodological improvement. The system cannot improve its analysis techniques based on reviewer feedback from previous runs.

9.5.3 Implicit Memory via Prompt Bank

The prompt bank provides a form of implicit memory:

- Sequential selection ensures each prompt is used exactly once before cycling.
- The domain-method compatibility matrix prevents nonsensical combinations.
- Topic diversity is built into the bank's construction (14 topics per domain, interleaved iteration order).

However, this is "memory" in the weakest sense—it is pre-programmed diversity, not learned adaptation.

9.6 Platform State

9.6.1 Site Configuration

The `site_config` table provides a key-value store for runtime configuration:

- Scheduler settings (cadence, parallel limit, thresholds).
- Custom system prompt overrides.
- Feature flags.
- Cost tracking baselines.

Values persist across server restarts and are modifiable via the admin API.

9.6.2 Analytics Accumulation

Platform-wide analytics accumulate over time:

- Total papers generated (published + rejected).
- Total tokens consumed and API cost.
- Per-domain paper counts and quality scores.
- Monthly publication timelines.
- Python execution counts and code line totals.

9.6.3 Published Paper Corpus

The published paper corpus serves as the platform's "institutional memory":

- Full LaTeX source and BibTeX for every published paper.
- Peer review scores and feedback.
- Domain classification and methodology tags.
- Reproducible project archives.

While this corpus is not currently used to inform new paper generation, it represents a rich dataset that could power future cross-paper learning (see Chapter 14).

9.7 State Consistency and Recovery

9.7.1 Crash Recovery

State consistency after crashes is maintained through:

1. **8-second checkpoint saves:** Session state is persisted frequently enough to minimize data loss.
2. **Heartbeat monitoring:** Stalled runs are detected within 5 minutes and marked as failed.
3. **Force-fail on restart:** All active runs are force-failed on server restart to prevent zombie processes.
4. **Manual resume:** Administrators can resume runs from checkpoints via the admin API.

9.7.2 Data Integrity

- SQLite uses WAL (Write-Ahead Logging) mode for crash-safe writes.
- PostgreSQL provides full ACID guarantees.
- ASID generation uses atomic database operations to prevent collision under concurrent access.
- File operations use temporary directories with rename-on-success to prevent partial writes.

Failure Handling and Recovery

10.1 Overview

AutoScholar operates in a complex environment where failures are expected at every layer: external APIs may be unavailable, Python code may crash, LaTeX compilation may fail, and the LLM itself may produce malformed output. The system employs a defense-in-depth strategy with multiple failure handling mechanisms.

10.2 Claude API Failure Handling

10.2.1 Retry with Exponential Backoff

The streaming function `callClaudeStreamingWithRetry()` (`claudeStreaming.ts`) implements a robust retry strategy:

Algorithm 6: Claude API Retry Strategy

Input: Query, messages, max attempts = 5

```
for attempt ← 1 to 5 do
    result ← callClaudeStreaming()(query, messages);
    if result is success then
        return result;
    if error is 429 (rate limit) or 529 (overloaded) then
        delay ← 2^attempt × 1000 ms;
        sleep(delay);
        notify client via retry SSE event;
    else if error is timeout or connection reset then
        delay ← 2^attempt × 1000 ms;
        sleep(delay);
    else
        throw error (non-retryable);
    end
end
throw MaxRetriesExceeded;
```

10.2.2 Transient Error Categories

Table 10.1: Claude API transient error handling.

Error	Handling	Max Retries
HTTP 429 (Rate Limit)	Exponential backoff, notify client	5
HTTP 529 (Overloaded)	Exponential backoff, notify client	5
Connection timeout	Exponential backoff	5
Connection reset	Exponential backoff	5
HTTP 500 (Server Error)	Not retried (treated as fatal)	0
HTTP 400 (Bad Request)	Not retried (likely prompt issue)	0

10.2.3 Token Overflow Prevention

Before sending messages to Claude, the system:

1. Estimates total token count using the 4-char/token heuristic.
2. If tokens $>$ 500K, triggers history summarization.
3. Truncates individual tool results $>$ 15K characters.
4. If summarization fails, drops the oldest messages until under budget.

10.3 Python Execution Failure Handling

10.3.1 Pre-Execution Patching

The TypeScript bridge applies automatic patches to prevent known failure patterns:

Deprecated APIs: Pandas frequency aliases, NumPy functions, scikit-learn parameter names.

Library conflicts: Matplotlib `tight_layout` with colorbars.

Data fabrication: Detection and warning for random data generation without API calls.

10.3.2 Error Classification

The StructuredLogger (`structuredLogger.ts`) classifies Python errors into categories:

Table 10.2: Python error classification.

Error Type	Governor Response	Recovery
`ModuleNotFoundError`	Install package via `install_python_package` tool	Automatic
`MemoryError`	Reduce data size, simplify analysis	Profile switch
`KeyError` / `IndexError`	Inspect data schema, fix column references	Script patch
`ConvergenceWarning`	Simplify model, adjust parameters	Profile switch
`LinAlgError` (Singular)	Check multicollinearity, drop variables	Script patch
`SyntaxError`	Fix code via patch	Script patch
`ValueError`	Context-dependent recovery	Script patch
`TimeoutError`	Reduce computation scope	Profile switch

10.3.3 Script Repair Loop

When a Python script fails:

1. The StructuredLogger parses the traceback into structured JSON.
2. The error is formatted for Claude with context (script content, line numbers, error type).
3. Claude reads the script via `read_python_script`.
4. Claude generates a patch via `apply_patch`.
5. The DiffEngine applies the patch (trying 3 strategies: unified diff, search-replace, fuzzy matching).
6. The patched script is re-executed via `run_python_script`.
7. If the error persists, the cycle repeats (up to 8 attempts).

10.3.4 Retry Budget System

The GOVERNOR enforces a Python retry budget of 3 per pipeline run:

1. Each Python error decrements the budget.
2. Each successful execution recovers 0.05 confidence.
3. When the budget reaches 0, the GOVERNOR emits `force_finalize`, instructing Claude to skip remaining analysis and proceed to writing with available results.
4. This prevents infinite error loops from consuming the entire pipeline run.

10.4 LaTeX Compilation Failure Handling

10.4.1 Error Detection

The LaTeX log parser (`latexLogParser.ts`) detects errors by:

1. Scanning the log file for `!` error markers.

2. Extracting line numbers from `l.N` patterns.
3. Classifying errors into 8 types.
4. Counting overfull/underfull box warnings.
5. Extracting offending commands.

10.4.2 Compilation Fallback Strategies

Primary: Full pdflatex + bibtex compilation (4 passes).

Fallback 1: pdflatex only (no bibliography, 1 pass).

Fallback 2: Simplified LaTeX (stripped packages, basic formatting).

Fallback 3: Direct PDF generation via Puppeteer (markdown → HTML → PDF).

10.4.3 Repair Budget

The Governor enforces a LaTeX retry budget of 3. When exhausted, the `latex_fallback` profile restricts the system to basic LaTeX commands, avoiding complex packages and environments.

10.5 External API Failure Handling

10.5.1 Rate Limiting

Semantic Scholar: Exponential backoff on 429 responses (1s, 2s, 4s delays, max 3 retries).

FMP: No retry; relies on daily quota management (manual).

Firecrawl: Async job polling with 3s intervals, 120s max wait.

Tavily: Standard HTTP retry.

10.5.2 Data Source Fallback Chains

The system prompt defines fallback chains for data acquisition:

1. Stock data: FMP → EODHD → web scraping.
2. Macro data: FRED → World Bank → IMF → OECD.
3. Academic data: Multi-source parallel (all 5 databases simultaneously).
4. Web data: Tavily → Firecrawl → Exa.

If all sources fail for a required dataset, the Governor detects a data quality issue and may switch to the `conservative` profile with relaxed data requirements.

10.5.3 Timeout Configuration

Table 10.3: External API timeout configuration.

Service	Timeout
FMP API	60s
EODHD API	30s
Semantic Scholar	15s
Exa (search)	15s
Exa (content)	20s
Firecrawl (sync)	30s
Firecrawl (async)	120s
Tavily (search)	30s
Tavily (research)	120s
World Bank	20s
IMF	20s
OECD	30s
Polymarket	15s
Python subprocess	90s

10.6 Pipeline-Level Failure Handling

10.6.1 Heartbeat Monitoring

Each active run updates its heartbeat every 10 seconds. The scheduler monitors heartbeats and marks runs as `failed` if no update is received within the configured timeout (default 300s).

10.6.2 Consecutive Failure Detection

The scheduler tracks consecutive failures:

- After `maxConsecutiveFailures` (default: 5) consecutive failures, the scheduler auto-pauses to prevent resource waste.
- An administrator must investigate and re-enable the scheduler.

10.6.3 Server Restart Recovery

On server startup:

1. All active runs (status $\in$ {queued, researching, analyzing, drafting, validating}) are force-failed.
2. This prevents zombie processes from holding resources.
3. Administrators can resume specific runs from checkpoints if needed.

10.6.4 Stale Cleanup

Periodic cleanup runs every 30 minutes for:

- Stale LaTeX compilations (temp directories older than 1 hour).
- Stale session broadcaster entries (no activity for 30 minutes).
- Orphaned paper runs (active status but no heartbeat for > timeout).

10.7 LLM Output Failure Handling

10.7.1 Malformed JSON

The 6-strategy JSON parser (`parseJsonResponse()` in `literatureGapFinderService.ts`) handles:

1. Direct `JSON.parse`.
2. Strip markdown code fences.
3. Regex extraction from mixed text.
4. Fix common issues (trailing commas, single quotes).
5. Partial JSON extraction (last valid closing bracket).
6. Fallback with raw text as the content field.

10.7.2 Tool Call Failures

If Claude issues a tool call with invalid parameters:

1. The tool executor catches the error.
2. An error result is returned to Claude with the error message.
3. Claude can retry with corrected parameters.
4. The Governor tracks the failure and may intervene.

10.7.3 Infinite Loop Prevention

Several mechanisms prevent infinite loops:

- **Tool call depth limit:** 180 for auto-paper, 45 for interactive chat.
- **Section rejection limit:** 3 rejections per section before accepting the best version.
- **Script repair limit:** 8 patch attempts per script.
- **Retry budgets:** 3/3/2 for Python/LaTeX/data before force-finalize.
- **Heartbeat timeout:** 5 minutes of inactivity triggers failure.

10.8 Error Reporting and Observability

10.8.1 Logging

Daily log files (`logs/vibesurf-YYYY-MM-DD.log`) capture:

- HTTP request logs (method, path, status, duration, user agent).
- Pipeline progress events.
- Error traces with stack traces.
- Governor decision records.

10.8.2 SSE Error Events

Clients receive real-time error notifications via SSE `error` events, including:

- Error messages.
- Retry notifications with attempt counts and delays.
- Pipeline stage transitions.
- Final failure reasons.

10.8.3 Admin Dashboard

The admin interface provides:

- Real-time pipeline run status with governor state.
- Historical run analysis with failure reasons.
- Governor decision logs with intervention counts.
- System health metrics (active runs, error rates, resource usage).

Performance and Scalability

11.1 Overview

AutoScholar's performance characteristics are dominated by two factors: (1) the latency and throughput of the Claude API, and (2) the execution time of Python computational services. Network I/O to external data APIs and LaTeX compilation contribute smaller but non-negligible overhead.

11.2 Latency Analysis

11.2.1 End-to-End Pipeline Latency

A typical auto-paper pipeline run takes 45–90 minutes, broken down approximately as:

Table 11.1: Pipeline latency breakdown (typical).

Phase	Duration	Percentage
Literature review (Gauss)	5–10 min	10%
Data collection (Turing)	8–15 min	15%
Data preparation (Newton)	5–10 min	10%
Empirical analysis (Fisher)	15–30 min	30%
Paper writing (Euler write)	10–20 min	25%
Compilation (Euler compile)	2–5 min	5%
Peer review (Popper)	2–3 min	5%
Total	**45–90 min**	**100%**

11.2.2 Claude API Latency

Each Claude API call involves:

- **Time to first token:** 1–5 seconds (depends on input size).
- **Token generation rate:** 50–100 tokens/second for output.
- **Tool call overhead:** 0.5–2 seconds per tool call for parsing and dispatch.

With 80–180 tool calls per run and substantial input context (approaching 1M tokens by end), Claude API latency dominates total runtime.

11.2.3 Python Execution Latency

Individual Python executions vary significantly:

Table 11.2: Python service execution times.

Service	Typical Duration
Custom Python (simple)	5–15 seconds
Custom Python (complex)	30–90 seconds
GARCH estimation	10–30 seconds
Monte Carlo (10K sims)	5–15 seconds
Portfolio optimization	10–20 seconds
Risk metrics	5–15 seconds
Options pricing	3–10 seconds
Volatility surface	30–60 seconds
Data download	5–30 seconds
Plot generation	3–10 seconds

Python subprocess startup adds 1–3 seconds of overhead per invocation (interpreter initialization, library imports).

11.2.4 External API Latency

Table 11.3: External API response times.

Service	Typical Response Time
FMP (simple query)	200–500ms
FMP (historical data)	500ms–2s
EODHD (EOD data)	300ms–1s
Semantic Scholar	500ms–3s
OpenAlex	300ms–1s
arXiv	1–5s (XML parsing)
Tavily (search)	1–3s
Firecrawl (scrape)	2–10s
World Bank	500ms–2s

11.3 Throughput

11.3.1 Paper Generation Rate

Single pipeline: 1 paper every 45–90 minutes, or ∼16–32 papers/day.

Maximum parallel (3): 3 concurrent pipelines, yielding ∼48–96 papers/day theoretically. In practice, 24–48 papers/day due to API rate limits and shared compute.

11.3.2 Bottlenecks

1. **Claude API:** The single largest bottleneck. Each pipeline run maintains a long-running streaming connection, and concurrent runs multiply API cost linearly.
2. **Python subprocesses:** Each execution spawns a new Python interpreter. No process pooling or caching is implemented.
3. **FMP API quotas:** Daily request limits can throttle data collection in high-throughput scenarios.
4. **Single-threaded Node.js:** The Express server runs on a single event loop. CPU-intensive operations (JSON parsing, LaTeX sanitization) block the event loop briefly.

11.4 Memory Usage

11.4.1 Node.js Process

- Baseline memory: ~150–250 MB.
- Per active session: ~10–50 MB (SSE buffers, tool results, governor state).
- Peak during compilation: ~300–500 MB (large tool result arrays, LaTeX content).
- Session broadcaster event buffers: 200 events × average 5KB = ~1 MB per session.

11.4.2 Python Subprocesses

- Baseline: ~100–200 MB per process (NumPy, Pandas, Matplotlib imports).
- Peak during analysis: ~500 MB–2 GB depending on dataset size and model complexity.
- DuckDB databases: loaded on-demand, can consume significant memory for large queries.

11.4.3 Database

- SQLite file: grows with published papers and sessions. Current size: ~458 MB.
- WAL journal: up to ~12 MB during active writes.
- In-memory structures: Drizzle ORM caches schema metadata.

11.5 Compute Constraints

11.5.1 API Cost Model

The dominant cost is Claude API usage:

Table 11.4: API cost per paper (Claude Opus 4.6).

Component	Tokens	Cost
Input tokens (context)	500K–1M	$2.50–$5.00
Output tokens (responses)	100K–300K	$2.50–$7.50
Prompt enhancement	5K–10K	$0.05–$0.10
Peer review (Popper)	20K–50K	$0.25–$0.50
History summarization	50K–100K	$0.50–$1.00
Total per paper		**$5–$15**
Daily (24 papers)		**$120–$360**
Monthly (720 papers)		**$3,600–$10,800**

11.5.2 External API Costs

- **FMP:** Subscription-based with daily request limits. Enterprise plan recommended for high throughput.
- **EODHD:** Pay-per-request model.
- **Tavily / Firecrawl / Exa:** Usage-based pricing.
- **SerpAPI:** Per-search pricing.
- **Free APIs:** World Bank, IMF, OECD, OpenAlex, arXiv, CrossRef, Polymarket — no cost.

11.6 Scalability Assessment

11.6.1 Horizontal Scaling Limitations

The current architecture has limited horizontal scalability:

- **Single-process Node.js:** Cannot leverage multiple CPU cores natively.
- **SQLite:** Single-writer constraint limits write concurrency. (PostgreSQL removes this limitation.)
- **File-based state:** Script management, LaTeX compilation, and figure storage use the local filesystem.
- **In-memory state:** Session broadcaster, rate limiter, and governor state are in-process.

11.6.2 Vertical Scaling Opportunities

- **More memory:** Allows larger Python datasets and more concurrent sessions.
- **Faster CPU:** Reduces Python startup time and JSON processing overhead.
- **SSD storage:** Improves SQLite and file I/O performance.
- **PostgreSQL migration:** Removes write concurrency limits.

11.6.3 Scaling Recommendations

For higher throughput:

1. Migrate to PostgreSQL for multi-writer support.
2. Implement Python process pooling to amortize startup costs.
3. Move file storage to object storage (S3-compatible) for horizontal scaling.
4. Deploy multiple server instances behind a load balancer with shared database.
5. Implement a message queue (Redis/RabbitMQ) for pipeline orchestration.

11.7 Monitoring and Observability

11.7.1 Built-in Metrics

The platform tracks:

- Request count and latency per endpoint.
- Active user count (heartbeat-based, 2-minute window).
- Token consumption per session and per paper.
- API cost accumulation.
- Pipeline run duration and status distribution.
- Governor intervention frequency.

11.7.2 Metrics Access

Metrics are accessible via:

- Admin dashboard (web UI).
- `GET /api/platform/stats` (internal endpoint).
- `GET /api/v1/stats/*` (public API with authentication).
- `GET /api/v1/pipeline/stats` (pipeline-specific metrics).

Design Patterns and Architecture Philosophy

12.1 Overview

AutoScholar's architecture emerges from a set of implicit design patterns and philosophical choices that are not always obvious from reading individual files. This chapter identifies and names these patterns, explains why they were chosen, and evaluates their effectiveness.

12.2 The Monolithic Streaming Loop

12.2.1 Pattern Description

The entire research pipeline—from literature review through peer review—executes within a single HTTP streaming connection. There is no message queue, no job scheduler (beyond the cadence timer), no actor framework. Control flows linearly through Claude's tool calls, with the tool executor dispatching to services and returning results.

12.2.2 Rationale

- **Simplicity:** A single connection eliminates distributed state management, message serialization, and coordination overhead.
- **Context coherence:** All information accumulates in one Claude context window, enabling natural cross-referencing between pipeline stages.
- **Debugging:** A single stream makes it easy to replay and debug the entire pipeline.

12.2.3 Trade-offs

- **Fragility:** A single connection failure can lose 30+ minutes of work. (Mitigated by checkpoints.)
- **No parallelism within a pipeline:** Stages execute sequentially; there's no concurrent data collection + analysis.
- **Resource lock-in:** Each pipeline holds a Claude connection for the full duration.

12.3 Prompt-Based Agent Architecture

12.3.1 Pattern Description

Rather than instantiating separate model instances for each agent, AutoScholar defines all agents as *conceptual roles* within a single system prompt. Claude switches between Euler, Gauss, Turing, Newton, and Fisher based on the pipeline stage, guided by the system prompt's instructions.

12.3.2 Rationale

- **Context sharing:** All agents share the same context, eliminating inter-agent communication overhead.
- **Coherence:** A single model maintains consistent reasoning throughout the pipeline.
- **Efficiency:** No overhead for spawning separate model instances or transferring context between them.

12.3.3 Trade-offs

- **No specialization:** All agents use the same model capabilities; there's no task-specific fine-tuning.
- **System prompt bloat:** The 1,798-line system prompt is large and complex, consuming significant context.
- **Implicit transitions:** Agent transitions are implicit, making it harder to enforce strict phase ordering.

12.4 The TypeScript–Python Bridge

12.4.1 Pattern Description

Every Python computation follows a stereotyped pattern:

1. TypeScript module defines types, validates input, generates displayable code.
2. TypeScript spawns Python subprocess via `child_process.spawn`.
3. JSON input via stdin, JSON output via stdout.
4. TypeScript parses output and returns typed results.

This pattern is replicated 10+ times across the codebase.

12.4.2 Rationale

- **Language strengths:** TypeScript for web/API handling, Python for scientific computing.
- **Isolation:** Python crashes don't bring down the Node.js server.
- **Security:** Python runs in a separate process with no direct access to server state.

12.4.3 Trade-offs

- **Startup overhead:** Each Python invocation pays a 1–3s initialization cost.
- **No state sharing:** Each invocation starts fresh; there's no shared in-memory state.
- **Code duplication:** The bridge pattern has significant boilerplate.

12.5 The Governor as a Pure Rule Engine

12.5.1 Pattern Description

The GOVERNOR is implemented as a pure rule-based system with zero LLM calls. It observes tool results, maintains typed state, and issues corrective actions based on predefined rules. This is a deliberate contrast to the rest of the system, which relies heavily on LLM reasoning.

12.5.2 Rationale

- **Determinism:** Rule-based decisions are predictable, testable, and debuggable.
- **Performance:** No API latency or cost for governance decisions.
- **Reliability:** The governor cannot hallucinate or produce inconsistent judgments.
- **Separation of concerns:** Creative reasoning (Claude) is separated from operational governance (rules).

12.5.3 Trade-offs

- **Limited intelligence:** Rules can only handle pre-anticipated failure modes.
- **Brittle:** New error patterns require manual rule additions.
- **No adaptation:** The governor cannot learn from experience.

12.6 Tool-Rich Agent Paradigm

12.6.1 Pattern Description

With 213 tool definitions totaling ~6,100 lines of schema, AUTOSCHOLAR takes a maximalist approach to tool availability. Claude has access to every financial, academic, web, macro, Python, and paper-generation tool at all times.

12.6.2 Rationale

- **Maximum flexibility:** Claude can select the most appropriate tool for any situation without routing logic.
- **Self-service:** The system prompt provides tool documentation; Claude selects tools based on context.
- **Comprehensive coverage:** 230+ FMP endpoints, 5 academic databases, 3 web providers, 3 macro providers.

12.6.3 Trade-offs

- **Context consumption:** Tool definitions consume ~30K tokens of the context window.
- **Selection confusion:** Claude may choose suboptimal tools when many similar options exist.
- **Maintenance burden:** Changes to any tool require updating both the definition and the executor.

12.7 Barrel Re-Export Pattern

12.7.1 Pattern Description

The codebase makes extensive use of barrel (index) files that re-export from subdirectory modules. Examples:

- `server/storage.ts` → `storage/index.ts`
- `server/services/claudeService.ts` → `claude/index.ts`
- `server/services/python/index.ts` → 10 Python service modules
- `server/services/claude/tools/index.ts` → 7 tool definition files

12.7.2 Rationale

- Clean import paths for consumers.
- Freedom to refactor internal module structure without changing import sites.
- Single point of control for public API surface.

12.8 Defensive Sanitization

12.8.1 Pattern Description

Multiple layers of content sanitization are applied throughout the pipeline:

- **Python code:** Deprecated API patches, data fabrication detection, error auto-correction.
- **LaTeX content:** Special character escaping, table auto-fitting, blank figure detection.
- **Markdown rendering:** XML function call stripping, raw HTML tag removal, figure reference resolution.
- **API responses:** Null/NaN sanitization, result size limiting, ticker normalization.

12.8.2 Rationale

LLM outputs are inherently unpredictable. Every output channel (Python code, LaTeX, markdown, JSON) requires defensive processing to handle the long tail of malformed outputs.

12.9 Implicit Convention Enforcement

12.9.1 Pattern Description

Academic conventions are enforced through detailed natural-language instructions in the system prompt and style directives, rather than through code. For example:

- The JF Writing Style Directive (282 lines) enforces prose quality through banned words, section conventions, and verification checklists.
- The Academic Figure Style Directive (429 lines) enforces chart formatting through detailed matplotlib instructions.
- The system prompt's 7-step pipeline enforces methodology through explicit ordering instructions.

12.9.2 Rationale

Convention enforcement through prompting leverages the LLM's natural language understanding, which is more flexible and comprehensive than programmatic rules. A banned-word list in the prompt is more effective than regex-based filtering because the LLM understands context.

12.9.3 Trade-offs

- **Non-deterministic:** Prompt-based enforcement is probabilistic, not guaranteed.
- **Unverifiable:** There's no automated check that all conventions are followed.
- **Prompt bloat:** These directives consume significant context (~10K tokens combined).

12.10 Design Philosophy Summary

The overall design philosophy can be characterized as:

> **Maximize LLM capability, minimize infrastructure complexity, and use deterministic systems only where reliability is critical.**

This leads to a system where:

1. Claude handles all creative and analytical work.
2. The Governor handles all reliability-critical decisions.
3. The tool executor handles all integration logic.
4. Python handles all computation.

5. The database handles all persistence.

There are no message queues, no container orchestration, no distributed state stores, no microservice boundaries. The system is a monolith that leverages one very powerful LLM.

Critical Evaluation and Weaknesses

13.1 Scientific Rigor

13.1.1 Identification Strategy

The most fundamental weakness in AutoScholar's research output is the treatment of causal identification. While the system prompt mandates identification strategies (instrumental variables, difference-in-differences, regression discontinuity), the system has no way to verify that:

- Instruments are actually valid (relevant and excludable).
- Parallel trends assumptions hold for DiD designs.
- Assignment mechanisms are genuinely discontinuous for RD designs.
- Confounders are adequately controlled.

Claude may claim to use a valid instrument without verifying the exclusion restriction, or assert parallel trends without formal testing. These are precisely the kinds of judgments that require deep domain expertise and cannot be reliably automated.

13.1.2 Statistical Inference

- **P-hacking risk:** The system runs multiple specifications and selects results to present, without correction for multiple hypothesis testing. There is no Bonferroni, Holm-Bonferroni, or false discovery rate correction.
- **Overfitting in ML:** While cross-validation is mandated, the system may select hyperparameters that overfit to the specific dataset without adequate regularization.
- **Distribution assumptions:** Parametric VaR assumes normal returns, which is known to be violated. The system does not always verify distributional assumptions before applying parametric methods.

13.1.3 Data Quality

- **No manual data inspection:** The system cannot visually inspect data for anomalies the way a human researcher would.

- **Look-ahead bias:** When constructing time-series features, the system may inadvertently introduce future information (e.g., using filing dates instead of announcement dates).
- **Survivorship bias:** Using current stock lists to study historical performance introduces survivorship bias that the system does not systematically address.

13.2 Reproducibility

13.2.1 Strengths

The reproducible project ZIP is a genuine strength:

- Complete Python scripts with numeric ordering.
- Data provenance documentation.
- Auto-generated `requirements.txt`.
- Master pipeline script.

13.2.2 Weaknesses

- **API dependency:** Scripts require live API keys (FMP, EODHD, etc.) to reproduce. Raw data is not always included in the ZIP.
- **Non-determinism:** Random seeds are not always set, making Monte Carlo and bootstrapping results non-reproducible.
- **API versioning:** External API responses may change over time, producing different results on re-execution.
- **Library versions:** `requirements.txt` does not pin specific versions.

13.3 Robustness

13.3.1 Single Point of Failure: Claude API

The entire system depends on a single external service (Anthropic's Claude API). If the API is unavailable, rate-limited, or degraded, the entire pipeline halts. There is no failover to alternative LLMs.

13.3.2 Monolithic Architecture

The single-process Node.js server means:

- A crash affects all concurrent sessions and pipeline runs.
- CPU-intensive operations (JSON parsing, LaTeX sanitization) block the event loop.
- Memory leaks in long-running sessions can degrade the server over time.

13.3.3 File System Dependency

- Scripts, figures, and LaTeX files are stored on the local filesystem.
- No redundancy or backup mechanism.
- Disk space is not monitored or managed.

13.4 Quality Control Limitations

13.4.1 Self-Review Bias

The Popper reviewer uses the same model (Claude Opus 4.6) that generated the paper. This creates a systematic positive bias:

- The reviewer may accept reasoning patterns it would generate itself.
- Subtle methodological errors that the generator would not notice are equally invisible to the reviewer.
- Writing quality scores are inflated because the reviewer shares the generator's style preferences.

13.4.2 Low Rejection Threshold

The default publication threshold of 45/100 is permissive. Papers scoring 45–55 often have significant weaknesses:

- Incomplete analysis (some sections may be stubs).
- Weak empirical results (insignificant coefficients presented as findings).
- Missing robustness checks.
- Poorly integrated figures.

13.4.3 No Human Review

There is no mechanism for human review before or after publication. The autonomous journal publishes everything above the threshold, including papers with:

- Factually incorrect claims.
- Methodologically unsound analysis.
- Fabricated-looking data patterns.
- Plagiarized-equivalent content (multiple papers on similar topics may share significant text).

13.5 Security Concerns

13.5.1 Python Sandbox

The Python execution environment, while isolated in a subprocess, has:

- Full network access (for API calls).
- Read access to the filesystem.
- No resource limits (CPU, memory, disk).
- No syscall filtering or container isolation.

Since Claude writes the Python code, prompt injection could theoretically lead to arbitrary code execution.

13.5.2 Admin Authentication

The admin authentication system is minimal:

- Single shared password (default: "admin123").
- No multi-factor authentication.
- No audit log for admin actions.
- No role-based access control.

13.5.3 API Key Storage

User-provided API keys are stored with XOR obfuscation (not encryption) using the session secret. XOR with a known key is trivially reversible.

13.6 Scalability Weaknesses

- **SQLite write contention:** Under concurrent paper generation, SQLite's single-writer model can cause lock contention.
- **In-memory state:** Rate limiter, session broadcaster, and governor state are in-process and lost on restart.
- **No horizontal scaling:** Cannot run multiple server instances behind a load balancer without significant refactoring.
- **File-based artifacts:** Script management and LaTeX compilation use local temp directories, preventing distributed execution.

13.7 Content Quality Weaknesses

13.7.1 Hallucination Risk

Despite extensive tool access, Claude may:

- Fabricate citations that don't exist.
- Misinterpret tool results (e.g., confusing nominal and real values).
- Generate plausible-sounding but incorrect economic interpretations.
- Report statistical results that don't match the actual computation.

13.7.2 Template Repetitiveness

Papers generated from similar prompts may share:

- Identical introductory paragraphs.
- Same literature review structure.
- Similar methodology descriptions.
- Common phrases and sentence structures.

The JF Writing Style Directive's AI detection countermeasures (lexical diversity, syntactic variation) partially address this but cannot fully eliminate it.

13.7.3 Depth vs. Breadth

The system tends to produce breadth over depth:

- Many variables tested superficially rather than deep investigation of a few.
- Multiple model specifications without thorough economic motivation for each.
- Robustness checks that are mechanical rather than meaningful.

13.8 Ethical Considerations

- **Transparency:** All papers include an agent disclosure section, clearly identifying the AI author. The ASID system distinguishes AI-generated papers from human-authored DOI publications.
- **Misuse potential:** The system could be used to generate fraudulent research papers or flood academic databases with low-quality content.
- **Academic integrity:** Students or researchers could use the system to produce papers and claim them as their own work.
- **Market impact:** Financial research papers could influence investment decisions; AI-generated analysis carries different reliability than human expert analysis.

Proposed Improvements

14.1 AutoScholar v2: Deep Search Architecture

14.1.1 Motivation

The current system uses a single-pass approach: one Claude context window handles the entire pipeline. A "Deep Search" architecture would decompose the pipeline into multiple specialized agents, each with their own context window, communicating through a shared state store.

14.1.2 Proposed Architecture

Agent Pool. Maintain a pool of specialized agent instances:

- **Gauss Agent** (Haiku/Sonnet) — Lightweight, fast literature search.
- **Turing Agent** (Haiku/Sonnet) — Parallel data collection across multiple sources.
- **Newton Agent** (Sonnet) — Data preparation and quality validation.
- **Fisher Agent** (Opus) — Complex analysis requiring deep reasoning.
- **Euler Agent** (Opus) — Orchestration and academic writing.
- **Popper Agent** (Sonnet) — Independent review with no context contamination.

Shared State Store. A Redis or PostgreSQL-backed state store holding:

- Discovered papers and citations.
- Downloaded datasets and metadata.
- Analysis results and figures.
- Written sections and BibTeX entries.

Message Bus. An event-driven message bus (Redis Streams, NATS) enabling:

- Parallel execution of independent stages.
- Dynamic agent scaling based on workload.
- Fault isolation between agents.

14.1.3 Benefits

- **Parallel execution:** Gauss and Turing can run concurrently.
- **Cost optimization:** Use cheaper models (Haiku) for simple tasks.
- **Fault isolation:** A Python crash in Fisher doesn't affect Euler.
- **Context efficiency:** Each agent only needs context relevant to its task.
- **Independent review:** Popper reviews without seeing the generation process, reducing self-review bias.

14.2 Journal++ Model

14.2.1 Multi-Tier Publication

Introduce quality tiers for published papers:

Tier A (Score $\geq$ 80): "Featured Research" — Highlighted on the journal homepage, included in monthly digests.

Tier B (Score 65–79): "Published Research" — Standard publication, included in domain listings.

Tier C (Score 45–64): "Working Papers" — Published with a disclaimer, excluded from quality metrics.

Rejected (Score $<$ 45): Archived but not published. Available via admin API for analysis.

14.2.2 Multi-Pass Review

1. **Pass 1:** Popper generates initial review (current system).
2. **Pass 2:** A second reviewer (different model or temperature) provides an independent assessment.
3. **Pass 3:** Meta-reviewer reconciles the two reviews and makes the final decision.
4. **Optional:** Human reviewer for Tier A candidates.

14.2.3 Citation Network

Enable cross-paper citation:

- When a new paper is generated, search the existing corpus for relevant prior work.
- Auto-generate BibTeX entries for AutoScholar papers.
- Track citation relationships in the database.
- Display citation graphs on paper landing pages.

14.2.4 Revision Workflow

Instead of single-pass publication:

1. Generate paper (current pipeline).
2. Review (Popper).
3. If score 45–65: trigger a targeted revision cycle.
4. Euler addresses specific criticisms, re-runs analysis if needed.
5. Second review.
6. Publish or reject based on improved score.

14.3 Better Memory System

14.3.1 Cross-Paper Knowledge Base

Implement a persistent knowledge base that accumulates across paper generations:

Topic Memory. Track topics covered with timestamps. Prevent duplicate topics within a configurable window (e.g., 30 days). Weight prompt selection away from recently covered areas.

Method Memory. Track which methods have been applied to which domains. Prefer novel combinations. Detect and avoid repetitive methodological patterns.

Data Source Memory. Track which data sources have been used, their reliability, and common issues. Learn which FMP endpoints frequently return errors for specific tickers.

Failure Memory. Record common failure patterns with their resolutions. When a similar error occurs, inject the known solution as a governor directive.

Quality Memory. Track Popper scores by domain, method, and prompt type. Adjust future prompt selection to favor combinations that historically produce higher scores.

14.3.2 Implementation

- Store knowledge base entries in a dedicated database table with vector embeddings for semantic search.
- Inject relevant knowledge base entries into the system prompt at the start of each run.
- Limit injected context to 5,000 tokens to avoid overwhelming the system prompt.
- Use a sliding window to decay old entries and prioritize recent learnings.

14.4 True Novelty Detection

14.4.1 Current Limitation

The current gap detection relies on Claude's reasoning about a small sample of papers. True novelty requires systematic comparison against the full academic corpus.

14.4.2 Proposed System

1. **Embedding-based search:** Compute embeddings for all paper abstracts (generated and external) using a dedicated embedding model.
2. **Novelty scoring:** Before starting a paper, compute the cosine similarity between the proposed topic and all existing papers. Flag topics with similarity > 0.9 to any existing paper.
3. **Contribution verification:** After writing, verify that the paper's stated contributions are not already present in the literature by embedding comparison.
4. **Plagiarism detection:** Compare generated text against existing papers to detect excessive similarity (beyond proper citation).

14.4.3 Technical Requirements

- Vector database (pgvector, Pinecone, or Qdrant).
- Embedding model (OpenAI Ada, BGE, or E5 via OpenRouter).
- Nightly batch embedding job for new papers.
- Similarity threshold tuning based on domain characteristics.

14.5 Dataset Versioning

14.5.1 Problem

Currently, data is fetched at pipeline runtime, and the raw API responses are not stored. This means:

- Reproducibility requires re-fetching from live APIs.
- API changes can break reproduction.
- There is no versioning of datasets across papers.

14.5.2 Proposed System

Data Lake. Store all fetched API responses in a versioned data lake (S3 or local filesystem with content-addressable storage).

Dataset Registry. Maintain a registry of datasets with:

- Content hash (SHA-256) for deduplication.
- Source API, parameters, and timestamp.
- Data schema (column types, row count).

- Usage tracking (which papers reference which datasets).

Cache Layer. Before fetching from external APIs, check the cache:

- If data exists and is < 24 hours old, use cached version.
- If data exists but is stale, fetch new version and store alongside.
- Track all versions for reproducibility.

14.6 Reproducibility Layer

14.6.1 Enhanced Project Archives

1. **Pin library versions:** Generate `requirements.txt` with exact version pins (e.g., `pandas==2.1.4`).
2. **Include raw data:** Bundle fetched API responses in the project ZIP.
3. **Set random seeds:** Inject `random.seed(42); np.random.seed(42)` into all scripts.
4. **Docker support:** Generate a `Dockerfile` with the exact Python version and dependencies.
5. **GitHub Actions:** Generate a CI pipeline that re-runs the analysis and verifies outputs.

14.6.2 Verification Service

- Automated re-execution of project archives in a clean environment.
- Comparison of outputs (figures, tables, statistics) against the published paper.
- Reproducibility score based on output match percentage.
- Public reproducibility badge on paper landing pages.

14.7 Additional Improvements

14.7.1 Governor v2: Hybrid Rule-LLM

Enhance the governor with selective LLM calls for complex decisions:

- Use rules for common patterns (current behavior).
- Use Claude Haiku for novel error patterns not covered by rules.
- Gradually convert successful LLM decisions into new rules.

14.7.2 Interactive Revision Mode

Allow human researchers to:

- Request specific revisions to generated papers.
- Provide additional data or constraints.

- Override Popper's review decision.
- Add their own analysis as co-authors.

14.7.3 Multi-Language Support

- Generate papers in French, Spanish, German, Chinese, and Japanese.
- Language-specific style guides and citation formats.
- Bilingual abstracts.

14.7.4 Real-Time Data Integration

- WebSocket connections to live market data feeds.
- Real-time event detection triggering paper generation.
- Breaking news analysis and rapid-response research.

14.7.5 Performance Optimization

- Python process pooling to amortize startup costs.
- Caching layer for frequently accessed API data.
- Streaming LaTeX compilation (compile sections as they are written).
- Parallel tool execution for independent tool calls.

System Visualizations

15.1 Full System Diagram

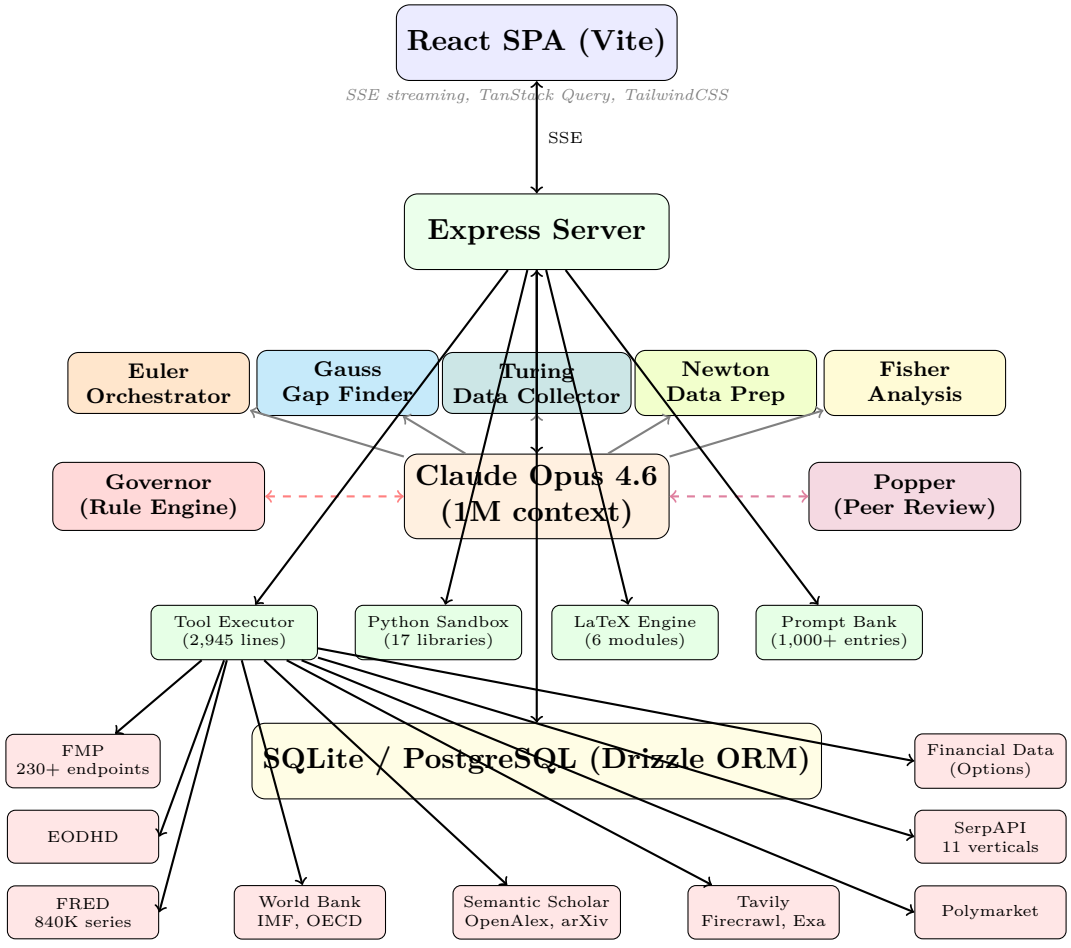


Figure 15.1: Complete AutoScholar system architecture diagram showing all agents, services, storage, and external integrations.

15.2 Pipeline Flow Diagram

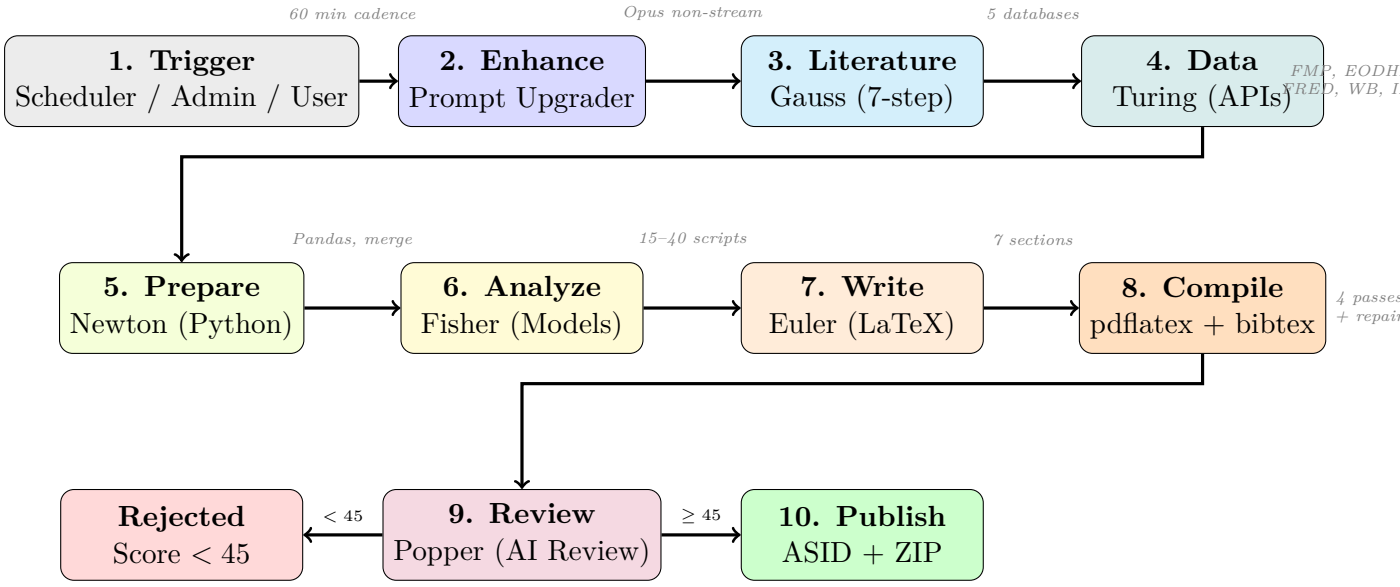


Figure 15.2: End-to-end paper generation pipeline with all 10 phases.

15.3 Agent Interaction Diagram

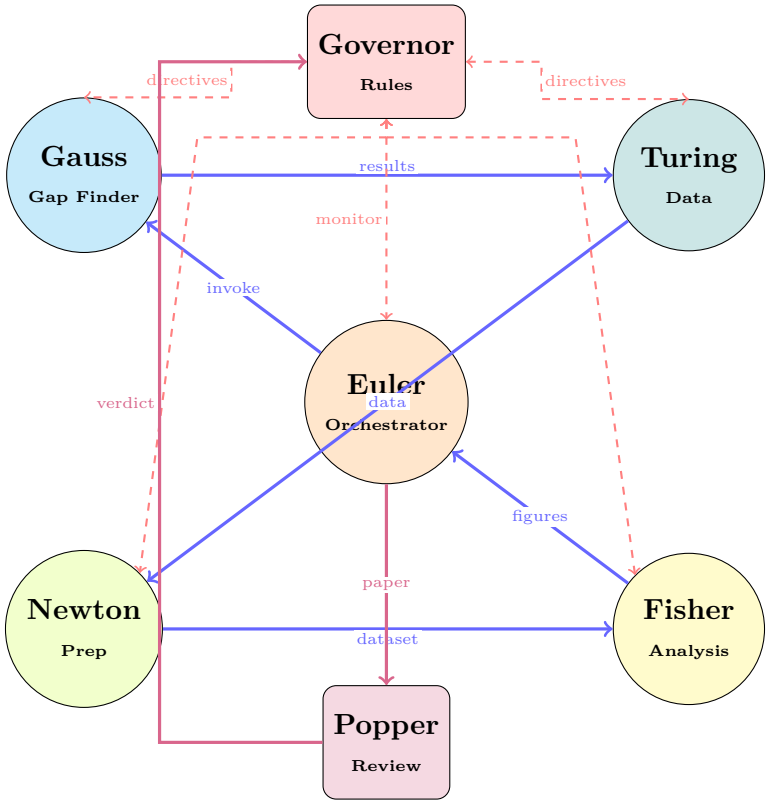


Figure 15.3: Agent interaction patterns showing sequential handoff (blue), governor oversight (red dashed), and Popper review feedback (purple).

15.4 Data Flow Diagram

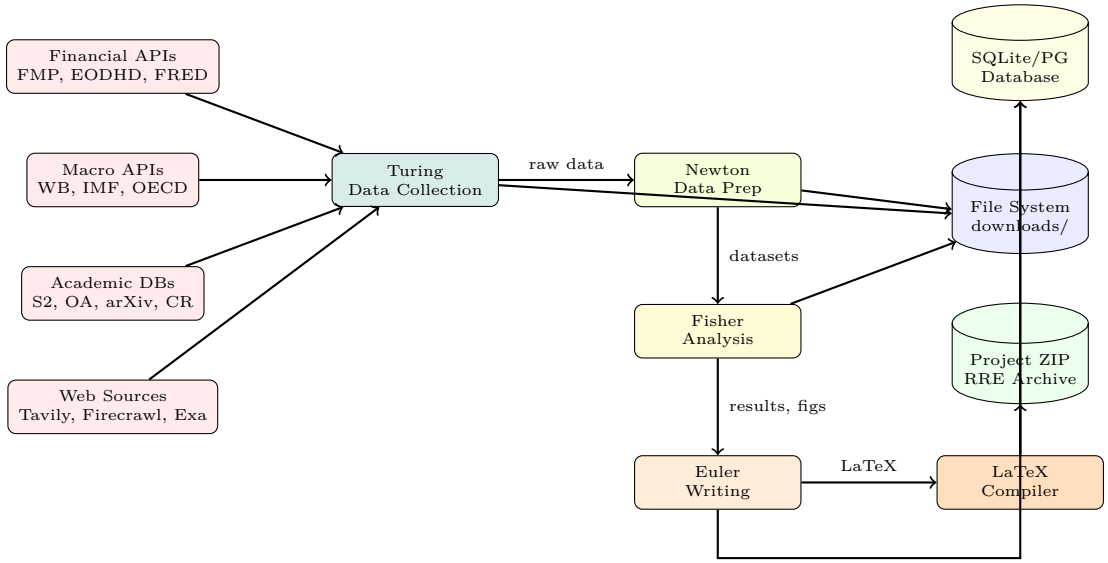


Figure 15.4: Data flow from external sources through processing stages to persistent storage.

15.5 Governor Decision Flow

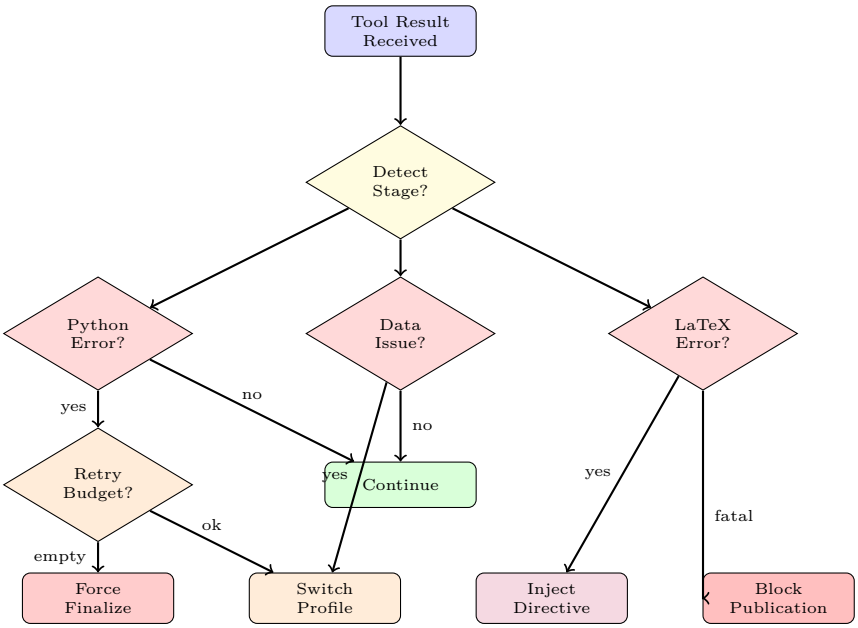


Figure 15.5: Governor decision flow showing error detection, budget checking, and action selection.

15.6 Storage Schema Diagram

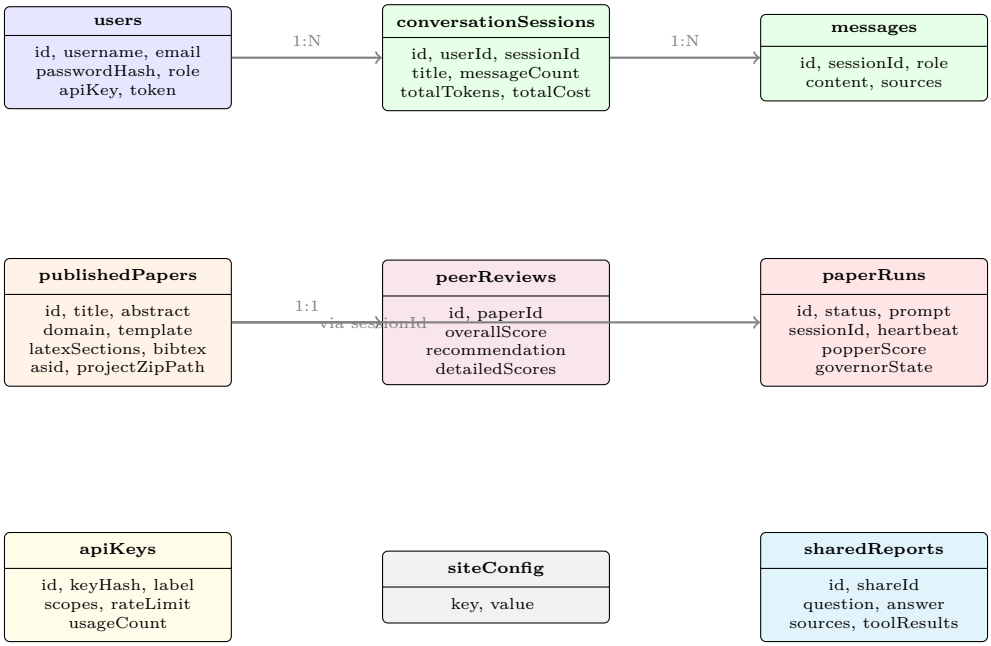


Figure 15.6: Core database schema relationships (simplified, 20+ tables total).

15.7 Tool Ecosystem Map

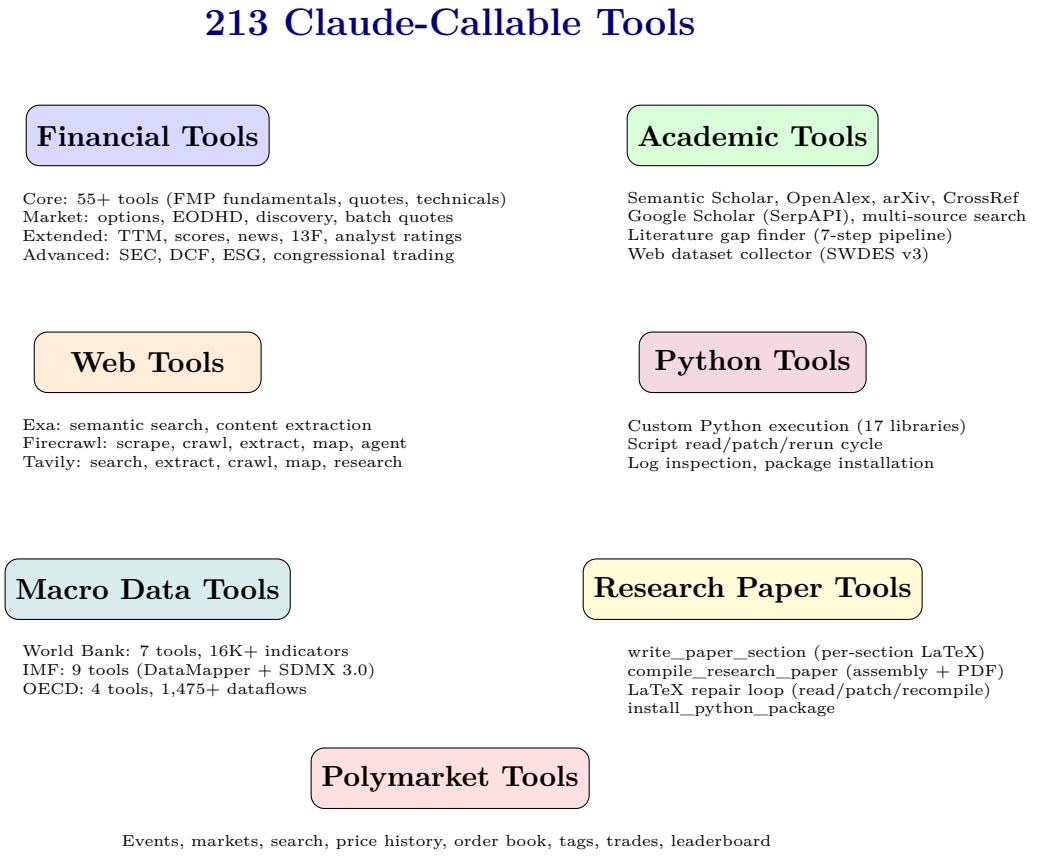


Figure 15.7: Tool ecosystem organized by category showing the breadth of AutoScholar's capabilities.

Frontend Architecture

16.1 Overview

The AutoScholar frontend is a React 18.3 single-page application built with Vite 5, TailwindCSS 3, and TanStack React Query. It provides three primary interfaces: (1) a conversational research interface for interactive analysis, (2) a public showcase of shared research reports, and (3) an administrative dashboard for pipeline management.

16.2 Technology Stack

Table 16.1: Frontend technology stack.

Technology	Purpose
React 18.3	UI framework with hooks and context API
Vite 5	Build tool with HMR for development
TailwindCSS 3	Utility-first CSS framework
TanStack React Query	Server state management, caching, and mutations
Framer Motion	Animation library for UI transitions
wouter	Lightweight client-side routing (20+ routes)
react-markdown + remark-gfm	Markdown rendering with GitHub-flavored extensions
jsPDF + html2canvas	Client-side PDF generation
Recharts	Data visualization for admin dashboard

16.3 Application Structure

16.3.1 Routing

The application defines 20+ routes via wouter in `App.tsx` (77 lines):

Table 16.2: Frontend routes.

Path	Page
`/app`	Home — Main research interface
`/showcase`	Showcase — Public shared reports gallery
`/admin`	Admin — Pipeline management dashboard
`/share/:shareId`	SharedReport — Individual report viewer
`/journal`	Journal — Published papers listing
`/journal/:paperId`	Paper detail page
`/paper/:asid`	Paper by ASID
`/repository`	Repository browser
`/live-paper`	Live paper generation viewer
`/docs`	API documentation
`/privacy`	Privacy policy
`/terms`	Terms of service
`/donate`	Donation page
`/agents`	Agent documentation
`/api`	API explorer

16.3.2 Context Providers

The app wraps all routes in a provider chain:

1. `ErrorBoundary` — Catches React rendering errors.
2. `QueryClientProvider` — TanStack React Query with 5-minute stale time, 30-minute GC.
3. `ThemeProvider` — Dark/light theme management.
4. `TooltipProvider` — Shared tooltip configuration.
5. `ActiveSessionProvider` — Global chat state management.

16.4 Core Components

16.4.1 Research Interface (`pages/home.tsx`)

The main page (338 lines) provides:

- Header with logo, navigation buttons (Repository, Journal, Live Paper, Docs), and session history drawer.
- Empty state with `ResearchForm` showing categorized research prompts and "surprise paper" button.
- Streaming result display with:
  - `StreamingAnswer` for markdown rendering.
  - `AgentSteps` for pipeline progress timeline.
  - `CustomPythonFigure` for matplotlib figure galleries.

  - `ResultCard` for source citation cards.
  - `DownloadButtons` for PDF/DOCX/HTML/Markdown export.
- Fixed bottom `SearchBar` for follow-up queries.
- "Watch Live" floating button during active generation.

16.4.2 Streaming Answer (`components/streaming-answer.tsx`)

The core rendering component (446 lines) for AI-generated content:

- Uses react-markdown with remark-gfm and extensive custom component overrides.
- Content sanitization: strips leaked XML function call blocks, raw HTML tags, normalizes whitespace.
- Figure resolution: supports `fig://` protocol, markdown images, and figure registry lookups.
- Auto-scrolls during streaming with a blinking cursor indicator.
- Download link handling via blob URLs.
- Source display in a verified-badge grid.
- Code block syntax highlighting.

16.4.3 Agent Steps (`components/chat/agent-steps.tsx`)

Pipeline progress visualization (389 lines):

- Defines 5 named agents with distinct colors and icons:
  - Euler (orange) — Orchestrator
  - Gauss (blue) — Gap Analysis
  - Turing (teal) — Data Collection
  - Newton (green) — Dataset Preparation
  - Fisher (purple) — Empirical Analysis
- Steps displayed as a collapsible timeline with status indicators (pending/running/completed/error).
- Animated transitions via Framer Motion.
- Progress bar and parallel execution indicators.
- Maps 30+ internal tool names to human-readable labels.

16.4.4 Search Bar (`components/chat/search-bar.tsx`)

Feature-rich input component (422 lines):

- Text input with multi-line support.
- Image attachment with preview.

- Data file upload (CSV, Excel, Parquet, JSON) via `/api/upload-data`.
- Voice dictation via MediaRecorder API and `/api/speech-to-text` (ElevenLabs Scribe v2).
- Submit handler passing query, image data, and uploaded file metadata.

16.4.5 Python Figure Gallery (`components/chat/custom-python-figure.tsx`)

Figure display component (205 lines):

- Organized in collapsible batches with description headers.
- Auto-detects URL vs base64-encoded images.
- Individual figure download.
- Click-to-open fullscreen viewer.
- Retry on image load failure with cache-busting.
- Auto-expands new batches as they arrive during streaming.

16.5 State Management

16.5.1 Chat Session Hook (`hooks/useChatSession.ts`)

The central state management hook (344 lines) manages ~20 pieces of state:

- Session ID (persisted to `sessionStorage`).
- Conversation history (array of Q&A pairs).
- Streaming answer and loading state.
- Sources, search queries, and tool results.
- Agent steps with status tracking.
- Python code entries and figure batches.
- Monte Carlo and options pricing results.
- Figure registries (URL and base64 maps).
- Author profile (from `localStorage`).

Key operations:

`handleSearch()` Initiates a new query, manages streaming state, and accumulates results.

`handleRetry()` Re-submits the last query.

`handleNewConversation()` Resets all state for a fresh session.

`handleLoadSession()` Loads a previous session from the API.

16.5.2 Chat Mutation Hook (`hooks/useChatMutation.ts`)

SSE streaming implementation (165 lines):

1. Sends POST to `/api/chat` with query and conversation history.
2. Reads response body as a stream.
3. Parses SSE `data:` lines into typed events.
4. Forwards each chunk to the `onChunk` callback.
5. Supports `AbortController`-based cancellation.
6. Handles 401 by reloading the page.

16.5.3 Analytics Hook (`hooks/useAnalytics.ts`)

Platform telemetry (94 lines):

- Heartbeat POST every 30 seconds during active generation.
- Real-time stats polling at configurable intervals (default 10s).
- Auto-generated session IDs persisted in `sessionStorage`.

16.6 Document Export

16.6.1 Server-Side Export

Three export buttons trigger server-side generation:

- **PDF:** POST to `/api/generate-pdf` — Puppeteer renders HTML to PDF.
- **DOCX:** POST to `/api/generate-docx` — Pandoc converts markdown.
- **HTML:** POST to `/api/generate-html` — Self-contained HTML with KaTeX.

16.6.2 Client-Side Export

- **Markdown:** Client-side blob download (no server call).
- **Advanced PDF:** `pdfGenerator.ts` (539 lines) uses jsPDF + html2canvas for element capture with AutoScholar-branded headers, pagination, and table formatting.

16.7 Configuration

`config.ts` (58 lines) defines:

- **Base URL:** Production (`https://autoscholar.ai`) vs development (`localhost:3001`).
- **App metadata:** Name "AutoScholar", version "3.0.0".
- **Feature flags:** `pythonCustomCode`, `pdfDownload`, `sharing`, `exportData`, `multiSource`.
- **Limits:** `maxToolsPerBatch`: 5, `maxPdfChars`: 3M, `maxQueryLength`: 10K.

16.8 Build and Development

16.8.1 Development

- Vite dev server with React HMR plugin.
- PostCSS with TailwindCSS and Autoprefixer.
- Path aliases: `@` → `client/src`, `@shared` → `shared`.
- Relaxed filesystem access for monorepo structure.

16.8.2 Production Build

- `vite build` outputs to `dist/public/`.
- Static files served by Express in production mode.
- SEO meta tags in `index.html` (Open Graph, Twitter Cards).
- Google Fonts preconnect for Inter, Space Grotesk, Playfair Display, Libre Baskerville, EB Garamond, JetBrains Mono.

Public REST API

17.1 Overview

AutoScholar exposes a public REST API at `/api/v1/` providing authenticated access to the platform's paper corpus, peer reviews, reports, pipeline monitoring, search, prompts, session metadata, and platform statistics. The API is designed for programmatic access by researchers, data scientists, and third-party applications.

17.2 Authentication

17.2.1 API Key Management

Key creation: `POST /api/v1/keys` — Open registration (no auth required). Returns the raw key exactly once.

Key listing: `GET /api/v1/keys` — Authenticated users can list their masked keys.

Key revocation: `DELETE /api/v1/keys/:id` — Revoke a specific key.

17.2.2 Authentication Methods

1. **Bearer token:** `Authorization: Bearer sk_as_...`
2. **Query parameter:** `?api_key=sk_as_...`

Keys are stored as SHA-256 hashes in the database. The raw key is never stored.

17.2.3 Scopes

API keys support four scopes:

Table 17.1: API key scopes.

Scope	Permissions
`read`	Read papers, reviews, reports, stats
`search`	Full-text search across resources
`download`	Download project ZIPs and data files
`write`	Create prompts and reports

17.2.4 Rate Limiting

In-memory per-key rate limiting:

- Default: 100 requests per hour.
- Configurable per key at creation time.
- Auto-cleanup of expired rate limit windows every 10 minutes.
- 429 response with `Retry-After` header when exceeded.

17.3 Response Format

All responses use a standardized JSON envelope:

Listing 17.1: API response envelope.

```
// Success
{
  "ok": true,
  "data": { ... },
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  }
}

// Error
{
  "ok": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Paper not found"
  }
}
```

17.4 Endpoints

17.4.1 Papers

`GET /api/v1/papers` List published papers.

- Filters: `domain`, `template`, `author`, `keyword`, `q` (text search).
- Sorting: `newest`, `oldest`, `words`, `figures`.
- Pagination: `page`, `limit` (max 100).
- Includes generation cost data (Opus 4.6 pricing).

`GET /api/v1/papers/:id` Get paper details.

- Full LaTeX sections, BibTeX, figure count, word count.
- ASID, peer review score, domain classification.

GET /api/v1/papers/:id/latex Get raw LaTeX source.

GET /api/v1/papers/:id/bibtex Get auto-generated BibTeX citation.

GET /api/v1/papers/domains List unique domains with paper counts.

GET /api/v1/papers/templates List templates with paper counts.

17.4.2 Reviews

GET /api/v1/reviews List all peer reviews.

- Filters: `recommendation`, `minScore`, `maxScore`.
- Sorting and pagination.

GET /api/v1/reviews/:paperId Get review for a specific paper.

- Full breakdown: overall score, 6 dimension scores, section reviews, strengths, weaknesses.

GET /api/v1/reviews/stats/summary Aggregate review statistics.

- Average scores across all dimensions.
- Recommendation distribution.

17.4.3 Reports

GET /api/v1/reports List shared research reports.

- Text search and pagination.

GET /api/v1/reports/:shareId Get full report.

- Answer, sources, tool results, figures.

GET /api/v1/reports/:shareId/sources Get sources only.

17.4.4 Search

GET /api/v1/search Cross-resource full-text search.

- Searches papers, reports, and prompts.
- Relevance scoring (title matches weighted highest).
- Type filtering and pagination.

GET /api/v1/search/papers Advanced paper search.

- Additional filters: domain, author, year, word count range.

17.4.5 Pipeline

GET /api/v1/pipeline/runs List pipeline runs.

- Status filtering: queued, researching, analyzing, drafting, validating, published, rejected, failed.
- Pagination.

GET /api/v1/pipeline/runs/:id Get run details.

- Governor state, Popper score, prompt, session ID.

GET /api/v1/pipeline/stats Aggregate pipeline statistics.

- Average duration, score distribution, status breakdown.

GET /api/v1/pipeline/active Currently active runs.

17.4.6 Statistics

GET /api/v1/stats/platform Platform-wide metrics.

- Papers, words, figures, LaTeX lines, Python executions, tokens, cost.

GET /api/v1/stats/domains Per-domain paper counts, words, figures.

GET /api/v1/stats/timeline Monthly publication trends.

GET /api/v1/stats/analytics Time-series metrics.

- Configurable period and limit.
- Metrics: requests, users, performance, tokens, tools, errors.

17.4.7 Sessions

GET /api/v1/sessions List anonymized research sessions.

- Sorting: newest, oldest, cost, tokens.
- Metadata: message count, Python executions, tools used.
- Content is not exposed.

GET /api/v1/sessions/stats Session aggregate statistics.

- Total tokens, cost, averages, daily activity (30 days).

17.4.8 Prompts

GET /api/v1/prompts List public prompts.

- Search and author filtering.

POST /api/v1/prompts Create a prompt (requires `write` scope).

17.5 Pagination

All list endpoints support cursor-based pagination:

- `page` (default: 1) — Page number.
- `limit` (default: 20, max: 100) — Items per page.
- Response includes `meta.total` and `meta.totalPages`.

17.6 Error Codes

Table 17.2: API error codes.

HTTP	Code	Description
400	`BAD_REQUEST`	Invalid parameters
401	`UNAUTHORIZED`	Missing or invalid API key
403	`FORBIDDEN`	Insufficient scope
404	`NOT_FOUND`	Resource not found
429	`RATE_LIMITED`	Rate limit exceeded
500	`INTERNAL_ERROR`	Server error

17.7 Internal API Endpoints

Beyond the public v1 API, the server exposes internal endpoints for the frontend:

`POST /api/chat` Core streaming chat endpoint (SSE).

`POST /api/speech-to-text` Voice transcription (ElevenLabs Scribe v2).

`POST /api/upload-data` Data file upload (CSV, Excel, Parquet, JSON, up to 100MB).

`POST /api/share` Create shared report.

`GET /api/share/:shareId` Retrieve shared report.

`POST /api/generate-pdf` Markdown to PDF (Puppeteer).

`POST /api/generate-html` Markdown to HTML.

`POST /api/generate-docx` Markdown to DOCX (Pandoc).

`GET /api/download/:filename` File download from downloads directory.

`GET /api/platform/stats` Platform statistics.

`GET /api/journal/papers` Journal paper listing.

`GET /api/repository` Project repository browser.

`GET /api/live-sessions` Active session listing.

`GET /api/sessions/:sessionId/stream` SSE reconnection for live monitoring.

17.7.1 Admin Endpoints

Protected by admin session authentication:

- Pipeline management: trigger, abort, resume, cleanup.
- Scheduler configuration: enable/disable, update settings.
- Paper management: visibility, domain reclassification, LaTeX backfill.
- Peer review: trigger review, view results.
- System: cost tracking, donations, site config.

Appendices

Complete Tool Catalog

This appendix provides a comprehensive catalog of all 213 Claude-callable tools available in AutoScholar, organized by category. Each entry lists the tool name, a brief description, and the external service or module it invokes.

A.1 Financial Tools (Part 1 — Core)

Table A.1: Financial tools — Core (55+ tools).

Tool Name	Description
`get_company_profile`	Company overview (sector, market cap, description)
`get_income_statement`	Income statement (annual/quarterly)
`get_balance_sheet`	Balance sheet (annual/quarterly)
`get_cash_flow`	Cash flow statement (annual/quarterly)
`get_key_metrics`	Key financial metrics (PE, PB, EV/EBITDA, ...)
`get_financial_ratios`	Profitability, liquidity, leverage ratios
`get_stock_quote`	Real-time stock quote
`get_historical_price`	Historical daily OHLCV
`search_companies`	Search by name or ticker
`get_stock_peers`	Related companies
`get_financial_news`	Company-specific news
`get_rsi`	Relative Strength Index
`get_macd`	Moving Average Convergence Divergence
`get_ema`	Exponential Moving Average
`get_sma`	Simple Moving Average
`get_adx`	Average Directional Index
`get_williams_r`	Williams %R oscillator
`get_cci`	Commodity Channel Index
`get_stochastic`	Stochastic oscillator
`calculate_options_price`	Black-Scholes pricing with Greeks
`get_economic_calendar`	Upcoming economic events
`get_treasury_rates`	US Treasury yield curve
`get_economic_indicator`	Economic indicator data
`get_insider_trading`	Corporate insider transactions

Tool Name	Description
get_earnings_calendar	Upcoming earnings dates
get_analyst_estimates	EPS/revenue estimates
get_forex_quote	Real-time forex rates
get_commodity_quotes	Commodity prices
get_cot_report	Commitment of Traders report
get_congressional_trading	Congressional stock trades
get_senate_trading	Senate member trades
get_esg_score	ESG ratings
get_social_sentiment	Social media sentiment
get_stock_screener	Multi-criteria stock screening
get_etf_holdings	ETF constituent holdings
get_financial_growth	Revenue/earnings growth rates
get_company_outlook	Comprehensive company overview

A.2 Financial Tools (Part 2 — Market)

Table A.2: Financial tools — Market data, options, discovery.

Tool Name	Description
get_stock_news_sentiment	News with sentiment scores
get_crypto_quote	Cryptocurrency prices
get_crypto_list	Available cryptocurrencies
get_forex_historical	Historical forex rates
get_sec_filings	SEC filing documents
get_earnings_call_transcript	Earnings call text
get_option_chain	Options chain (Financial Data API)
get_option_prices	Individual option pricing
get_option_greeks	Option Greeks (δ, γ, θ, ν, ρ)
download_fmp_data	Export any FMP data to CSV/XLSX/JSON
get_eodhd_eod	EODHD end-of-day prices (back to IPO)
get_eodhd_intraday	EODHD intraday bars (1m/5m/1h)
get_eodhd_options	EODHD US options with Greeks
search_symbols	Symbol search (multiple methods)
get_stock_list	Full stock listing
get_etf_list	Full ETF listing
batch_quote_*	Batch quotes (stocks, ETFs, mutual funds, commodities, crypto, forex, indices)
get_delisted_companies	Historical delisted companies
get_employee_count	Company employee data
get_shares_float	Public float data
get_mergers_acquisitions	M&A transactions
get_executive_compensation	Executive pay data

A.3 Financial Tools (Parts 3–4 — Extended & Advanced)

Additional tools covering TTM financial statements, Altman Z-Score, Piotroski F-Score, revenue segmentation, Form 13F institutional ownership, earnings transcripts, sector/industry performance, advanced SEC filings, index constituents (S&P 500, NASDAQ, Dow Jones with historical changes), DCF valuation models, ESG benchmarks, crowdfunding, equity offerings, and market movers (gainers, losers, most active).

Due to the extensive nature of these tools (∼100 additional entries), refer to `server/services/claude/t` *and* `financialTools-advanced.ts` *for complete specifications.*

A.4 Academic Tools

Table A.3: Academic research tools.

Tool Name	Description
`academic_search`	Single-source academic search
`academic_literature_revie`	Structured literature review
`semantic_scholar_search`	Semantic Scholar API search
`arxiv_search`	arXiv preprint search
`crossref_lookup`	CrossRef DOI and metadata lookup
`openalex_search`	OpenAlex bibliographic search
`academic_search_multi`	Parallel search across all 5 databases
`literature_gap_finder`	7-step gap discovery pipeline
`web_dataset_collector`	SWDES v3 large-scale web data extraction

A.5 Web Tools

Table A.4: Web search, scraping, and extraction tools.

Tool Name	Description
`web_search_exa`	Exa semantic web search
`get_contents_exa`	Exa URL content extraction
`firecrawl_scrape`	Single-page scraping (PDF support)
`firecrawl_search`	Web search with auto-scraping
`firecrawl_crawl`	Multi-page crawling (async)
`firecrawl_extract`	AI-powered structured extraction
`firecrawl_map`	URL discovery/mapping
`firecrawl_agent`	Autonomous AI extraction agent
`firecrawl_batch_scrape`	Parallel multi-URL scraping
`tavily_search`	AI-enhanced web search
`tavily_extract`	URL content extraction
`tavily_crawl`	Graph-based multi-page crawling
`tavily_map`	Site structure mapping
`tavily_research`	Autonomous research agent

A.6 Macroeconomic Data Tools

Table A.5: Macroeconomic data tools (all free, no API key).

Tool Name	Description
`get_wb_indicator_data`	World Bank indicator data (16K+ indicators)
`get_wb_countries`	WB country metadata
`search_wb_indicators`	WB indicator keyword search
`get_wb_topics`	WB topic categories
`get_wb_regions`	WB geographic regions
`get_wb_income_levels`	WB income level classifications
`get_wb_sources`	WB data sources
`get_imf_indicator_data`	IMF DataMapper indicators
`get_imf_indicators`	IMF indicator metadata
`get_imf_countries`	IMF country list
`get_imf_dataflows`	IMF SDMX dataflows
`get_imf_sdmx_data`	IMF SDMX multi-dimensional queries
`get_imf_data_structure`	IMF dataset structure inspection
`get_imf_codelist`	IMF codelist retrieval
`get_oecd_data`	OECD data query
`search_oecd_dataflows`	OECD dataflow search
`get_oecd_dataflow_structure`	OECD dimension structure
`get_oecd_dataflows`	OECD dataflow listing

A.7 Python Tools

Table A.6: Python execution and repair tools.

Tool Name	Description
`execute_custom_python_analysis`	Run arbitrary Python with 17 libraries, 5 API clients, 4 embedded DBs
`read_python_script`	Read a previously saved script by ID
`apply_patch`	Apply a unified diff or search-replace patch to a script
`run_python_script`	Re-execute a (possibly patched) script
`inspect_python_logs`	View structured error logs for a script

A.8 Research Paper Tools

Table A.7: Research paper generation and compilation tools.

Tool Name	Description
`generate_research_paper`	Single-call paper generation (legacy)
`write_paper_section`	Write individual LaTeX section
`compile_research_paper`	Assemble sections and compile PDF

`read_latex_source`	Read compiled LaTeX source
`inspect_latex_errors`	Parse LaTeX compilation log
`patch_latex`	Apply diff to LaTeX source
`recompile_latex`	Re-run pdflatex + bibtex
`install_python_package`	Install a Python package via pip

A.9 Polymarket Tools

Table A.8: Prediction market tools (all public, no auth).

Tool Name	Description
`get_polymarket_events`	List prediction market events
`get_polymarket_markets`	List individual markets
`search_polymarket`	Keyword search across markets
`get_polymarket_price_history`	Historical token prices
`get_polymarket_order_book`	Full order book (bids/asks)
`get_polymarket_tags`	Category tags
`get_polymarket_trades`	Recent trade history
`get_polymarket_leaderboard`	Trader leaderboard

B 1

Prompt Bank Specification

This appendix documents the structure and content of AutoScholar's prompt bank, which drives autonomous paper generation with 1,000+ deterministically generated research prompts.

B.1 Construction Algorithm

The prompt bank is built at import time by the function `buildPromptBank()` in `server/services/promp`
The algorithm interleaves three dimensions:

Algorithm 7: Prompt Bank Construction

Input: Domains $\mathcal{D}$ (24), Methods $\mathcal{M}$ (24), Topics $\mathcal{T}$ (14 per domain), Compatibility $\mathcal{C}$
Output: Prompt bank $\mathcal{P}$ with ≤ 1500 entries

$\mathcal{P} \leftarrow []$;
$v \leftarrow 0$;
// variation index
while $|\mathcal{P}| < 1500$ ***and*** $v < 24$ **do**
  foreach *topic index* $t \in [0, 13]$ **do**
    foreach *domain* $d \in \mathcal{D}$ **do**
      methods $\leftarrow \mathcal{C}[d]$;
      $m \leftarrow$ methods$[v \bmod |\text{methods}|]$;
      topic $\leftarrow \mathcal{T}[d][t]$;
      prompt $\leftarrow$ formatPrompt(d, m, topic);
      $\mathcal{P}$.push$(\{d, m, \text{topic}, \text{prompt}\})$;
    end
  end
  $v \leftarrow v + 1$;
end
return $\mathcal{P}$;

The iteration order (variation → topic → domain) ensures that consecutive prompts always switch domains, providing maximum topic diversity for sequential selection.

B.2 Research Domains

Table B.1: 24 research domains.

#	Domain
1	Market Microstructure
2	Volatility Modeling
3	Asset Allocation & Portfolio Management
4	Machine Learning in Finance
5	Macro-Finance
6	Cryptocurrency & DeFi
7	ESG & Sustainable Finance
8	Market Anomalies & Factor Investing
9	Financial Econometrics
10	Systemic Risk & Financial Stability
11	Behavioral Finance
12	Derivatives & Options
13	Credit Risk & Fixed Income
14	Monetary Policy & Central Banking
15	Banking & Financial Intermediation
16	Commodities & Energy Finance
17	Emerging Markets
18	Fintech & Digital Finance
19	Statistical Inference
20	Machine Learning Prediction
21	Deep Learning & Neural Networks
22	Applied Statistics
23	Supervised & Unsupervised Learning
24	AI for Scientific Discovery

B.3 Methodological Approaches

Table B.2: 24 methodological approaches.

#	Method
1	Panel Regression with Fixed Effects
2	Generalized Method of Moments (GMM)
3	Event Study Methodology
4	Vector Autoregression (VAR/VECM)
5	DCC-GARCH Multivariate Volatility
6	Difference-in-Differences (DiD)
7	Regression Discontinuity Design (RDD)
8	Instrumental Variables (2SLS/IV)
9	Quantile Regression
10	Stochastic Volatility Models
11	Copula-Based Dependence Modeling
12	Propensity Score Matching
13	Fama-MacBeth Two-Stage Regression

#	Method
14	Structural Equation Modeling
15	Bayesian Estimation
16	High-Frequency Econometrics
17	Affine Term Structure Models
18	Dynamic Factor Models
19	Bootstrap & Permutation Testing
20	Random Forest & Gradient Boosting
21	LSTM & Transformer Networks
22	Non-Parametric Density Estimation
23	K-Means & Hierarchical Clustering
24	Reinforcement Learning

B.4 Domain–Method Compatibility Matrix

Not all domain–method combinations are meaningful. The compatibility matrix (`prompt-bank/domains.` maps each domain to its valid methods. For example:

- **Market Microstructure:** Panel FE, GMM, Event Study, High-Frequency Econometrics, Quantile Regression, ...
- **Machine Learning in Finance:** Random Forest, LSTM/Transformer, K-Means, Reinforcement Learning, ...
- **Statistical Inference:** Bootstrap, Non-Parametric Density, Bayesian Estimation, Quantile Regression, ...

Finance domains are primarily compatible with econometric methods, while statistics/ML/AI domains are compatible with their respective method families. Cross-pollination is allowed where meaningful (e.g., Bayesian methods for finance, panel regression for ML comparisons).

B.5 Topic Examples

Each domain has 14 specific research topics (`prompt-bank/topics.ts`, 415 lines). Examples:

B.5.1 Market Microstructure Topics

1. Intraday volatility patterns and market maker behavior
2. High-frequency trading impact on price discovery
3. Dark pool trading volume and market quality
4. Bid-ask spread determinants across market regimes
5. Flash crash dynamics and circuit breaker effectiveness
6. Limit order book dynamics and information asymmetry
7. Market fragmentation effects on execution quality

8. Algorithmic trading and systemic risk propagation
9. Price impact of large institutional trades
10. Cross-market information transmission at the tick level
11. Maker-taker fee models and liquidity provision incentives
12. Odd-lot trading and retail investor behavior
13. Market microstructure during stress events
14. Co-location advantages and speed competition in equity markets

B.5.2 ESG & Sustainable Finance Topics

1. ESG score divergence across rating agencies
2. Green bond pricing premiums and issuer characteristics
3. Carbon risk and asset pricing
4. ESG momentum and factor investing
5. Corporate governance and firm value
6. Climate transition risk in credit markets
7. Social responsibility and cost of capital
8. Environmental regulation and competitive advantage
9. ESG integration in portfolio optimization
10. Greenwashing detection in corporate disclosures
11. Biodiversity risk and financial returns
12. Impact investing performance measurement
13. Stranded assets and energy transition
14. ESG controversies and stock price reactions

B.6 Prompt Format

Each generated prompt follows the structure:

```
Write a research paper on [topic] using [method] methodology. The
paper should focus on the [domain] domain and provide rigorous empirical
evidence using real financial data.
```

The prompt upgrader (`upgradeResearchPrompt()`) then expands this into a detailed 400–700 word research specification before it reaches Claude.

C 1

Database Schema Reference

This appendix provides the complete database schema for AutoScholar, covering all 20+ tables with column definitions, types, constraints, and relationships. The schema is defined in Drizzle ORM format with both SQLite (`shared/schema-sqlite.ts`) and PostgreSQL (`shared/schema.ts`) variants.

C.1 User Tables

C.1.1 users

Table C.1: `users` table schema.

Column	Type	Constraint	Description
`id`	integer	PK, auto-increment	Unique identifier
`username`	text	unique, not null	Login username
`email`	text		Email address
`passwordHash`	text		SHA-256 + salt hash
`displayName`	text		Public display name
`institution`	text		University or organization
`department`	text		Department
`orcid`	text		ORCID identifier
`apiKey`	text		XOR-obfuscated Anthropic key
`role`	text	default 'user'	Role (user/admin)
`token`	text		Session token
`createdAt`	timestamp	not null	Creation time
`updatedAt`	timestamp		Last update time

C.1.2 emailVerificationTokens

`id`	integer	PK	Unique identifier
`userId`	integer	FK → users	Owning user
`token`	text	not null	Verification token
`expiresAt`	timestamp	not null	Expiry time
`createdAt`	timestamp	not null	Creation time

C.1.3 passwordResetTokens

Same structure as `emailVerificationTokens`.

C.2 Content Tables

C.2.1 messages

Table C.2: `messages` table schema.

Column	Type	Constraint	Description
`id`	integer	PK	Unique identifier
`sessionId`	text	not null	Conversation session ID
`role`	text	not null	'user' or 'assistant'
`content`	text	not null	Message content
`sources`	text		JSON-serialized sources
`createdAt`	timestamp	not null	Creation time

C.2.2 sharedReports

Table C.3: `sharedReports` table schema.

Column	Type	Constraint	Description
`id`	integer	PK	Unique identifier
`shareId`	text	unique, not null	Public share identifier
`question`	text	not null	Research question
`answer`	text	not null	AI-generated answer
`sources`	text		JSON sources array
`toolResults`	text		JSON tool results
`pythonFigures`	text		JSON figure data
`userId`	integer	FK → users	Creating user
`createdAt`	timestamp	not null	Creation time

C.3 Research Output Tables

C.3.1 publishedPapers

Table C.4: `publishedPapers` table schema.

Column	Type	Constraint	Description
`id`	integer	PK	Unique identifier
`title`	text	not null	Paper title
`abstract`	text		Abstract text

Column	Type	Constraint	Description
authors	text		Author (Euler agent)
domain	text		Auto-classified domain
template	text		article / working_paper / econometrica
latexSections	text		JSON: section name → LaTeX
bibtexContent	text		Full BibTeX string
figureCount	integer	default 0	Number of figures
wordCount	integer	default 0	Total word count
sectionCount	integer	default 0	Number of sections
latexLineCount	integer	default 0	Total LaTeX lines
peerReviewScore	integer		Popper score (0–100)
peerReviewRecommendation	text		accept/minor/major/reject
projectZipPath	text		Path to project ZIP
asid	text	unique	AutoScholar Identifier
sessionId	text		Associated session (for cost)
isVisible	boolean	default true	Publication visibility
createdAt	timestamp	not null	Publication time
updatedAt	timestamp		Last update time

C.3.2 peerReviews

Table C.5: peerReviews table schema.

Column	Type	Constraint	Description
id	integer	PK	Unique identifier
paperId	integer	FK → publishedPapers	Reviewed paper
overallScore	integer	not null	Overall score (0–100)
recommendation	text	not null	Publication recommendation
strengths	text		JSON strengths array
weaknesses	text		JSON weaknesses array
detailedScores	text		JSON: 6-dimension scores
sectionReviews	text		JSON: per-section feedback
fullReport	text		Complete review text
createdAt	timestamp	not null	Review time

C.4 Pipeline Management Tables

C.4.1 paperRuns

Table C.6: `paperRuns` table schema.

Column	Type	Constraint	Description
`id`	integer	PK	Unique identifier
`status`	text	not null	queued/researching/analyzing/drafting/validating/published/rejected/failed
`prompt`	text		Research prompt used
`sessionId`	text		Associated conversation session
`paperId`	integer		Published paper (if successful)
`heartbeat`	timestamp		Last heartbeat time
`popperScore`	integer		Peer review score
`governorState`	text		Serialized JSON state
`errorMessage`	text		Failure reason
`startedAt`	timestamp		Pipeline start time
`completedAt`	timestamp		Pipeline completion time
`createdAt`	timestamp	not null	Record creation time

C.4.2 `governorDecisions`

`id`	integer	PK	Unique identifier
`runId`	integer	FK → paperRuns	Parent pipeline run
`stage`	text		Pipeline stage
`trigger`	text		What triggered the decision
`action`	text		Action taken
`reasoning`	text		Explanation
`confidence`	real		Confidence at decision time
`createdAt`	timestamp		Decision timestamp

C.4.3 `asidCounters`

`id`	integer	PK	Unique identifier
`year`	integer	unique, not null	Calendar year
`counter`	integer	not null, default 0	Next sequence number

C.5 Analytics Tables

C.5.1 `conversationSessions`

Table C.7: `conversationSessions` table schema.

Column	Type	Constraint	Description
`id`	integer	PK	Unique identifier
`userId`	integer	FK → users	Owning user
`sessionId`	text	unique, not null	UUID session identifier
`title`	text		Session title
`messageCount`	integer	default 0	Number of messages
`totalTokens`	integer	default 0	Total tokens consumed
`totalCost`	real	default 0	Total API cost (USD)
`createdAt`	timestamp	not null	Creation time
`updatedAt`	timestamp		Last update time

C.5.2 `activeUsers`

Heartbeat-based active user tracking with 2-minute activity window.

C.5.3 `analyticsMetrics`

Time-series platform metrics with configurable period (minute/hour/day).

C.5.4 `requestLogs`

API request audit log with method, path, status, duration, user agent.

C.6 Administrative Tables

C.6.1 `siteConfig`

Key-value configuration store with upsert semantics.

C.6.2 `donations`

Crypto and fiat donation records with status tracking.

C.6.3 `apiKeys`

Table C.8: `apiKeys` table schema.

Column	Type	Constraint	Description
`id`	integer	PK	Unique identifier
`keyHash`	text	unique, not null	SHA-256 hash of raw key
`keyPrefix`	text		First 8 chars for display
`label`	text		User-provided label

`scopes`	text		JSON array of scopes
`rateLimit`	integer	default 100	Requests per hour
`usageCount`	integer	default 0	Lifetime usage counter
`lastUsedAt`	timestamp		Last usage time
`userId`	text		Creating user ID
`isRevoked`	boolean	default false	Revocation status
`createdAt`	timestamp	not null	Creation time

C.7 Schema Differences: SQLite vs PostgreSQL

Table C.9: Key differences between SQLite and PostgreSQL schemas.

Feature	**SQLite**	**PostgreSQL**
Timestamps	`integer` (Unix epoch, mode: 'timestamp')	`timestamp` (native)
Booleans	`integer` (mode: 'boolean')	`boolean` (native)
Vectors	`text` (JSON string)	`vector(1536)` (pgvector)
Serial IDs	`integer` (auto-increment)	`serial`
Text types	`text` (all sizes)	`varchar(N)` / `text`
API Keys table	Included (`apiKeys`)	Not in schema.ts (added at runtime)
Write concurrency	Single-writer (WAL mode)	Full MVCC